

基于信息集分层的微网电力调控策略：因果预测、风险定价与滚动优化

摘要

本文以**信息集边界**为主线，将四个微网调控问题统一到母线能量平衡、储能状态递推、容量 / 功率边界与弃光约束构成的公共模型中；各问仅改变可用信息与结算规则。每次滚动决策都继承上一执行块的实测储电量，未启用节点则延续最近有效计划，从而严格保持时序一致性。

问题一为确定性日前调度。以全天购电费最小为目标、首末储电量均为 6000 kWh 为闭环约束建立线性规划，并用“费用—吞吐量—弃光量”三层词典序目标消除退化最优解。最优全天购电量 59482.70 kWh，购电费 35126.95 元，较无储能基准的 48052.05 元节省 12925.10 元（26.8981%）。

问题二采用只依赖历史观测的负载—光伏因果预测和逐日确定性等价优化。5 倍紧急电价在孤立单时段下对应 4 : 1 非对称损失及光伏输出 0.2 分位启发；全年模型通过样本外费用在多个残差分位候选中在线选择，并用“先撤充电、再增放电、最后紧急购电”的实时反馈执行。2 月至 12 月总费用为 14773804.78 元，紧急购电 348036.96 kWh。对照实验表明，把当天真实负载误作已知会低估 1497816.65 元（10.14%）。

问题三按题意采用“计划费 + 调整费 + 紧急费”的叠加结算，并用 0-1 互斥约束禁止同时充放电。仅 0:00 决策时费用为 15731164.40 元，依次加入 6:00、12:00、18:00 更新后降至 14510735.12、14213145.74、13917403.19 元，三节点均有正收益，累计节省 1813761.21 元。配对控制把其中 1225632.49 元归因于负载观测刷新、588128.71 元归因于新光伏预报，避免把综合节点收益误称为纯光伏价值。

问题四按题面直接代入附件 4 电价重算，Q4-2 与 Q4-3 主结果分别为 15523884.83 元和 14589632.38 元，滚动更新节省 934252.45 元。另设因果价格扩展，费用分别为 15590096.88 元和 14722175.07 元；两种价格信息情形下滚动更新均有效。

全部指定日期表均由最终结果工作簿自动生成。物理残差、信息泄漏 *mutation*、结算分解、计划持续性与回归测试全部通过；主方案同时充放电量与费用分解回代差均低于 10^{-8} 。

关键词：微网经济调度；因果预测；滚动优化；混合整数规划；信息集

一、问题重述

1.1 问题背景

研究对象是一个经公共连接点接入外网的小区级能源系统。光伏出力具有间歇性，居民用电具有日周期，电池则把不同时刻的能量联系起来；三者共同决定每个 10 min 时段应从外网买入多少电。调度既要逐时满足用电需求，又要利用电池在低价或光伏富余时吸收能量、在缺电或高价时释放能量，因而本质上是带状态递推、预测误差和偏差结算的动态优化问题。

本问题的物理对象由三部分组成：光伏发电单元、储能设备与小区负载，并通过公共连接点与外网相连。附录 1 给定储能参数：最大容量 12000 kWh，最大充放电功率 5000 kW，2025 年 1 月 1 日 0:00 储电量为 6000 kWh，运行期储电量必须保持在 1200 ~ 10800 kWh 之间，充放电效率均为 90%。附件数据的时间分辨率均为 10 min，即一天可分为 144 个调度时段。

问题的核心困难在于信息的可获得性：微网必须在决策时刻仅依据当时已知的信息制定策略，而负载、光伏与实际电价在一天内是逐步揭晓的。因此四个问题的差异并不在物理约束，而在于决策时刻能看到什么信息，以及计划与实际不一致时如何结算。

1.2 问题提出

问题 1. 在典型日电价、负载和光伏预测均给定时，制定 0:00 日前计划，并满足首末储电量相同。论文须按题面表 1、表 2 给出六个指定时段、全天购电量与费用，以及六个 4 h 分块的充放电量；完整结果写入 result1.xlsx。

问题 2. 在负载和光伏逐日变化、计划缺口按 5 倍电价紧急补购时，制定每日 0:00 计划。论文须对 2025-03-20、06-21、09-23、12-21 按表 1-3 报告购电、储能和紧急购电结果；完整结果写入 result2.xlsx。

问题 3. 每日四个发布时刻可获得未来 24 h 光伏预报，并允许调整 0:00 基准购电量。需要写清计划费、上调费、下调违约金与紧急购电费，判断新增预报是否值得采用；同样对四个指定日期给出表 1-3，并写入 result3.xlsx。

问题 4. 将附件 4 的逐时电价代入问题二、三重新求解，给出相应年度指标与四个指定日期结果，分别写入 result4-2.xlsx 和 result4-3.xlsx。

1.3 研究现状综述

微网经济调度问题在数学上通常被建模为以购电费或综合运行成本最小为目标的线性规划或混合整数规划，外网购电、分布式电源消纳与储能充放电被统一纳入同一组能量平衡约束中 [1, 2]。确定性模型适用于负载、电价与光伏均已知的日前场景，其优

点是约束结构清晰、求解稳定可靠。

当光伏出力存在预测误差时，主流处理方式有三类：一是情景法 / 两阶段随机规划，用有限个历史或生成情景刻画不确定性，第一阶段确定与情景无关的日前计划，第二阶段按情景确定执行量 [3, 4]；二是鲁棒优化，在最坏情景下保证可行性，代价是结果偏保守 [5]；三是滚动时域优化（模型预测控制），在新信息到达后仅重优化剩余时域并只执行近期决策，天然适配“预报分批发布”的决策结构 [6, 7]。含光伏与储能的日前调度研究普遍指出，储能的价值高度依赖预测精度与调度时域长度 [8]，而储能的运行成本又与其循环深度密切相关 [9]。

针对“预测值应偏向哪一侧”的问题，报童模型给出了经典答案：当高估与低估的边际损失不对称时，最优的点预测不是条件均值，而是某个分位数 [10]。这一结论在电力系统中已被用于发电商的投标策略与偏差电量定价 [11]。在预测方法层面，基于数值天气预报的机器学习模型是目前光伏功率预测的主流 [12]；在市场机制层面，实时电价与需求响应的耦合会显著改变用户的购电行为 [13]。

现有研究多聚焦于算法本身，而对信息集边界与结算口径的刻画往往不够严格：若在日前优化中直接使用当天真实负载，会把不可实现的信息优势混入结果；若比较策略时同时改变预测方法、约束和费用口径，差异也无法归因。本文据此把信息集边界作为建模原则，并用单因素消融与配对控制支持归因；求解则利用线性 / 混合整数线性结构，不依赖随机性较强的元启发式算法。

二、问题分析

2.1 总体思路

四个问题共享同一套物理对象与同一组物理约束，差别只在两处：决策时刻能获得哪些信息，以及计划与实际不一致时如何结算。因此本文采用“一个物理内核+四条信息集主线”的建模框架：先把母线能量平衡、储能状态转移、容量与功率限值、弃光上界写成公共约束组，再对每一问替换相应费用项、扩展决策变量，并重新界定信息集。这样既保持四问口径一致，也能把结果差异归因到信息更新与结算规则本身。

具体地，问题 1 是完全信息下的确定性经济调度，用于验证物理约束与求解器；问题 2 在完全无法获得当天预报的极端信息集下，仅用历史观测构造因果预测，并把 5 倍紧急电价转化为非对称损失；问题 3 引入官方多时刻预报，形成“0:00 基准计划 → 每 6 h 滚动调整 → 实际执行 → 状态递推”的闭环；问题 4 只把电价序列换成逐日逐时的波动电价，用于检验前两类策略在价格波动下的稳健性。

整体流程为“数据对齐与单位换算 $\rightarrow$ 公共物理模型 $\rightarrow$ 因果预测与滚动重优化 $\rightarrow$ 主口径求解 $\rightarrow$ 消融、回代与提交文件核验”，各环节共享同一数据索引和费用定义。

2.2 各问的具体分析

问题一的分析

题目给定附件 1 的电价、负载与光伏发电预测，且“每天的电价和小区负载相同”，因此属于确定性优化。富余光伏必须经储能充电或弃光消纳，故公共模型保留弃光变量；本问的最优解实际实现零弃光。题目还要求 0:00 与 24:00 储电量相同，该闭环条件必须作为硬约束显式写入。

问题二的分析

本问的信息集最弱：题目没有给出任何当天预报，只能依赖已经发生的历史观测。这与问题一有本质区别——若仍沿用当天真实数据，紧急购电将几乎不会触发，模型退化为完美信息基准。因此必须构造只使用历史数据的因果预测器，并让预测误差真实地转化为远高于计划成本的紧急购电费用。5 倍电价使“高估光伏”的局部边际损失是“低估光伏”的 4 倍，据此可在单时段报童损失下给出保守分位点；全年调度则采用因果点预测的确定性等价模型。计划在 0:00 锁定后，实际偏差由储能进行 10 min 级实时缓冲。

问题三的分析

本问引入 6:00、12:00、18:00 三个新增预报时刻，题目据此询问“是否需要引入”。本文把“允许更新决策的节点集合”作为消融变量，构造 S_0 （仅 0:00） $\rightarrow S_1$ （+6:00） $\rightarrow S_2$ （+12:00） $\rightarrow S_3$ （+18:00）四组对照；节点未启用时，最近一次有效计划持续执行，绝不回退到 0:00 基准计划。由于启用节点会同时获得已实现负载观测与新发布光伏预报，主消融量度的是**整个决策更新节点的价值**；再用配对控制实验分别识别负载刷新与光伏预报刷新的贡献。题目将计划费、调整费与紧急购电费并列相加，故叠加结算作为主口径，差额结算仅作敏感性对照，两者分别重新优化。

问题四的分析

本问按题面字面要求，直接把附件 4 给定的逐日实时电价代入问题二、三重新计算，并将这一口径作为主答案与结果工作簿来源。附件 4 的电价在日内与日间均有明显波动（最低 0.0076 元/kWh，最高 1.7936 元/kWh），峰谷价差扩大了储能套利空间。考虑到实际市场中完整日内价格的发布时间可能受交易规则限制，本文另设只使用已发生价格的因果预测情形作为现实性扩展；二者之差用于说明价格预知信息的价值，但不替代题面主结果。

2.3 求解框架

问题一、二及差额结算对照为线性规划；问题三和问题四的叠加结算主模型加入充放电互斥的 0-1 变量，形成混合整数线性规划。二者均由 HiGHS 求解。连续模型再以“真实费用—储能吞吐量—弃光量”词典序目标排除无经济意义的退化最优解；叠加模型由互斥约束直接禁止同一时段充放电。所有电量按 10 min 时段能量 (kWh) 核算，功率与能量换算系数为 $\Delta t = 1/6$ h。

三、模型假设

为突出购电、光伏与储能之间的主要矛盾，并保证四问的信息边界严格一致，本文作如下假设。每条假设都将在相应模型中显式使用。

1. **调度周期与时间尺度：**以 10 min 为一个调度时段，一天划分为 $T = 144$ 个时段，单时段长度 $\Delta t = 1/6$ h。附件中标注 0:00+1 的列即当天 24:00。
2. **不允许向外部电网售电：**题目仅给出微网从外网购电的机制，未给出反向售电的交易规则，因此外网购电量恒为非负，光伏富余只能用于储能充电或弃光。
3. **储能建模的简化：**储能充放电效率在全年内恒为 $\eta_c = \eta_d = 0.9$ （口径敏感性见 7.5.2 节），不计自放电、容量衰减、循环寿命成本与启停损耗；叠加结算主模型用 0-1 变量严格禁止同一时段充放电，其余连续模型由词典序目标排除退化解。
4. **储能功率约束按能量折算：**最大充放电功率 5000 kW 对应单时段最大充放电量 $5000\Delta t = 833.33$ kWh。
5. **初始储电量：**2025 年 1 月 1 日 0:00 储电量为 6000 kWh；1 月 31 天作为储能状态与预测误差的预热期，正式结果自 2 月 1 日起输出，这既与官方结果模板的日期范围一致，也避免冷启动误差污染统计。
6. **负载预测口径：**题目只为光伏提供了专门预报数据，未给出负载预报。因此本文用历史实际负载构造因果预测，方法族为持续法、7/14/30 日滚动均值、14/30 日线性加权均值与同星期均值，并在历史执行块上滚动在线选型，选型标准为平均绝对误差。
7. **附件 1 作为冷启动先验：**1 月 1 日之前无任何历史观测，此时以附件 1 给出的典型日负载与光伏预测曲线作为先验基预测。该先验仅在历史观测为空的当天使用，一旦有了至少一天观测便自动失效。
8. **实时储能反馈：**计划锁定后，日内允许储能依据当前时刻已经发生的真实光伏与当前储电量做 10 min 级调整，调整顺序为“先撤充电、再增放电、最后紧急购电”，不得借助任何未来信息。
9. **终端储电量策略：**题目仅对问题一明确要求 $S_{0:00} = S_{24:00}$ 。本文主模型在问题二、三

的计划层同样采用“计划终端储电量等于当日实际初始储电量”的日循环口径（记为 `daily_cycle`），以保证储能长期可持续运行；同时给出不做终端约束（`free`）的敏感性结果，二者的差异在 7.5.1 节报告。

10. **问题四的价格口径：**题面主结果直接使用附件 4 给定的电价数据；因果预测仅作为交易信息受限时的扩展情形，其预测只使用决策时刻之前已经发生的价格，跨日部分由历史同刻价格外推。

四、符号说明

本文使用的主要符号及其含义如表 1 所示。表中未列出的局部符号将在正文首次出现处说明。

表 1 主要符号说明

符号	单位	含义
d, t	—	日期下标与日内 10 min 时段下标 ($t = 1, \dots, 144$)
Δt	h	单个调度时段长度, $\Delta t = 1/6$
$L_{d,t}$	kW	小区负载功率
$P_{d,t}$	kW	光伏实际发电功率
$\hat{P}_{d,t}$	kW	决策时刻可用的光伏预测功率
$\pi_{d,t}$	元/kWh	外网正常购电单价
$G_{d,t}^p$	kWh	0:00 制定的计划购电量（第一阶段变量）
$G_{d,t}^a$	kWh	执行前最后一次有效调整后的购电量（第二阶段变量）
$C_{d,t}$	kWh	母线侧进入储能的充电量
$H_{d,t}$	kWh	储能送往母线的放电量
$S_{d,t}$	kWh	第 t 时段结束时储能内部储电量（状态变量）
$W_{d,t}$	kWh	未被利用的弃光电量
$E_{d,t}$	kWh	供电不足时的紧急购电量
$u_{d,t}, v_{d,t}$	kWh	调整购电量相对计划的上调量 $(G^a - G^p)^+$ 与下调量 $(G^p - G^a)^+$
$z_{d,t}$	—	充放电互斥变量； $z = 1$ 允许充电， $z = 0$ 允许放电
η_c, η_d	—	充电效率与放电效率，均取 0.9
$S_{\min}, S_{\max}$	kWh	储电量上下限，1200 与 10800
$\bar{E}$	kWh	单时段充放电电量上限， $\bar{E} = 5000\Delta t = 833.33$
$\mathcal{I}_{d,\tau}$	—	第 d 天决策节点 τ 已经获得的信息集

五、数据预处理

附件 1 为某典型日的电价、小区负载与光伏发电预测，共 144 个时段；附件 2 给出 2025 年 1 月 1 日至 12 月 31 日每天 144 个时段的小区负载与光伏实际功率；附件 3 给出每天 0:00、6:00、12:00、18:00 四次发布的未来 24 小时整点光伏预报，共 $365 \times 4 \times 24$ 个数值；附件 4 给出 365×144 的实时电价。所有附件的时间标签均按“列 j 对应时刻 $j \times 10 \text{ min}$ ”的约定统一，不作人为平移。

原始功率单位为 kW，进入能量平衡前统一乘以 $\Delta t = 1/6 \text{ h}$ 换算为每时段电量 (kWh)；电价单位为元/kWh；弃光、紧急购电、充放电均以 kWh 表示。逐项检查表明：各附件所需区域均为有限数值，负载与光伏非负、电价为正，附件 2 与附件 3 的日期完全对齐，附件 3 每行严格按 0:00, 6:00, 12:00, 18:00 排列。全年负载电量 $40.52 \times 10^6 \text{ kWh}$ ，光伏电量 $20.25 \times 10^6 \text{ kWh}$ ，光伏高于负载的时段占 19.27%，其中 241 个时段的光伏富余功率超过 5000 kW，因此公共模型必须允许弃光。附件 1 电价的峰谷比为 3.76，而附件 4 的实时电价在 0.0076 ~ 1.7936 元/kWh 之间波动，两者的日平均电价均为 0.7662 元/kWh，使固定电价与波动电价的结果具有可比基础。

图 1 给出了数据总览。由图 1(a) 可见，典型日的光伏出力集中在 6:00 ~ 18:00，而电价的高峰出现在上午与傍晚两个时段，二者并不重合，这正是储能可以创造价值的空间；图 1(b)(c) 显示负载与光伏都存在明显的季节性：夏季负载与光伏同时走高，冬季负载抬升而光伏显著衰减；图 1(d) 表明实时电价在日内呈现规律的峰谷结构，但日间水平差异较大。

图 2 进一步量化了光伏与负载的匹配程度。净负荷（负载减光伏）在春、秋两季的中午时段大量转为负值，说明存在系统性富余；夏季光伏渗透率最高，6 ~ 8 月的中位数接近 1，即光伏出力与负载相当；冬季净负荷整体抬升，储能的价值主要体现在峰谷套利而非消纳光伏。

相关性复算表明，净负荷与负载、光伏的相关系数分别为 -0.1140、-0.8796，实时电价与日内相位的相关系数为 0.3912。净负荷受光伏变化的影响更强；附件 4 的价格同时具有日内结构与日际波动，不能用附件 1 的典型日电价代替，故问题四必须独立重算。

六、模型的建立与求解

6.1 公共调度模型

四个问题共用同一套母线侧能量流结构。本节先给出公共决策变量、目标函数与约束，后续各问只替换信息集与费用项；加入充放电互斥变量时，模型由线性规划变为混合整数线性规划。

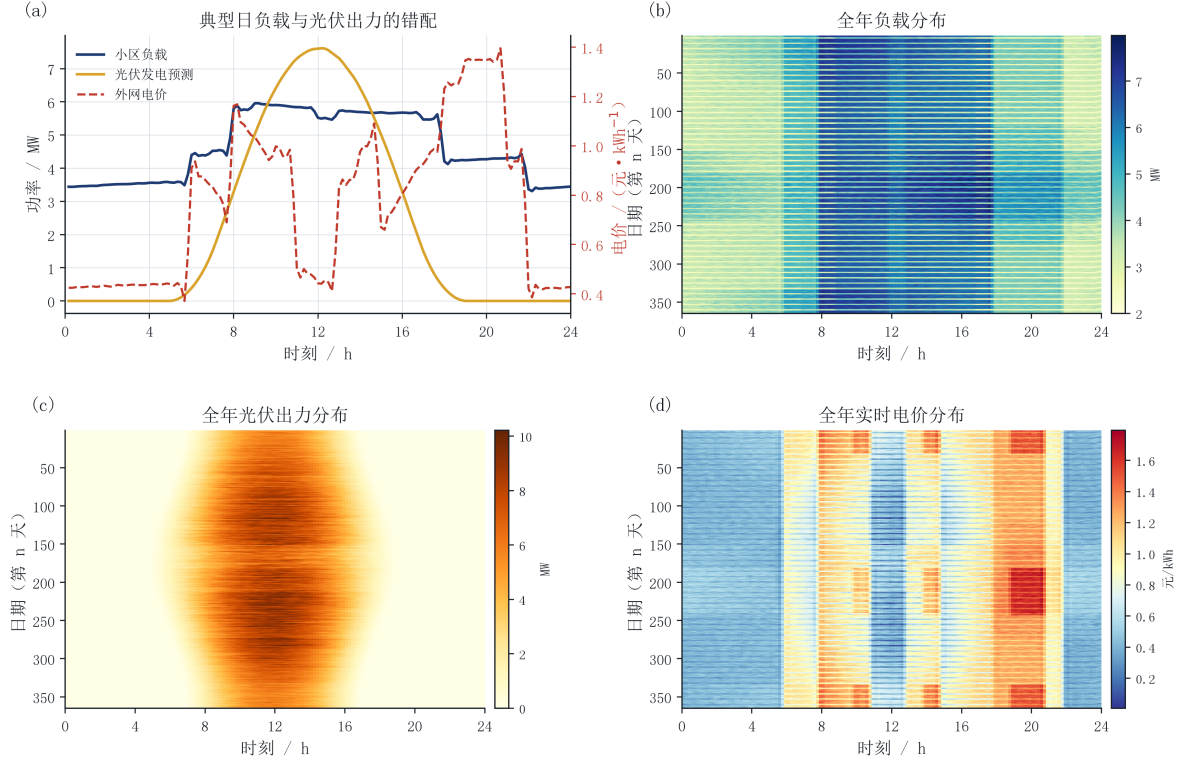


图 1 附件数据总览

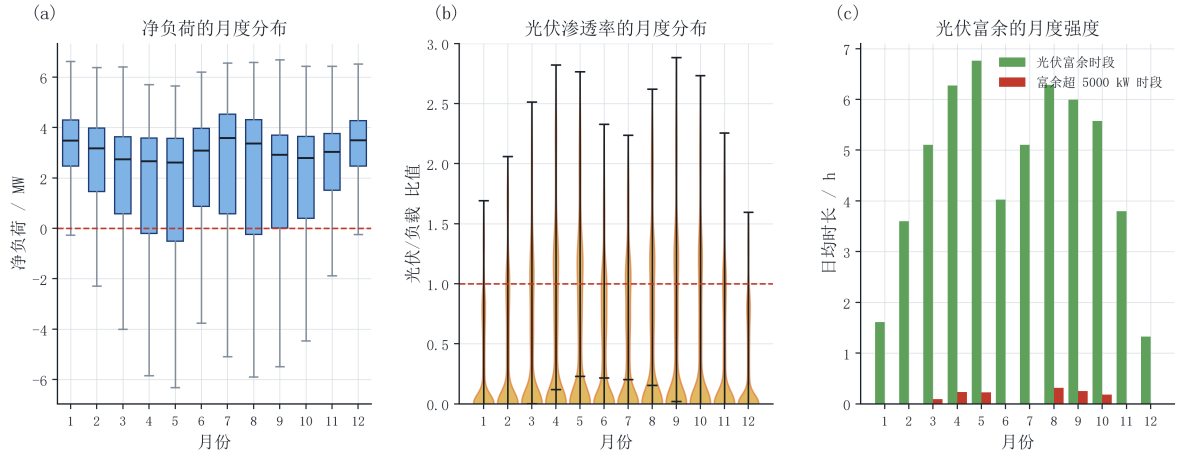


图 2 净负荷与光伏渗透的季节特征

6.1.1 决策变量

以第 d 天第 t 个 10 min 时段为基本决策单元，定义非负决策变量：

- 计划购电量 $G_{d,t}^p \geq 0$ ：0:00 制定的、全天固定不再改动的外网购电量 (kWh)；
- 调整购电量 $G_{d,t}^a \geq 0$ ：执行前最后一次有效调整后的外网购电量 (kWh)；问题一中 $G^a \equiv G^p$ ；
- 充电量 $C_{d,t} \geq 0$ 与放电量 $H_{d,t} \geq 0$ ：母线侧进入 / 流出储能的电量 (kWh)；

- 弃光电量 $W_{d,t} \geq 0$: 光伏中未被负载或储能吸收的部分 (kWh);
- 紧急购电量 $E_{d,t} \geq 0$: 实际执行时供电不足的补充量 (kWh);
- 储电量 $S_{d,t}$: 状态变量, 第 t 时段结束时储能内部电量 (kWh)。

需要特别强调的是 G^p 与 G^a 的角色差异: G^p 是在不确定性揭晓之前确定的与场景无关的变量; G^a 、 C 、 H 、 W 、 E 、 S 则是与真实场景相关的执行变量。这一“计划—执行”两阶段结构是问题二与问题三的建模基础。

6.1.2 目标函数

按题面“计划购电费用、调整相关费用和紧急购电费用相加”的字面口径, 主模型的费用写为

$$\min F = \underbrace{\sum_{d,t} \pi_{d,t} G_{d,t}^p}_{\text{计划购电费}} + \underbrace{\sum_{d,t} \pi_{d,t} (1.5u_{d,t} + 0.5v_{d,t})}_{\text{调整相关费用}} + \underbrace{\sum_{d,t} 5\pi_{d,t} E_{d,t}}_{\text{紧急购电费}}, \quad (1)$$

其中 $u = (G^a - G^p)^+$ 、 $v = (G^p - G^a)^+$ 。问题一、二没有调整交易, 令 $u = v = 0$; 问题三、四按式 (1) 作为叠加结算主口径。因子 5 来自题目“紧急购电电价是交易时刻电价的 5 倍”。差额结算只在 6.4.2 节作为口径敏感性对照。

6.1.3 约束条件

约束 1 (母线能量平衡)。这是全模型的核心守恒关系: 任一 10 min 时段内, 微网的电力来源必须等于电力消耗。

$$\underbrace{G_{d,t} + P_{d,t}\Delta t + H_{d,t} + E_{d,t}}_{\text{电力来源}} = \underbrace{L_{d,t}\Delta t + C_{d,t} + W_{d,t}}_{\text{电力消耗}}, \quad \forall d, t. \quad (2)$$

式中 $P_{d,t}\Delta t$ 为该时段可用的光伏电量, $W_{d,t}$ 为被主动舍弃的光伏电量 (弃光)。该式之所以必须写成等式而非“供电量不低于负载”, 是因为光伏富余时必须有出口, 否则模型将不可行。

约束 2 (储电量状态转移方程)。储能是唯一跨时段耦合的元件, 其状态递推本模型最关键的约束:

$$S_{d,t} = S_{d,t-1} + \eta_c C_{d,t} - \frac{H_{d,t}}{\eta_d}, \quad S_{d,0} = S_d^{\text{init}}, \quad \forall d, t = 1, \dots, 144. \quad (3)$$

式 (3) 表明: 本时段末储电量等于上一时段末储电量, 加上充电量经效率折算后真正进入电池的部分, 再减去为送出放电量而从电池内部取出的部分。 $S_{d,t}$ 是跨时段状态变量; 每个预报节点都以当时实际测得的储电量为初值, 所以后续方案随实际执行状态递推更新。

约束 3（储电量安全区间）。为避免过充过放，储电量必须始终保持在安全区间内：

$$1200 \leq S_{d,t} \leq 10800, \quad \forall d, t. \quad (4)$$

约束 4（充放电功率上限）。最大充放电功率为 5000 kW，折算到单时段电量为

$$0 \leq C_{d,t} \leq \bar{E}, \quad 0 \leq H_{d,t} \leq \bar{E}, \quad \bar{E} = 5000\Delta t = 833.33 \text{ kWh}. \quad (5)$$

约束 5（购电非负与不售电）。题目未给出微网向外网售电的机制，因此

$$G_{d,t}^p \geq 0, \quad G_{d,t}^a \geq 0, \quad E_{d,t} \geq 0. \quad (6)$$

约束 6（弃光上界）。弃光不能超过该时段实际可用的光伏电量：

$$0 \leq W_{d,t} \leq P_{d,t}\Delta t. \quad (7)$$

约束 7（问题一首末储电量相等）。题目对问题一明确要求 0:00 与 24:00 的储电量相同：

$$S_{d,144} = S_{d,0} = 6000 \text{ kWh}. \quad (8)$$

问题二、问题三的计划层采用“计划终端储电量等于当日实际初始储电量”的日循环口径（daily_cycle），其敏感性见 7.5.1 节。

约束 8（信息集与非预见性）。设 $\tau \in \{0, 6, 12, 18\}$ 为第 d 天的计划节点， $\mathcal{I}_{d,\tau}$ 为该节点已经获得的数据。节点 τ 生成的计划变量 $X_{d,t}^{(\tau)} = (G^a, C, H, W, S)_{d,t}^{(\tau)}$ 必须关于该信息集可测：

$$X_{d,t}^{(\tau)} \in \sigma(\mathcal{I}_{d,\tau}), \quad t \geq 6\tau + 1. \quad (9)$$

等价地，若两个数据场景在 $\mathcal{I}_{d,\tau}$ 上历史完全相同，则其节点 τ 生成的计划必须相同。各节点的信息边界与执行区间见表 2；当前执行时段的真实功率只进入实时反馈，不得进入节点计划器。

表 2 滚动节点的信息集与执行区间

节点	可使用的信息 $\mathcal{I}_{d,\tau}$	当次执行区间
0:00	前一日及更早实际负载、光伏、价格；0:00 发布的光伏预报；当日初始储电量	0:00–6:00
6:00	加入 0:00–6:00 已实现观测、6:00 光伏预报及实测储电量	6:00–12:00
12:00	加入 6:00–12:00 已实现观测、12:00 光伏预报及实测储电量	12:00–18:00
18:00	加入 12:00–18:00 已实现观测、18:00 光伏预报及实测储电量	18:00–24:00

该约束不是普通线性不等式, 而由程序的数据切片和预测器接口保证, 并用 **mutation** 测试验证 (见七、节)。

6.1.4 模型类型与求解

公共模型由目标式 (1)、母线平衡式 (2)、状态递推式 (3)、边界式 (4)–(7) 和非预见性条件 (9) 组成。问题一、二和差额结算对照均为线性规划; 对叠加结算主模型, 另引入互斥变量 $z_{d,t} \in \{0, 1\}$:

$$0 \leq C_{d,t} \leq \bar{E}z_{d,t}, \quad 0 \leq H_{d,t} \leq \bar{E}(1 - z_{d,t}), \quad (10)$$

从而严格保证 $C_{d,t}H_{d,t} = 0$, 得到混合整数线性规划。连续模型采用“真实费用—储能吞吐量—弃光量”三层词典序目标; 混合整数主模型直接最小化真实费用。全部模型均由 HiGHS 求解, 并在输出后逐时回代验证。

6.2 问题一: 确定性日前调度

6.2.1 模型建立

问题一中附件 1 的电价、负载与光伏预测均视为已知, $G^a \equiv G^p$, $E \equiv 0$, 且不存在调整量 ($u = v = 0$)。目标函数退化为

$$\min F_1 = \sum_{t=1}^{144} \pi_t G_t, \quad (11)$$

约束为式 (2)~(7) 加上首末储电量约束式 (8)。式 (11)~(8) 构成问题一的确定性线性规划模型。

6.2.2 求解结果

求解得到全天计划购电量 59482.70 kWh, 购电费 35126.95 元。储能充电 20740.67 kWh、放电 16799.94 kWh, 首末均为 6000 kWh, 且无弃光。禁用储能时费用为 48052.05 元, 故净节省 12925.10 元 (26.8981%)。

表 3 按题面格式给出问题一结果; 完整 144 时段明细见支撑材料 **result1.xlsx**。

由图 3 可见, 储能呈现清晰的“低价充电、高价放电”策略: 在 0:00 ~ 4:00 的低价时段和中午光伏富余时段充电, 在 4:00 ~ 8:00、10:00 ~ 12:00 及 18:00 ~ 20:00 的较高电价时段释电量; 储电量始终处于 1200 ~ 10800 kWh 的安全区间。表 4 也显示 4:00 ~ 8:00 分块以放电为主, 与最优明细一致。

表 3 问题一指定时段购电量及全天购电量与购电费

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	486.40	14:00–14:10	0.00
16:00–16:10	394.93	18:00–18:10	636.98	20:00–20:10	0.00
全天购电量/kWh	59482.70	全天购电费/元	35126.95		

表 4 问题一储能分时段充放电量与首末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
0:00–4:00	4500.00	0.00	12:00–16:00	5286.04	91.10
4:00–8:00	833.33	6365.84	16:00–20:00	0.00	5780.13
8:00–12:00	4787.96	1703.00	20:00–24:00	5333.33	2859.87
0:00 储电量/kWh	6000.00		24:00 储电量/kWh	6000.00	

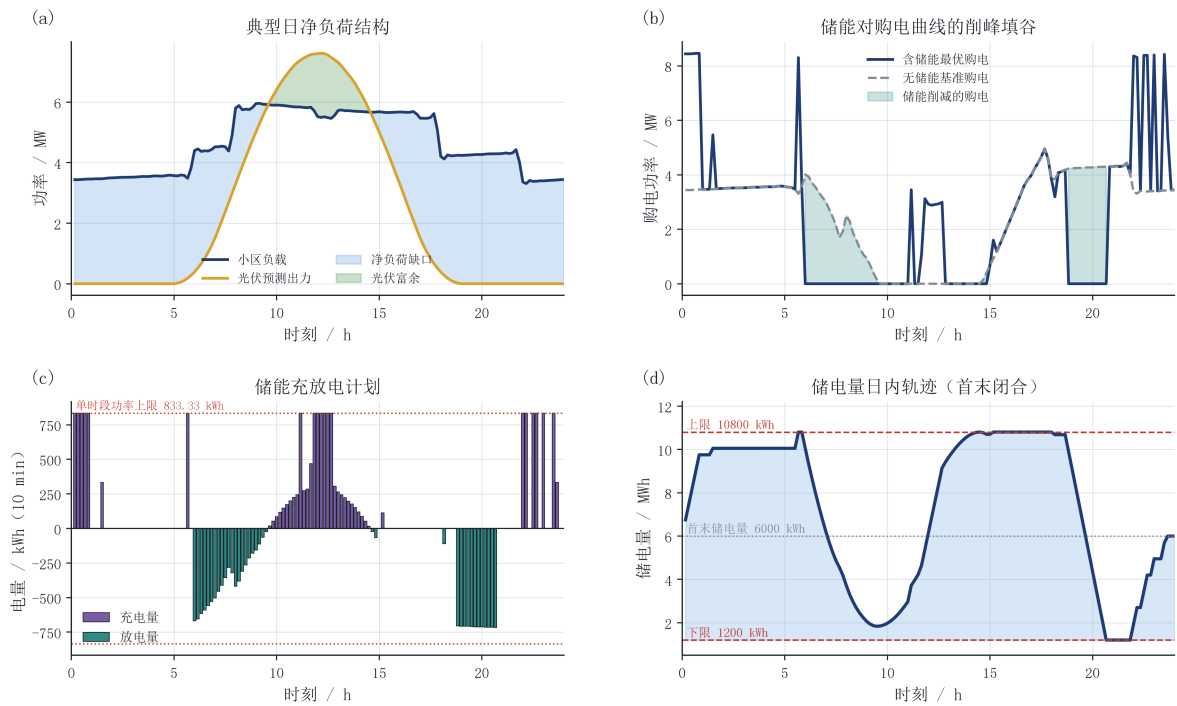


图 3 问题一购电量、储能充放电与储电量曲线

6.3 问题二：因果预测与实时储能反馈

6.3.1 信息集界定与建模难点

问题二与问题一的信息集有本质区别：题目没有给出负载预报，也没有给出光伏预报，只提供历史实际数据。因此当天的负载与光伏在 0:00 都是**未知量**，只能依据历史观测制定计划。若直接读入当天真实负载，就会产生未来信息泄漏，并把方案错误地变成完美信息基准。

于是问题二可写成如下的两阶段形式：

$$\min_{G^p} \mathbb{E} \left[\sum_t \pi_t G_t^p + \sum_t 5\pi_t E_t \right], \quad (12)$$

期望取在负载—光伏的联合不确定性上。由于赛中有限历史不足以稳健估计联合场景及其概率，实际求解不显式生成情景树，而采用**逐日因果确定性等价**：用决策前历史构造 $\hat{L}, \hat{P}$ ，据此求解全天计划，再以真实实现值进行逐时反馈和费用结算。因此式 (12) 给出决策结构，数值算法则是其可复现近似；近似误差由消融实验与样本外结算量化。

6.3.2 风险定价：单时段 0.2 分位启发

直接用条件均值预测光伏并不最优。考虑单位电量上“高估光伏”与“低估光伏”的边际损失：若预测偏高 $\delta > 0$ ，计划购电将少买 δ ，真实缺口需按 5π 紧急购入，相比按 π 正常购入多支出 $4\pi\delta$ ；若预测偏低 δ ，计划多买 δ ，仅多支出 $\pi\delta$ 。因此单位偏差的非对称损失为

$$\ell(\hat{P} - P) = 4\pi(\hat{P} - P)^+ + \pi(P - \hat{P})^+, \quad (13)$$

即高估的权重是低估的 4 倍。设光伏的条件分布为 F ，最小化 $\mathbb{E}[\ell]$ 得到的一阶条件为

$$F(\hat{P}^*) = \frac{b}{a+b} = \frac{1}{4+1} = 0.2, \quad \hat{P}^* = F^{-1}(0.2), \quad (14)$$

其中 $a = 4$ 、 $b = 1$ 分别为高估与低估的局部损失权重。式 (14) 是经典报童模型的分位点结论 [10]：在**孤立单时段、其他条件固定**时，5 倍紧急电价对应光伏输出的 0.2 条件分位数。这一风险定价思路也用于可再生能源投标 [11]。由于全年模型还含负载误差、SOC 跨期耦合、容量与功率上限及实时补救，该结论不等同于对全年最优性的证明；本文把它作为候选安全边际，并通过严格样本外费用在线选择。

据此构造光伏预测。设第 m 个基方法给出 $B_{d,t}^m$ ，并明确定义历史残差为

$$R_{d,t}^m = B_{d,t}^m - P_{d,t}. \quad (15)$$

若希望把输出移到光伏条件分布的 α 分位，则应减去残差的 $1 - \alpha$ 分位数：

$$\hat{P}_{d,t}^{(m,q)} = \max \{0, B_{d,t}^m - Q_q(\mathcal{R}_m)\}, \quad \alpha = 1 - q, \quad (16)$$

其中 $Q_q(\mathcal{R}_m)$ 为残差的经验分位数。故式 (14) 对应 $q = 0.80$ ，而非残差的 0.20 分位。为适应跨期约束，程序把 $q = 0.80, 0.85, 0.90, 0.95$ （即输出分位 0.20, 0.15, 0.10, 0.05）连同多个基预测器共同作为候选，在线选型准则为

$$J_m = \sum_{d' \in \mathcal{W}} \sum_t \pi_{d',t} \left[4 \left(\hat{P}_{d',t}^{(m)} - P_{d',t} \right)^+ + \left(P_{d',t} - \hat{P}_{d',t}^{(m)} \right)^+ \right] \Delta t, \quad (17)$$

其中窗口 $\mathcal{W}$ 为过去 30 天。每日只依据此前已实现的样本外损失选择次日候选；四个残差分位候选的累计费用代理与被选天数分别为： $q = 0.80$ 时 1889056.42 元、162 天， $q = 0.85$ 时 1866485.67 元、109 天， $q = 0.90$ 时 1944258.96 元、53 天， $q = 0.95$ 时 2278103.51 元、10 天。该记录由程序直接导出，且选型只使用已经发生的误差，满足式 (9)。

负载侧同理构造因果预测器：候选方法族为持续法、7/14/30 日滚动均值、14/30 日线性加权均值与同星期均值；日内 6:00 之后的决策还可利用当天已经执行完的时段对基预测做水平校正。由于负载预测的损失基本对称，选型准则采用平均绝对误差。

6.3.3 计及实时储能反馈的执行阶段

0:00 计划一经确定便不再修改，但日内允许储能依据当前时刻已发生的真实光伏做 10 min 级调整。记计划值为 $(\cdot)^p$ ，执行规则为：

- (1) 若真实供电不足，记缺口为 $D_t = L_{d,t} \Delta t + C_t^p - G_t^p - P_{d,t} \Delta t - H_t^p > 0$ 。先撤销充电量 $r_t = \min(C_t^p, D_t)$ ，令 $C_t = C_t^p - r_t$ ；剩余缺口 $D_t - r_t$ 再由增加放电补充（受功率与储电量下限约束），仍不足时才按 5 倍电价紧急购电。
- (2) 若真实光伏富余，记富余电量为 $S_t > 0$ ：先减少放电，再增加充电（受功率上限与储电量上限约束），仍有余量则弃光。

该规则在每个时段只读取当前的真实光伏与当前储电量，不涉及任何未来信息。若关闭该反馈（ $C_t \equiv C_t^p$ 、 $H_t \equiv H_t^p$ ，仅做储电量边界的被动钳位），缺口将全部转为 5 倍紧急购电，因此二者的费用差即为实时储能调节的价值。

6.3.4 求解结果与原因拆解

主方案（因果负载预测 + 非对称光伏风险边际 + 实时储能反馈）在 2 月至 12 月共 334 天的总费用为 14773804.78 元，其中计划购电费 12877657.23 元、紧急购电费 1896147.55 元；紧急购电量 348036.96 kWh，弃光 2326222.67 kWh，共 232 天发生紧急购电；负载预测平均绝对误差 188.88 kW，光伏预测平均绝对误差 198.14 kW。

为回答“原先偏低的费用究竟由什么造成”，本文固定其余条件不变，只改变单一因素，构造四组对照（表 5）：A 组用当天真实负载（完美信息基准），B 组用因果负载预测，C 组把光伏风险边际换成无安全下修的点预测，D 组关闭实时储能反馈。

表 5 问题二单因素对照实验

方案	负载信息	光伏预测	实时反馈	总费用/元	紧急购电/kWh	弃光/kWh
A	完美	风险边际	开	13275988.13	61482.52	2000031.09
B	因果	风险边际	开	14773804.78	348036.96	2326222.67
C	因果	点预测	开	16038813.56	656904.88	1688048.18
D	因果	风险边际	关	15360709.59	672280.72	2566068.96

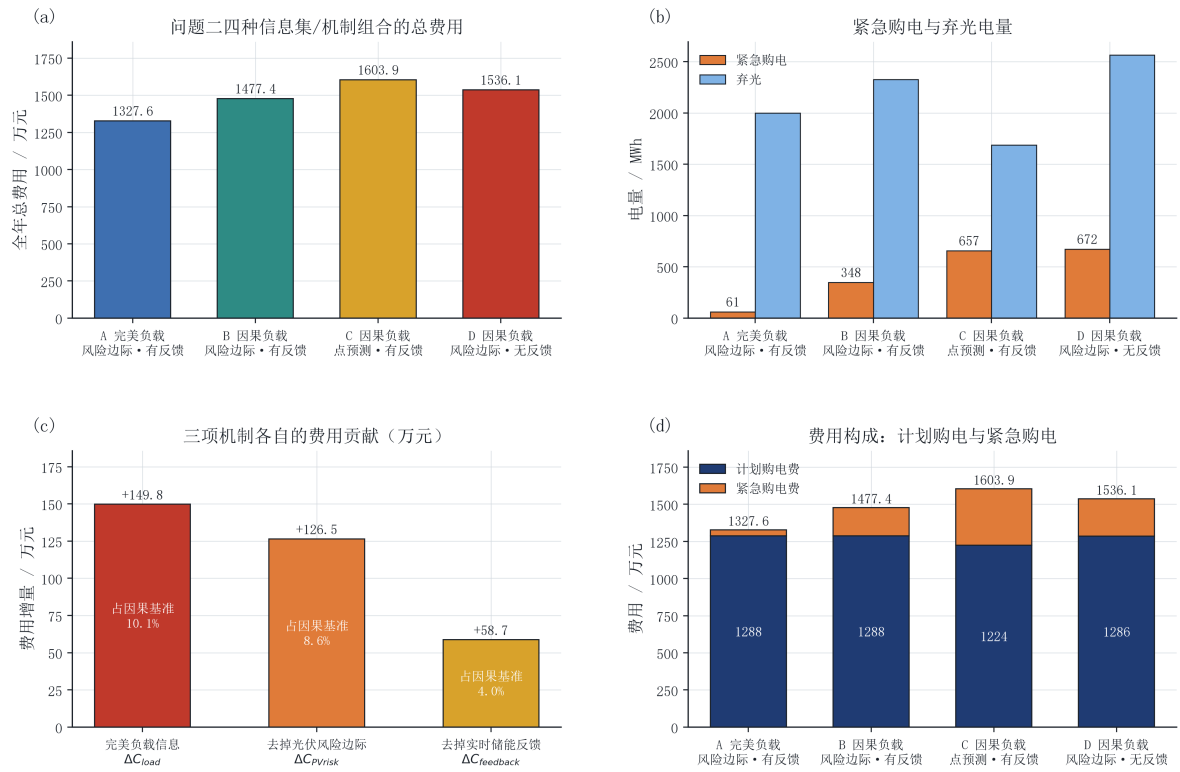


图 4 问题二费用构成与原因拆解

由图 4(c) 的瀑布分解可以量化三个因素：

$$\begin{aligned}
 \Delta C_{load} &= C_B - C_A = 1497816.65 \text{ 元}, \\
 \Delta C_{PVrisk} &= C_C - C_B = 1265008.78 \text{ 元}, \\
 \Delta C_{feedback} &= C_D - C_B = 586904.81 \text{ 元}.
 \end{aligned} \tag{18}$$

结果表明，误把当天真实负载作为 0:00 已知输入，会低估全年费用 1497816.65 元，占修正结果的 10.14%，这是原结果偏低的首要原因。去掉风险边际后费用增加 1265008.78

元；关闭实时反馈后费用增加 586904.81 元，紧急购电由 348036.96 kWh 增至 672280.72 kWh。

图 4(b)(d) 还显示，A 组的弃光量最大而紧急购电最少：完美负载信息使计划与执行几乎无偏差，光伏富余被完整地“计划”进储能与弃光通道，这进一步印证了 A 组是一个不可实现的理想化下界。

按题面要求，表 6–8 给出四个指定日期的购电、储能和紧急购电结果；所有数值由最终 result2.xlsx 自动抽取，完整 144 时段明细见支撑材料。

表 6 问题二指定日期购电结果

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	计划费/元
2025-03-20	0.00	628.79	0.00	478.85	716.29	0.00	70508.76	42753.80
2025-06-21	0.00	0.00	0.00	136.81	458.04	0.00	33673.85	18879.29
2025-09-23	0.00	507.89	0.00	519.64	831.20	5.90	71887.69	45083.22
2025-12-21	0.00	947.47	0.00	821.34	725.20	0.00	92741.53	59280.18

表 7 问题二指定日期储能结果

日期	各 4 h 时段充电量/放电量 (kWh)						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	356.39/185.93	1361.00/4608.26	6269.69/2777.38	2604.97/288.51	112.53/5894.56	9737.25/2860.63	9963.05	9899.29
2025-06-21	2376.47/537.80	1359.38/1738.75	9741.37/0.00	0.00/0.00	0.00/6210.53	158.74/2511.19	1200.00	1252.06
2025-09-23	3.43/2.78	158.81/5967.15	5693.12/1896.62	4388.33/430.85	0.00/5509.97	10293.05/2827.40	10800.00	10800.00
2025-12-21	3646.73/622.21	1638.21/2255.21	5301.47/6488.22	8099.28/2311.29	177.27/5918.77	6323.72/2804.89	6832.77	6833.46

表 8 问题二指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (127.41); 21:40–21:50 (5.16)	132.57	43662.61
2025-06-21	20:30–20:40 (56.42); 21:30–23:40 (555.36)	611.79	20614.91
2025-09-23	20:20–20:30 (6.38)	6.38	45126.21
2025-12-21	7:50–8:00 (19.18); 9:50–10:50 (1391.06); 20:30–20:40 (89.74)	1499.99	66728.16

6.4 问题三：多时刻滚动优化与节点价值消融

6.4.1 滚动优化框架

问题三在每天 0:00、6:00、12:00、18:00 可获得未来 24 h 整点的光伏预报。本文的决策链为：

- (1) **0:00 基准计划**：由节点 0 的预报降尺度得到 144 个时段的光伏预测，在公共约束下得到全天基准计划 G^p ，并执行 0:00 ~ 6:00 的 36 个时段；
- (2) **滚动调整**：在 6:00、12:00、18:00 各节点，以当时实际测得的储电量为初值，用新到达的预报重新优化未来 24 h 的购电与储能，但只执行到下一个决策节点前的 36 个时段；
- (3) **状态与计划递推**：执行块结束时把实测储电量传递给下一个节点；若某节点不允许更新，则继续执行最近一次有效计划的对应切片，不回退到 0:00 方案。跨午夜只传递储电量，次日 0:00 生成新基准计划。

这一“重优化—短执行—状态传递”的结构使每次更新都建立在上一执行块的**实际状态**之上；未更新时则保持最近计划继续生效。该机制既符合题目的时序语义，也与微网能量管理中的滚动时域控制一致 [6, 7]。

附件 3 给出的是整点值，需降尺度到 10 min 端点。本文比较两种方式：阶梯保持（每个小时值重复 6 次）与线性插值。图 5(c) 显示，在真正会被执行的 6 h 块上，线性插值的平均绝对误差为 110.22 kW，显著低于阶梯保持的 294.37 kW，因此主方案采用线性插值。此外，本文对每个发布节点分别维护最近 30 天已执行块的预报残差，并按式 (13) 的费用代理在 0.80 ~ 0.95 分位数间在线选择下修边际，实现分发布节点的因果偏差校正。

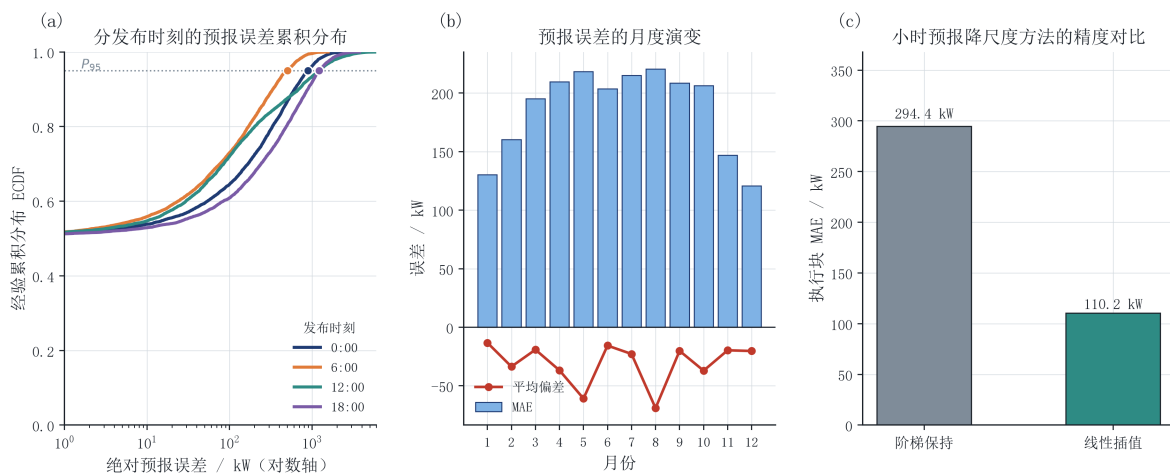


图 5 附件 3 预报误差诊断与降尺度对比

6.4.2 主结算口径与敏感性口径

题目明确写出总费用由计划购电费、调整相关费用和紧急购电费相加，故本文以**叠加结算**作为主口径。设 G^p 为 0:00 计划购电量、 G^a 为执行前最后一次有效计划，并记 $u = (G^a - G^p)^+$ 、 $v = (G^p - G^a)^+$ ，单时段主结算费用为

$$C_t^{\text{add}} = \pi_t G_t^p + 0.5\pi_t v_t + 1.5\pi_t u_t. \quad (19)$$

为检验文字解释的稳健性，另设**差额结算**敏感性口径，即把调整后的实际购电结算额与违约 / 增购费用合并：

$$C_t^{\text{rep}} = \pi_t [\min(G_t^p, G_t^a) + 0.5 v_t + 1.5 u_t], \quad (20)$$

当 $G^a < G^p$ 时实付 $\pi(G^a + 0.5v)$ ，当 $G^a > G^p$ 时实付 $\pi(G^p + 1.5u)$ 。

两种口径的目标函数不同，因此**最优调度本身也不同**。本文把 u_t, v_t 显式引入模型，并施加等式约束

$$G_t^a - G_t^p = u_t - v_t, \quad u_t, v_t \geq 0, \quad (21)$$

从而精确表示两侧调整量。式 (21) 必须为等式；若放松为不等式，下调时会漏付违约费用。主模型还施加式 (10) 的充放电互斥约束，避免连续松弛利用内部耗散绕过功率上限。两种口径均从头重新优化，并以直接回代费用验证，而非对同一调度事后换算。

叠加口径下总费用可严格拆成“计划购电费 + 调整相关费用 + 紧急购电费”；差额口径只用于敏感性分析，不作为题面提交工作簿的来源。

6.4.3 决策节点价值消融实验

题目询问“是否需要引入其他时刻的预报”。先把**允许更新决策的节点集**作为消融变量，构造四组方案：

$$\begin{aligned} S_0 &= \{0:00\}, & S_1 &= \{0:00, 6:00\}, \\ S_2 &= \{0:00, 6:00, 12:00\}, & S_3 &= \{0:00, 6:00, 12:00, 18:00\}. \end{aligned} \quad (22)$$

四组的算法、终端策略与实时反馈保持一致；节点关闭时沿用最近一次有效计划。由于一次更新同时使用截至该节点的负载观测和新发布的光伏预报， $S_0 \sim S_3$ 给出的是**综合节点价值**。定义

$$V_6 = C_{S_0} - C_{S_1}, \quad V_{12} = C_{S_1} - C_{S_2}, \quad V_{18} = C_{S_2} - C_{S_3}, \quad V_{\text{total}} = C_{S_0} - C_{S_3}. \quad (23)$$

为避免把综合节点价值误称为“纯光伏预报价值”，再为每个节点加入一个配对控制组 C_k ：在该节点允许用新增负载观测重优化，但光伏预测冻结为上一有效节点版本。于是

$$V_{\tau}^L = C_{S_{k-1}} - C_{C_k}, \quad V_{\tau}^P = C_{C_k} - C_{S_k}, \quad \tau \in \{6, 12, 18\}, \quad (24)$$

分别表示负载信息刷新和官方光伏预报刷新的边际贡献，且两者之和严格等于该节点综合价值。

6.4.4 求解结果

叠加结算主口径下的四组结果列于表 9。

表 9 问题三决策更新节点消融结果（叠加结算）

方案	可用更新节点	总费用/元	增量价值/元	累计价值/元	紧急购电/kWh	调整电量（上/下）/kWh
S_0	0:00	15731164.40	—	0.00	539486.13	0.00/0.00
S_1	+6:00	14510735.12	1220429.28	1220429.28	165129.37	949099.00/29976.01
S_2	+12:00	14213145.74	297589.38	1518018.66	146857.80	706080.83/73982.00
S_3	+18:00	13917403.19	295742.55	1813761.21	68481.28	786757.05/106014.05

由图 6 与表 9 可以得到明确结论：

$$\begin{aligned} V_6 &= 1220429.28 \text{ 元}, & V_{12} &= 297589.38 \text{ 元}, \\ V_{18} &= 295742.55 \text{ 元}, & V_{\text{total}} &= 1813761.21 \text{ 元}. \end{aligned} \quad (25)$$

三个更新节点的综合增量价值均为正，全年累计节省 1813761.21 元，故应引入 6:00、12:00、18:00 三次更新。其中 6:00 节点价值最大，因为它首次利用当日已经实现的负载轨迹并替换距执行更近的光伏预报。图 6(c) 的季节分解显示，这一结论并非由单一日期驱动。

配对控制进一步给出表 10。例如 C_1 允许 6:00 用新增负载观测重优化，但继续使用 0:00 光伏预测；从 $S_0 \rightarrow C_1 \rightarrow S_1$ 即可把该节点的综合收益拆为两部分。三个节点累计的负载刷新贡献为 1225632.49 元，官方光伏预报刷新贡献为 588128.71 元。因此 $S_0 \sim S_3$ 应解释为决策更新的综合价值，而不是把全部收益归给光伏预报。

主方案（ S_3 ，叠加结算）的全年总费用为 13917403.19 元，且

$$12640688.81 + 934466.71 + 342247.67 = 13917403.19 \text{ 元}, \quad (26)$$

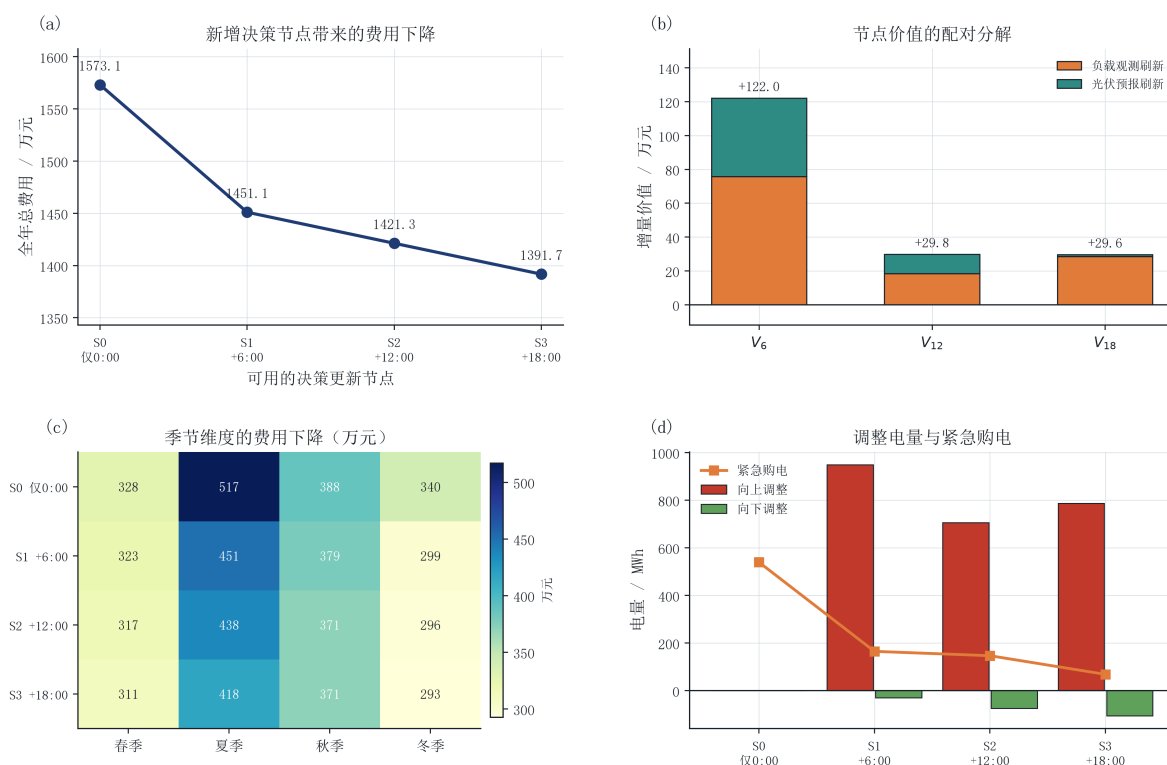


图 6 决策更新节点的增量价值与季节分解

表 10 节点价值的负载刷新与光伏预报配对分解

节点	控制组费用/元	负载刷新贡献/元	光伏刷新贡献/元	两项合计/元
6:00	14974345.92	756818.48	463610.80	1220429.28
12:00	14326797.96	183937.16	113652.22	297589.38
18:00	13928268.89	284876.85	10865.70	295742.55
合计	—	1225632.49	588128.71	1813761.21

三项依次为计划购电费、调整相关费用与紧急购电费，回代差低于 10^{-8} 元；全年紧急购电 68481.28 kWh。相对问题二主方案，滚动调整节省 856401.59 元。差额结算对照的 S_3 费用为 13727989.23 元，累计节点价值为 2003175.17 元，结论方向一致，但它不作为提交主答案。

按题面要求，表 11–13 给出四个指定日期的计划 / 调整购电、储能和紧急购电结果；数值均由最终 result3.xlsx 自动生成。

6.4.5 典型日调度过程

图 7 给出 2025 年 6 月 21 日的调度全景。该日为夏季高辐照日，光伏峰值超过 8 MW，而小区负载仅约 3.8 MW，二者严重错配。

表 11 问题三指定日期购电结果

日期	口径	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	费用/元
2025-03-20	计划	0.00	561.89	0.00	615.11	716.29	0.00	69520.52	42031.40
	调整	0.00	561.89	0.00	615.11	716.29	0.00	71118.10	43975.30
2025-06-21	计划	0.00	0.00	0.00	75.05	399.26	0.00	31110.55	17009.78
	调整	0.00	0.00	0.00	75.05	399.26	0.00	32775.70	19480.32
2025-09-23	计划	0.00	416.97	0.00	566.94	831.20	5.90	73888.09	46335.87
	调整	0.00	436.86	0.00	566.94	831.20	5.90	72865.79	46796.73
2025-12-21	计划	0.00	937.06	0.00	796.19	725.20	0.00	90602.15	57486.24
	调整	0.00	937.06	0.00	825.73	725.20	0.00	94204.59	61748.15

表 12 问题三指定日期储能结果

日期	各 4 h 时段充电量/放电量 (kWh)						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1774.80/185.93	1263.10/5143.78	5669.44/2777.38	3965.38/254.72	19.98/5853.36	8370.56/2897.33	8685.45	8628.49
2025-06-21	1469.50/642.66	1255.97/1285.96	10322.21/0.00	0.00/0.00	172.24/5758.17	397.48/2685.89	1200.00	1930.46
2025-09-23	3.43/2.78	138.54/6205.13	6293.08/1896.62	4068.83/403.62	36.12/5596.21	10129.96/2827.40	10800.00	10589.91
2025-12-21	3261.87/622.21	1944.62/1464.57	5424.47/7563.28	8080.05/2298.66	47.67/5925.83	6759.39/2804.89	7236.46	7225.56

图 7(a) 显示，储能在凌晨与上午充电，功率一度达到 -5 MW 。图 7(c) 中，储电量由 1.2 MWh 升至 10.8 MWh 上限，并在中午前后维持高位，表明 12 MWh 额定容量不足以吸纳该日全部富余光伏。不能向外网售电后，余量只能弃光；图 7(d) 的弃光峰值超过 4500 kW ，而同时段紧急购电接近零。

图 7(b) 对比了 0:00 基准计划与最终调整购电：两者走势基本一致，说明 0:00 的预报已经把握了当日的主要形态；红色（向上调整）与绿色（向下调整）区域集中出现在 18:00 ~ 21:00 的傍晚时段，这正是 6:00、12:00、18:00 新预报对午后云量变化做出修正

表 13 问题三指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (127.39); 21:40–21:50 (5.16)	132.55	44884.02
2025-06-21	6:20–7:20 (452.79)	452.79	21390.79
2025-09-23	20:20–20:30 (6.38)	6.38	46839.73
2025-12-21	7:50–8:00 (29.09); 10:10–10:50 (630.86); 20:30–20:40 (89.74)	749.70	65550.73

的结果。

这一典型日也揭示了模型的主要经济矛盾：夏季的正午弃光与傍晚高价购电同时存在，二者都由储能容量与充放电功率上限决定，而非由预测精度决定。这解释了为什么图 6(c) 中夏季的费用改善幅度最大——后续预报能在傍晚前提前调整储能的放电节奏，把有限的容量留给真正高价时段。

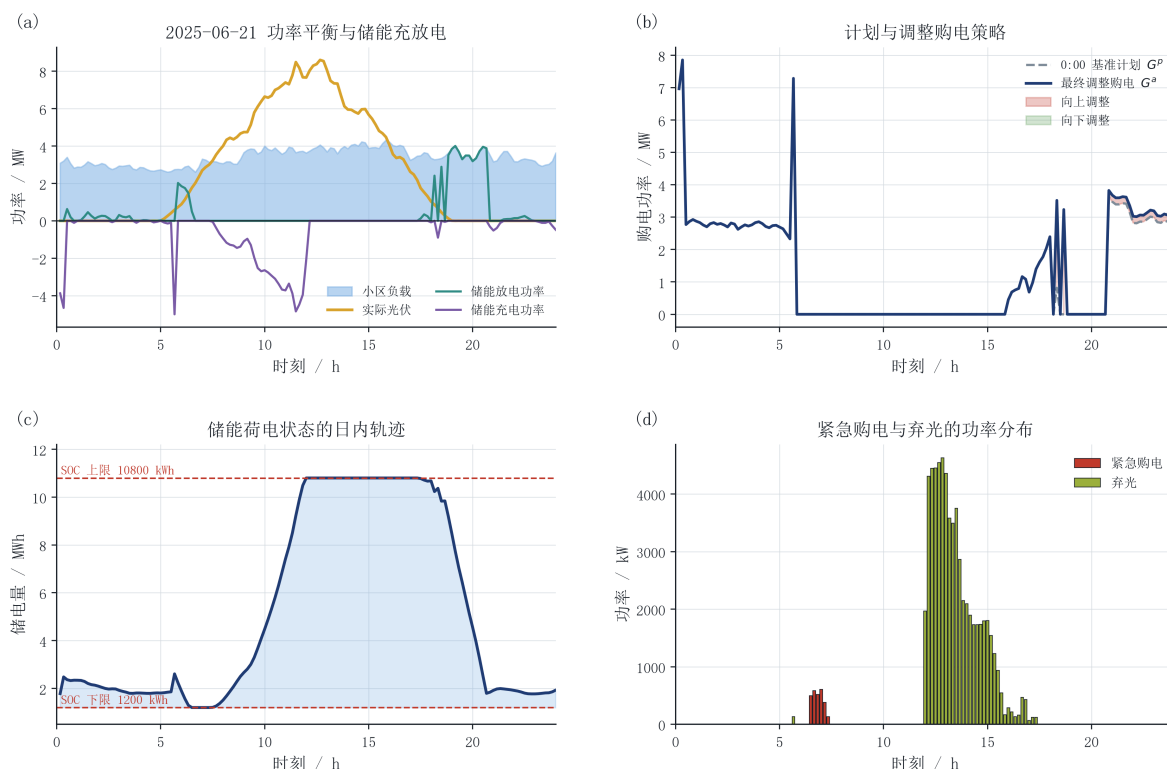


图 7 典型日调度过程（2025-06-21）

6.5 问题四：波动电价下的重算与价格信息边界

6.5.1 模型迁移

问题四保持问题二、三的数据范围、负载 / 光伏信息集、储能反馈与结算规则不变，只把固定典型日电价 π_t 替换为附件 4 的 $\pi_{d,t}$ 。题面明确要求“根据附件 4 中的电价数据”重算，因而直接代入附件 4 是主口径，也是 result4-2.xlsx 与 result4-3.xlsx 的唯一来源。另考虑实际交易中价格可能逐步发布的情形，构造因果价格预测扩展；其候选方法族与负载侧一致，只使用决策节点之前已经发生的价格，并按历史平均绝对误差在线选型。

- 附件 4 直用（主结果）：调度目标中的逐时价格直接取附件 4；
- 因果价格（扩展）：仅使用决策时刻之前已发生的价格预测后续价格，用于考察发布机制受限时的稳健性。

6.5.2 求解结果与价格信息价值

四组结果列于表 14。

表 14 问题四波动电价下的四组结果

方案	价格信息	总费用/元	紧急购电/kWh	弃光/kWh
Q4-2	附件 4 直用（主）	15523884.83	355425.99	2365390.66
Q4-2	因果预测	15590096.88	354599.54	2322027.45
Q4-3	附件 4 直用（主）	14589632.38	87429.50	2391744.79
Q4-3	因果预测	14722175.07	84956.17	2387567.36

相对于因果扩展，直接使用附件 4 所降低的费用为

$$V_{\text{price},Q2} = 66212.05 \text{ 元 (0.42\%)}, \quad V_{\text{price},Q3} = 132542.68 \text{ 元 (0.90\%)}, \quad (27)$$

这两个差额衡量的是价格发布信息受限的机会成本，而非对题面主口径的修正。其相对幅度见式 (27)。一方面，因果预测已经抓住较稳定的日内结构；另一方面，不允许售电时，储能套利受 12000 kWh 容量和 5000 kW 功率的双重限制，即使掌握完整价格也无法无限转移电量。

与问题二的负载信息对照相比，负载未来信息泄漏造成的费用低估为 1497816.65 元，明显大于上述价格情形差额。工程上应优先改善负载与光伏预测；但竞赛交付仍严格采用题面指定的附件 4 直用结果。

另一方面，动态电价下的滚动调整价值为

$$C_{Q4-2}^{\text{perfect}} - C_{Q4-3}^{\text{perfect}} = 934252.45 \text{ 元}, \quad C_{Q4-2}^{\text{causal}} - C_{Q4-3}^{\text{causal}} = 867921.82 \text{ 元}, \quad (28)$$

两者均为正，说明在附件 4 直用与因果价格扩展下，多时刻决策更新都能降低费用。其数值差别来自价格序列改变了储能充放电时点，而不改变滚动状态递推的机制。

因果扩展中，0:00 与 6:00 的价格 MAE 分别为 0.053238、0.107166 元/kWh；12:00 与 18:00 分别为 0.112235、0.064665 元/kWh。0:00 最小、18:00 次之，误差并非随发布时间单调下降。

题面指定日期的完整重算结果列于表 15–20，均由附件 4 直用主方案的两个最终工作簿自动生成。

表 15 问题四对应问题二指定日期购电结果

日期	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	计划费/元
2025-03-20	0.00	628.79	0.00	478.85	716.29	709.49	70933.40	43543.79
2025-06-21	0.00	0.00	0.00	136.81	458.04	0.00	34143.61	16105.02
2025-09-23	0.00	530.79	0.00	542.54	831.20	487.35	73783.38	47932.82
2025-12-21	0.00	947.47	0.00	821.34	725.20	0.00	92738.65	69147.78

表 16 问题四对应问题二指定日期储能结果

日期	各 4 h 时段充电量/放电量 (kWh)						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1896.69/1399.06	2151.59/5232.95	6269.69/2777.38	2615.35/288.51	138.93/6617.32	9701.81/2172.38	9897.92	9852.79
2025-06-21	1626.39/1304.00	3854.32/1733.02	8935.37/0.00	0.00/0.00	0.00/6210.53	890.45/3117.22	1200.00	1237.25
2025-09-23	945.57/1171.57	771.33/5594.21	5534.21/1896.62	3946.84/407.95	116.98/6713.26	10237.57/1673.93	10800.00	10800.00
2025-12-21	4516.10/563.53	818.23/2242.93	5301.47/6493.23	7698.88/2313.01	579.78/5273.92	6224.28/3480.03	6715.04	6710.31

七、模型检验与分析

本文构建物理可行性、信息集 mutation、计划持续性、结算分解和历史配置回归五类自动验证，最终主结果全部通过。

7.1 物理可行性与数值精度

对问题一、问题二四组对照、问题三主消融与配对控制、结算敏感性以及问题四全部年度方案逐时回代，检验母线平衡、SOC 递推、边界、互斥和费用分解，结果见表 21。

连续模型由词典序目标排除无效吞吐，叠加结算主模型由式 (10) 严格禁止同时充放电。问题三总费用与“计划费 + 调整费 + 紧急费”直接回代一致。五个结果工作簿

表 17 问题四对应问题二指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (113.09); 21:40–21:50 (5.16)	118.25	44311.65
2025-06-21	0:30–0:40 (10.28); 1:10–2:30 (532.14); 3:30–3:40 (151.46); 20:30–20:40 (56.42); 21:30–23:30 (527.96)	1278.26	19150.64
2025-09-23	20:20–20:30 (6.38)	6.38	47976.90
2025-12-21	7:50–8:00 (19.18); 10:00–10:50 (1386.06); 20:40–20:50 (81.48)	1486.71	79176.58

表 18 问题四对应问题三指定日期购电结果

日期	口径	10:00	12:00	14:00	16:00	18:00	20:00	全天/kWh	费用/元
2025-03-20	计划	0.00	561.89	0.00	615.11	716.29	709.49	69636.12	42721.49
	调整	0.00	561.89	0.00	615.11	716.29	709.49	71055.34	44468.01
2025-06-21	计划	0.00	0.00	0.00	75.05	399.26	0.00	31580.30	14682.69
	调整	0.00	0.00	0.00	75.05	399.26	0.00	33253.38	16931.47
2025-09-23	计划	0.00	416.97	0.00	566.94	831.20	487.35	74177.20	48179.34
	调整	0.00	436.86	0.00	566.94	831.20	487.35	73123.14	48590.66
2025-12-21	计划	0.00	937.06	0.00	796.19	725.20	0.00	90599.28	66885.94
	调整	0.00	937.06	0.00	825.73	725.20	0.00	94456.90	72431.25

表 19 问题四对应问题三指定日期储能结果

日期	各 4 h 时段充电量/放电量 (kWh)						$S_{0:00}$	$S_{24:00}$
日期	0:00–4:00	4:00–8:00	8:00–12:00	12:00–16:00	16:00–20:00	20:00–24:00	kWh	kWh
2025-03-20	1896.69/772.49	2225.87/5234.86	5669.44/2777.38	3986.75/254.72	46.92/6576.11	8120.06/2172.85	8442.96	8429.22
2025-06-21	1626.39/1304.00	2765.24/1733.02	10024.45/0.00	0.00/0.00	206.50/5792.43	1152.74/3286.53	1200.00	1935.59
2025-09-23	945.57/1171.57	623.69/6189.47	5952.05/1896.62	4406.19/403.62	116.98/6713.26	9953.80/1673.93	10800.00	10544.61
2025-12-21	4011.81/609.78	2026.32/2127.30	5424.47/7563.28	7737.10/2298.66	390.62/5280.98	6746.94/3468.33	7210.53	7193.69

保留附件 5 的工作表结构与格式，回读数值与逐时明细一致。

7.2 未来信息泄漏的 **mutation** 测试

物理约束只能保证结果可行，不能保证结果可实现。为此本文设计了四组人为篡改实验，直接检验信息集因果性：

(1) 问题二负载因果性：把第 d 天真实负载整体乘以 10，在不改动 d 日之前历史数据的

表 20 问题四对应问题三指定日期紧急购电结果

日期	紧急购电时段及对应电量/kWh	合计/kWh	当日总费用/元
2025-03-20	20:30–20:40 (112.63)； 21:40–21:50 (5.16)	117.78	45232.79
2025-06-21	0:30–0:40 (10.28)； 1:10–2:30 (532.14)； 3:30–3:40 (151.46)	693.88	18396.97
2025-09-23	20:20–20:30 (6.38)	6.38	48634.74
2025-12-21	7:50–8:00 (29.09)； 10:20–10:50 (630.86)； 20:40–20:50 (81.48)	741.43	77404.53

表 21 模型检验结果汇总

检验项目	最大值	验收阈值	结论
母线能量平衡残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量递推残差	$< 10^{-12}$ kWh	10^{-7} kWh	通过
储电量取值区间	[1200, 10800] kWh	同上	通过
主方案同时充放电量	$< 10^{-8}$ kWh	10^{-5} kWh	通过
问题三费用分解回代差	$< 10^{-8}$ 元	10^{-7} 元	通过
Excel 回读与逐时明细差异	$< 10^{-10}$	数值一致	通过

条件下重新求解，要求第 d 天 0:00 的计划输入完全不变（最大偏差 $< 10^{-10}$ ）；

- (2) **问题二光伏因果性：**同法篡改第 d 天真实光伏，要求 0:00 的光伏计划输入完全不变；
- (3) **问题三节点因果性：**篡改第 d 天 6:00 之后的真实负载与真实光伏，要求 6:00 决策输入与结果不变；后续节点可合法使用已经执行完的观测；
- (4) **问题四价格因果性：**篡改第 d 天 6:00 之后的附件 4 真实电价，要求因果价格模式下 6:00 决策不变，而附件 4 直用模式下应当改变。
- (5) **计划持续性：**仅启用 0:00, 6:00 时，核对后续切片均来自 6:00 更新的有效计划，而非回退到 0:00 基准计划。

五组测试全部通过。第 (4) 组中同一篡改确实会改变附件 4 直用结果，说明因果扩展下的“不变”来自信息隔离而非篡改无效；第 (5) 组直接覆盖了节点消融最容易出现计划回退错误。

7.3 结算口径的单元测试

按题目文字直接手算校验两种口径的费用公式。取 $\pi = 1$ 元/kWh：

- **replacement:** $G^p = 900, G^a = 700 \Rightarrow 700 + 0.5 \times 200 = 800$; $G^p = 700, G^a = 900 \Rightarrow 700 + 1.5 \times 200 = 1000$;
- **additive:** $G^p = 900, G^a = 700 \Rightarrow 900 + 0.5 \times 200 = 1000$; $G^p = 700, G^a = 900 \Rightarrow 700 + 1.5 \times 200 = 1000$ 。

两组手算结果与程序输出一致。在随机负载 / 光伏 / 价格序列上，优化目标与直接回代公式之差小于 10^{-7} ；两种口径分别重优后，各自费用均不高于把另一口径方案直接代入所得的费用。

7.3.1 充放电互斥的混合整数检验

连续松弛在叠加结算下可能利用“边充边放”耗散 SOC。V4 已将修复直接纳入主模型：每个 24 h 滚动窗口设置至多 144 个二元变量 z_t ，由式 (10) 精确互斥。全年主方案最大同时充放电量低于 10^{-8} kWh，所有滚动子问题均达到最优状态；因此该现象不再是结果缺陷或待改进项。

7.4 历史配置的回归复现

为确认修订没有破坏原有工程，本文在旧参数组合（完美负载、附件 4 直用、实时反馈、非对称风险边际、差额结算、日循环终端且不用典型日先验）下重新运行四问，与既有结果逐项比对：

表 22 历史配置结果回归比对

结果	修订前/元	修订后/元	绝对偏差/元
问题一	35 126.9486	35 126.9486	$< 10^{-3}$
问题二	13 276 966.0012	13 276 966.0012	$< 10^{-3}$
问题三	13 048 124.9508	13 048 124.9508	$< 10^{-3}$
问题四-2	13 949 811.6640	13 949 811.6640	$< 10^{-3}$
问题四-3	13 734 900.0034	13 734 900.0034	$< 10^{-3}$

表 22 说明，新增的信息集模式、结算口径与参数化储能没有改变历史配置行为。旧结果不是数值错误，而是完美信息基准；本文仅用它量化信息泄漏造成的乐观偏差。

7.5 灵敏度分析

7.5.1 终端储电量策略

题目只对问题一明确要求 $S_{0:00} = S_{24:00}$ ，问题二、三并未提出该要求。本文主模型采用“计划终端储电量等于当日实际初始储电量”的日循环口径（daily_cycle），以保证储能长期可持续运行；同时构造不做终端约束（free）的对照组。结果为：daily_cycle 总费用 14773804.78 元、全年末储电量 7107.51 kWh；free 总费用 14728422.40 元、全年末储电量 1235.46 kWh。

free 口径在数值上便宜约 0.307%，但其日末储电量均值仅为 1660 kWh，全年末储电量为 1235.46 kWh，接近 1200 kWh 下限。这是有限时域优化的末端透支：不计后续价值时，模型倾向于在窗口末端放空电池。

图 8(c) 中 free 的日末 SOC 标准差为 896 kWh，确实小于 daily_cycle 的 3798 kWh；这只是因为 free 结果集中在低 SOC 区间，不能解释为更可持续。综合费用差、均值位

置和全年末状态，本文保留 `daily_cycle` 为主口径。

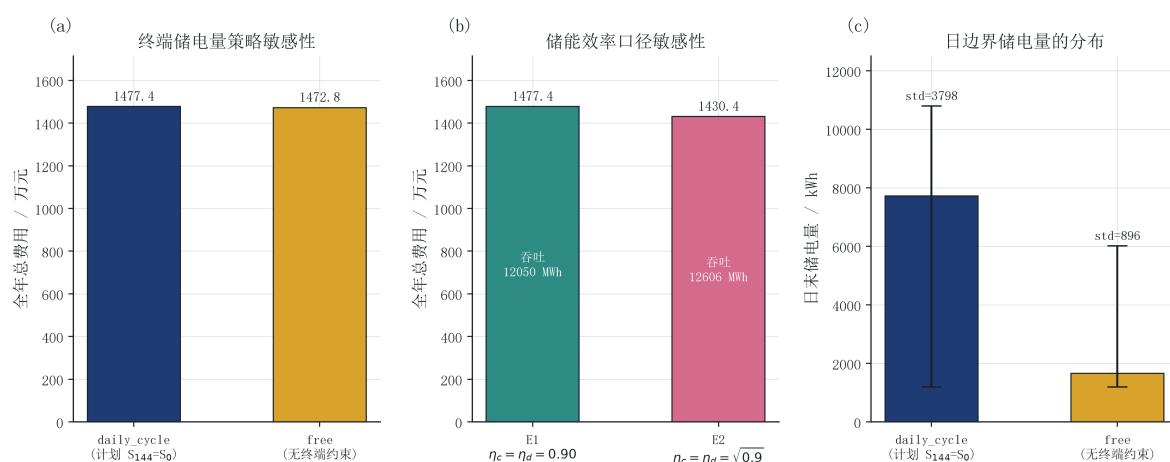


图 8 终端策略与储能效率口径敏感性

7.5.2 储能效率口径

题目只写“充放电效率为 90%”，存在两种解释：其一，充电效率与放电效率各为 0.9（往返效率 0.81）；其二，往返总效率为 0.9（充放电效率各为 $\sqrt{0.9} \approx 0.9487$ ）。前者是更保守也更为工程界常用的口径，本文主模型采用前者。作为对照，两种口径下的结果列于表 23。

表 23 储能效率口径敏感性

效率口径	全年总费用/元	储能吞吐量/kWh	紧急购电量/kWh
口径 A: $\eta_c = \eta_d = 0.90$ （往返 0.81）	14773804.78	12049920.76	348036.96
口径 B: $\eta_c = \eta_d = \sqrt{0.9}$ （往返 0.90）	14304371.70	12606462.13	349968.83

采用往返效率 0.90 时费用更低，因为储能损耗更小、可用于峰谷套利的电量更多。两种口径的结论方向完全一致，说明本文的策略结论不依赖于效率口径的具体解释。

7.5.3 其他误差来源

主要误差来源包括三方面：光伏与负载点预测误差、小时预报到 10 min 的降尺度误差，以及实际市场价格发布时间的不确定性。前两者已通过样本外在线选择、实时反馈和节点消融量化；第三项不改变题面“附件 4 直用”的主答案，因果价格扩展仅用于说明部署时可能的机会成本。这些误差不破坏数值结果的内部一致性，但限制模型直接外推到真实长期运行的精度。连续松弛在叠加结算下可能利用“边充边放”耗散 SOC。V4 已将修复直接纳入主模型：每个 24 h 滚动窗口设置至多 144 个二元变量 z_t ，由式 (10)

精确互斥。全年主方案最大同时充放电量低于 10^{-8} kWh，所有滚动子问题均达到最优状态；因此该现象不再是结果缺陷或待改进项。

八、模型的评价

8.1 模型的优点

- **物理内核统一、四问口径一致。**四个问题共用式 (2)~(7) 的物理约束，各问只改变信息集与费用项；全部年度方案均通过逐时回代。
- **信息集边界严格且可自动验证。**所有预测器只使用决策时刻之前已发生的观测；五组 mutation 与计划持续性测试把“无未来信息泄漏”变成可重复执行的验收项。
- **风险边际有可解释依据。**5 倍紧急电价在单时段下导出 4 : 1 非对称损失和光伏输出 0.2 分位启发；全年模型不直接外推该局部结论，而在多个残差分位候选中按历史样本外费用在线选择。
- **状态与计划递推严格。**每次滚动优化以上一执行块实测 SOC 为初值；节点关闭时最近有效计划持续生效，不会回退到日初方案。
- **结论可归因。**问题二 A/B/C/D 拆分负载信息、风险边际与实时反馈；问题三在主节点消融外增加配对控制，分别量化负载观测刷新和光伏预报刷新；问题四区分附件 4 直用主结果与因果价格扩展。
- **结算与物理约束一致。**叠加结算按题面作为主口径，费用三项可直接相加； u, v 等式精确表示调整方向， z_t 二元变量严格排除同时充放。差额口径从头重优，仅作敏感性对照。
- **工程可复现。**四问均有独立求解脚本，输出保留环境信息、随机种子与结果哈希；历史参数组合可复现修订前数值。

8.2 模型的不足

- 问题二与问题三采用“计划终端储电量等于当日实际初值”的日循环口径。该处理抑制日末透支，但限制跨日峰谷套利；灵敏度分析中费用影响为 0.307%。
- 光伏预测未引入数值天气预报、辐照度、温度与云量等外生变量，仅使用同刻历史序列与残差分位数。当气象形态发生突变（例如连续阴雨转晴）时，预测与安全边际的调整会滞后一天左右。
- 储能模型忽略自放电、容量衰减、循环寿命成本与效率随功率变化的特性，因此年度费用中没有反映电池的长期老化代价，得到的充放电深度可能比实际最优更大。
- 问题四按题面直接使用附件 4 全部价格；若真实市场并非提前发布完整曲线，则因果扩展更接近部署条件，但其预测精度仍可提高。

- 问题二采用逐日滚动的确定性等价形式求解两阶段模型，安全边际由历史残差分位数估计。当可用历史不足（如年初的前 7 天）时，分位数估计的稳定性有限，本文以附件 1 的典型日曲线作为冷启动先验，但这仍是一个近似。

九、模型的改进与推广

9.1 模型的改进

(1) 把单日滚动扩展为多日滚动时域。当前模型每天重新生成基准计划，跨日只传递储电量。若把优化窗口扩展到 3 ~ 7 天并只执行首日决策，同时在窗口末端加入基于未来价格与负载的储能残值函数，就可以量化跨日套利收益并进一步降低边界效应。

(2) 把确定性等价升级为显式随机规划。本文只在单时段获得 0.2 分位启发；若显式建立负载—光伏联合情景，可进一步构造情景抽样（SAA）或条件风险价值（CVaR）模型，直接刻画跨时段风险，而无需把局部分位结论近似推广到全年。

(3) 引入更完整的价格预测。可在电价预测中加入日内周期项、工作日 / 节假日效应与温度特征，或采用分位数回归直接给出价格分布，从而把电价不确定性也纳入目标函数而非仅做信息集分层。

(4) 细化储能模型。可把循环深度相关的寿命损耗、效率曲线、自放电与维护成本写入目标函数，并用历史运行数据重新标定参数；当模型规模增大时，可采用模型预测控制与 Benders 分解等算法保持滚动求解速度。

9.2 模型的推广

本文的“统一物理内核+信息集分层+可解释风险定价”框架不依赖本题数据。园区综合能源系统可将母线平衡扩展为电、热、气多能流平衡，并增加储热、储气状态 [14]；电动汽车充换电站与工业用户侧储能可分别替换负载对象、增加最大需量费；含风光互补的微网可对各电源误差分别设置非对称损失。更一般地，库存、水库与供应链等“状态逐期递推且计划—执行存在偏差结算”的问题，也可复用本文的滚动优化、单因素消融和信息集验证方法。

AI 工具使用声明

本参赛队使用了 MathModel 内置 AI 助手（OpenAI GPT-5），用于审阅既有模型、协助修改 Python 求解与测试代码、分析报错和数值矛盾、生成绘图与 LaTeX 排版草稿，以及语言校订。团队向工具提供赛题、既有工程和两份审阅意见，逐项要求修正结算口径、滚动计划持续性、指定日期表格及提交材料。所有采纳内容均由人工复核；数值结论由本地脚本实际运行，约束与费用逐时回代，并经自动测试和 Excel 回读核验。详细提示过程、采纳范围与人工核验见支撑材料 AI 工具使用详情.pdf。

参考文献

- [1] Wu Hongbin, Liu Xingyue, Ding Ming. Dynamic economic dispatch of a microgrid: mathematical models and solution algorithm[J/OL]. International Journal of Electrical Power & Energy Systems, 2014, 63: 336-346. <http://dx.doi.org/10.1016/j.ijepes.2014.06.002>.
- [2] Nemati M, Braun M, Tenbohlen S. Optimization of unit commitment and economic dispatch in microgrids based on genetic algorithm and mixed integer linear programming[J/OL]. Applied Energy, 2018, 210: 944-963. <http://dx.doi.org/10.1016/j.apenergy.2017.07.007>.
- [3] Wu Zhi, Zeng Pingliang, Zhang X-P, et al. A solution to the chance-constrained two-stage stochastic program for unit commitment with wind energy integration[J/OL]. IEEE Transactions on Power Systems, 2016, 31 (6): 4185-4196. <http://dx.doi.org/10.1109/tpwrs.2015.2513395>.
- [4] 孙硕, 杨仕友. 考虑光伏出力不确定性的分布鲁棒优化调度方法[J/OL]. 激光与光电子学进展, 2025, 62 (9): 0925001. <http://dx.doi.org/10.3788/lop241935>.
- [5] Zhang Yan, Fu Lijun, Zhu Wanlu, et al. Robust model predictive control for optimal energy management of island microgrids with uncertainties[J/OL]. Energy, 2018, 164: 1229-1241. <http://dx.doi.org/10.1016/j.energy.2018.08.200>.
- [6] Elkazaz M, Sumner M, Thomas D. Energy management system for hybrid PV-wind-battery microgrid using convex programming, model predictive and rolling horizon predictive control with experimental validation[J/OL]. International Journal of Electrical Power & Energy Systems, 2020, 115: 105483. <http://dx.doi.org/10.1016/j.ijepes.2019.105483>.
- [7] Silva V A, Aoki A R, Lambert-Torres G. Optimal day-ahead scheduling of microgrids with battery energy storage system[J/OL]. Energies, 2020, 13 (19): 5188. <http://dx.doi.org/10.3390/en13195188>.
- [8] Luo Liang, Abdulkareem S S, Rezvani A, et al. Optimal scheduling of a renewable based microgrid considering photovoltaic system and battery energy storage under uncertainty[J/OL]. Journal of Energy Storage, 2020, 28: 101306. <http://dx.doi.org/10.1016/j.est.2020.101306>.
- [9] Maheshwari A, Paterakis N G, Santarelli M, et al. Optimizing the operation of energy storage using a non-linear lithium-ion battery degradation model[J/OL]. Applied Energy, 2020, 261: 114360. <http://dx.doi.org/10.1016/j.apenergy.2019.114360>.
- [10] Qin Yan, Wang Ruoxuan, Vakharia A J, et al. The newsvendor problem: review and directions for future research[J/OL]. European Journal of Operational Research, 2011, 213 (2): 361-374. <http://dx.doi.org/10.1016/j.ejor.2010.11.024>.
- [11] Polat E, Güler M G, Ulukuş M Y. Renewable GenCo bidding strategy using newsvendor-based neural networks: an example from turkish electricity market[J/OL]. Electric Power Systems Research, 2024, 231: 110301. <http://dx.doi.org/10.1016/j.epr.2024.110301>.
- [12] Markovics D, Mayer M J. Comparison of machine learning methods for photovoltaic power forecasting based on numerical weather prediction[J/OL]. Renewable and Sustainable Energy Reviews, 2022, 161: 112364. <http://dx.doi.org/10.1016/j.rser.2022.112364>.
- [13] Wang Qi, Zhang Chunyu, Ding Yi, et al. Review of real-time electricity markets for integrating distributed energy resources and demand response[J/OL]. Applied Energy, 2015, 138: 695-706. <http://dx.doi.org/10.1016/j.apenergy.2014.10.048>.
- [14] 刘继春, 周春燕, 高红均, et al. 考虑氢能-天然气混合储能的气-电综合能源微网日前经济调度优化[J/OL]. 电网技术, 2018, 42 (1): 170-178. <https://doi.org/10.13335/j.1000-3673.pst.2017.0966>.

附录

附录 A 支撑材料文件列表

表 24 支撑材料文件清单

文件	说明
README.md	支撑材料目录、运行环境与唯一复现命令
__init__.py	Python 包标记，保证从项目根目录稳定导入
AI 工具使用详情.pdf	AI 工具名称、版本、使用环节与人工核验说明
results/	五个最终工作簿：result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx
solve/	全部求解、验证、制表与绘图 Python 源码
tests/	信息泄漏、计划持续性、结算、主结果和历史配置回归测试
reports/MODEL_AUDIT_V4.md	V4 对照实验、采用口径与数值审计报告
reports/model_audit_v4_raw.json	审计报告对应的机器可读原始指标

附录 B 源程序代码

全部代码使用 Python 3.12 编写，线性规划和混合整数线性规划均由 SciPy 调用 HiGHS 求解。项目根目录的唯一测试命令为 `python -m pytest MathModel/tests -q --import-mode=importlib`；各问可用 `python -m MathModel.solve.< 模块名 >` 复现。以下按调用关系列出完整源码。

B.1 物理参数与口径定义（params.py）

Listing 1:

```
1  """储能物理参数与口径变体。
2
3  题目附录 1 只给出"充放电效率为 90%"，存在两种等价解释：
4      E1  充电效率 = 放电效率 = 0.90      （往返效率 0.81）
5      E2  往返总效率 = 0.90                （充电效率 = 放电效率 = sqrt(0.90)）
6  主模型采用 E1；E2 作为效率口径敏感性对照。
7  """
8  from __future__ import annotations
9
10 import math
11 from dataclasses import dataclass
12
13 from .io_data import DT_HOURS, SLOTS_PER_DAY
14
```



```

15 POWER_LIMIT_KW = 5000.0
16 ENERGY_LIMIT = POWER_LIMIT_KW * DT_HOURS # 833.333... kWh / 10 min
17
18
19 @dataclass(frozen=True)
20 class StorageParams:
21     """储能物理参数。充电量 C 为母线侧电量，放电量 H 为母线侧电量。"""
22
23     eta_ch: float = 0.9
24     eta_dis: float = 0.9
25     soc_min: float = 1200.0
26     soc_max: float = 10800.0
27     power_limit_kw: float = POWER_LIMIT_KW
28
29     @property
30     def energy_limit(self) -> float:
31         """单个 10 min 时段的最大充放电量 (kWh)。"""
32         return self.power_limit_kw * DT_HOURS
33
34     @property
35     def round_trip(self) -> float:
36         return self.eta_ch * self.eta_dis
37
38     def as_dict(self) -> dict:
39         return {
40             "eta_ch": self.eta_ch,
41             "eta_dis": self.eta_dis,
42             "round_trip_efficiency": self.round_trip,
43             "soc_min": self.soc_min,
44             "soc_max": self.soc_max,
45             "power_limit_kw": self.power_limit_kw,
46             "energy_limit_per_slot_kwh": self.energy_limit,
47         }
48
49
50 DEFAULT_STORAGE = StorageParams()
51 SQRT_HALF = math.sqrt(0.9)
52 EFFICIENCY_VARIANTS: dict[str, StorageParams] = {
53     "E1_eta_ch_0.90_eta_dis_0.90": StorageParams(0.9, 0.9),
54     "E2_round_trip_0.90": StorageParams(SQRT_HALF, SQRT_HALF),
55 }
56
57 # 终端 SOC 策略
58 TERMINAL_POLICIES = ("daily_cycle", "free")
59 # 信息集口径
60 LOAD_MODES = ("perfect", "causal")
61 PRICE_MODES = ("perfect", "causal")

```

```

62 PV_RISK_MODES = ("asymmetric", "point")
63 SETTLEMENT_MODES = ("replacement", "additive")

```

B.2 数据读取与时间对齐 (io_data.py)

Listing 2:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  from datetime import date, datetime
4  from pathlib import Path
5  import numpy as np
6  import openpyxl
7
8  REPO_ROOT=Path(__file__).resolve().parents[2]
9  DEFAULT_DATA_ROOT=REPO_ROOT/"CUMCM2026Problems"/"C题"/"附件"
10 DT_HOURS=1.0/6.0; SLOTS_PER_DAY=144
11
12 @dataclass(frozen=True)
13 class DayData:
14     labels:list[str]; price:np.ndarray; load_kw:np.ndarray; pv_kw:np.ndarray
15 @dataclass(frozen=True)
16 class AnnualData:
17     dates:list[date]; load_kw:np.ndarray; pv_kw:np.ndarray
18 @dataclass(frozen=True)
19 class RollingForecastData:
20     dates:list[date]; hourly_kw:np.ndarray
21 @dataclass(frozen=True)
22 class AnnualPriceData:
23     dates:list[date]; price:np.ndarray
24
25 def _as_float_array(values:list[object],name:str)->np.ndarray:
26     array=np.asarray(values, dtype=float)
27     if array.shape!=(144,) or not np.isfinite(array).all(): raise ValueError(f"{name}
28         invalid: {array.shape}")
29     return array
30
31 def _coerce_date(value:object)->date:
32     if isinstance(value,datetime): return value.date()
33     if isinstance(value,date): return value
34     text=str(value).strip()
35     for fmt in ("%Y-%m-%d", "%Y/%m/%d", "%m/%d/%Y"):
36         try: return datetime.strptime(text,fmt).date()
37         except ValueError: pass
38     raise ValueError(f"无法解析日期: {value!r}")

```

```

39 def read_attachment1(data_root:Path=DEFAULT_DATA_ROOT)->DayData:
40     wb=openpyxl.load_workbook(Path(data_root)/"附件1.xlsx",data_only=True,read_only=True)
41     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=145,min_col=1,max_col=4,
42         values_only=True))
43     labels=[str(r[0]) for r in rows]; price=_as_float_array([r[1] for r in rows],"price")
44     load=_as_float_array([r[2] for r in rows],"load"); pv=_as_float_array([r[3] for r in
45         rows],"pv")
46     if (price<=0).any() or (load<0).any() or (pv<0).any(): raise ValueError("attachment1
47         bounds")
48     return DayData(labels,price,load,pv)
49
50 def read_attachment2(data_root:Path=DEFAULT_DATA_ROOT)->AnnualData:
51     wb=openpyxl.load_workbook(Path(data_root)/"附件2.xlsx",data_only=True,read_only=True)
52     arrays=[]; date_lists=[]
53     for sheet_name in wb.sheetnames[:2]:
54         rows=list(wb[sheet_name].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,
55             values_only=True))
56         date_lists.append([_coerce_date(r[0]) for r in rows]); values=np.asarray([r[1:] for
57             r in rows],dtype=float)
58         if values.shape!=(365,144) or not np.isfinite(values).all() or (values<0).any():
59             raise ValueError(f"{sheet_name} invalid")
60         arrays.append(values)
61     if date_lists[0]!=date_lists[1]: raise ValueError("attachment2 date mismatch")
62     return AnnualData(date_lists[0],arrays[0],arrays[1])
63
64 def read_attachment3(data_root:Path=DEFAULT_DATA_ROOT)->RollingForecastData:
65     wb=openpyxl.load_workbook(Path(data_root)/"附件3.xlsx",data_only=True,read_only=True)
66     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=1461,min_col=1,max_col=26,
67         values_only=True))
68     if len(rows)!=1460: raise ValueError("attachment3 must have 1460 forecast rows")
69     dates=[]; blocks=[]; current_date=None
70     expected=("0:00","6:00","12:00","18:00")
71     for day in range(365):
72         chunk=rows[day*4:(day+1)*4]
73         current_date=_coerce_date(chunk[0][0])
74         dates.append(current_date)
75         issues=tuple(str(r[1]) for r in chunk)
76         if issues!=expected: raise ValueError(f"attachment3 issue times invalid on {
77             current_date}: {issues}")
78         values=np.asarray([r[2:] for r in chunk],dtype=float)
79         if values.shape!=(4,24) or not np.isfinite(values).all() or (values<0).any(): raise
80             ValueError(f"attachment3 invalid {current_date}")
81         blocks.append(values)
82     return RollingForecastData(dates,np.stack(blocks))
83
84 def hourly_to_ten_min(hourly_kw:np.ndarray,method:str="step")->np.ndarray:
85     hourly=np.asarray(hourly_kw,dtype=float)

```

```

77     if hourly.shape!=(24,): raise ValueError("hourly forecast must have 24 values")
78     if method=="step": return np.repeat(hourly,6)
79     if method=="linear":
80         return np.interp(np.arange(1,145)/6.0,np.arange(1,25),hourly,left=hourly[0],right=
            hourly[-1])
81     raise ValueError(f"unknown downscale method: {method}")
82
83 def read_attachment4(data_root:Path=DEFAULT_DATA_ROOT)->AnnualPriceData:
84     wb=openpyxl.load_workbook(Path(data_root)/"附件4.xlsx",data_only=True,read_only=True)
85     rows=list(wb[wb.sheetnames[0]].iter_rows(min_row=2,max_row=366,min_col=1,max_col=145,
            values_only=True))
86     dates=[_coerce_date(r[0]) for r in rows]; price=np.asarray([r[1:] for r in rows],dtype=
            float)
87     if price.shape!=(365,144) or not np.isfinite(price).all() or (price<=0).any(): raise
            ValueError("attachment4 invalid")
88     return AnnualPriceData(dates,price)
89
90 def template_path(name:str,data_root:Path=DEFAULT_DATA_ROOT)->Path:
91     path=Path(data_root)/"附件5"/name
92     if not path.exists(): raise FileNotFoundError(path)
93     return path

```

B.3 公共线性规划内核 (lp_core.py)

Listing 3:

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4  from time import perf_counter
5
6  import numpy as np
7  from scipy.optimize import Bounds, LinearConstraint, linprog, milp
8
9  from .io_data import DT_HOURS
10 from .params import DEFAULT_STORAGE, ENERGY_LIMIT, StorageParams
11
12 ETA_CH = DEFAULT_STORAGE.eta_ch
13 ETA_DIS = DEFAULT_STORAGE.eta_dis
14 SOC_MIN = DEFAULT_STORAGE.soc_min
15 SOC_MAX = DEFAULT_STORAGE.soc_max
16 POWER_LIMIT_KW = DEFAULT_STORAGE.power_limit_kw
17
18
19 @dataclass
20 class DispatchResult:

```

```

21     grid: np.ndarray
22     charge: np.ndarray
23     discharge: np.ndarray
24     soc: np.ndarray
25     curtail: np.ndarray
26     objective: float
27     primary_objective: float
28     curtail_total: float
29     throughput_total: float
30     status: int
31     message: str
32     nit: int
33     solve_seconds: float
34     equality_marginals: np.ndarray
35
36
37 def _slices(t: int):
38     return slice(0, t), slice(t, 2 * t), slice(2 * t, 3 * t), slice(3 * t, 4 * t), slice(4 *
39         t, 5 * t)
40
41 def _solve(c, a_eq, b_eq, bounds, a_ub=None, b_ub=None):
42     return linprog(
43         c,
44         A_ub=a_ub,
45         b_ub=b_ub,
46         A_eq=a_eq,
47         b_eq=b_eq,
48         bounds=bounds,
49         method="highs",
50         options={"primal_feasibility_tolerance": 1e-9, "dual_feasibility_tolerance": 1e-9},
51     )
52
53
54 def _solve_milp(c, a_eq, b_eq, bounds, integrality, a_ub=None, b_ub=None):
55     """用 HiGHS MILP 求解带充放电互斥的调整模型。"""
56     lower = np.asarray([( -np.inf if lo is None else lo) for lo, _ in bounds], dtype=float)
57     upper = np.asarray([( np.inf if hi is None else hi) for _, hi in bounds], dtype=float)
58     constraints = [LinearConstraint(np.asarray(a_eq), np.asarray(b_eq), np.asarray(b_eq))]
59     if a_ub is not None and len(a_ub):
60         constraints.append(
61             LinearConstraint(
62                 np.asarray(a_ub, dtype=float),
63                 np.full(len(b_ub), -np.inf, dtype=float),
64                 np.asarray(b_ub, dtype=float),
65             )
66         )

```

```

67     return milp(
68         np.asarray(c, dtype=float),
69         integrality=np.asarray(integrality, dtype=int),
70         bounds=Bounds(lower, upper),
71         constraints=constraints,
72         options={"presolve": True, "mip_rel_gap": 1e-10},
73     )
74
75
76 # 词典序层间松弛沿用 legacy 公式：绝对 1e-5 与相对 1e-10 取大者。
77 # 该公式使后序层只能在"同一个最优解"内部换一个顶点，不会移动日调度结果；
78 # 由于微网是带状态反馈的滚动系统，日调度的一丝变化会沿 334 天放大，
79 # 因此这里必须与 legacy 逐位一致，才能保证 S11 的回归复现验收。
80 SLACK_ABS = 1e-5
81 SLACK_REL = 1e-10
82
83
84 def _lexicographic_slack(value: float, factor: float = 1.0) -> float:
85     return factor * max(SLACK_ABS, SLACK_REL * max(1.0, abs(value)))
86
87
88 def _lexicographic(objectives, a_eq, b_eq, bounds, extra_rows=None, extra_rhs=None,
89                     slack_factor=1.0):
90     """依次求解 objectives；后一层在前一层最优值的容差带内继续优化。
91
92     数值退化（大规模最优面上的容差冲突）会让后序层被过紧的界判为不可行。
93     此时按三级逐级退让：（1）原界 → （2）容差放大 100 倍 → （3）只保留**最后一条**界。
94     第（3）级是刻意这样设计的：最后一条界来自第二层（吞吐量）目标，而"消除同时
95     充放"正是靠这一层实现的；若把它也丢掉，最优面上的退化解（C=H=大值，母线
96     平衡不变但储电量缓慢下降）就会重新出现。
97
98     返回 (x, 最后一层的求解结果, 是否发生退让, 各层目标值)。
99     """
100     base_rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
101     base_rhs = [float(v) for v in (extra_rhs or [])]
102     rows = list(base_rows)
103     rhs = list(base_rhs)
104     x = None
105     result = None
106     degraded = False
107     values: list[float] = []
108     for index, c in enumerate(objectives):
109         solution = None
110         for factor, keep in ((1.0, None), (100.0, None), (1.0, 1)):
111             if keep is None:
112                 sel_rows, sel_rhs = rows, rhs

```

```

113         sel_rows, sel_rhs = rows[-keep:], rhs[-keep:]
114         a_ub = np.asarray(sel_rows) if sel_rows else None
115         b_ub = np.asarray(sel_rhs) if sel_rhs else None
116         solution = _solve(c, a_eq, b_eq, bounds, a_ub, b_ub)
117         if solution.success:
118             if factor != 1.0 or (keep is not None and len(rows) > keep):
119                 degraded = True
120                 break
121         if solution is None or not solution.success:
122             if x is None:
123                 raise RuntimeError(f"lexicographic LP stage {index} failed: {solution.
124                                     message}")
125                 degraded = True
126                 break
127             x = solution.x
128             result = solution
129             value = float(solution.fun)
130             values.append(value)
131             rows.append(np.asarray(c, dtype=float))
132             rhs.append(value + _lexicographic_slack(value, slack_factor))
133         return x, result, degraded, values
134
135 def _milp_lexicographic(objectives, a_eq, b_eq, bounds, integrality, extra_rows=None,
136                        extra_rhs=None):
137     """MILP 的三层词典序求解；每一层均保留前序目标的最优容差带。"""
138     rows = [np.asarray(r, dtype=float) for r in (extra_rows or [])]
139     rhs = [float(v) for v in (extra_rhs or [])]
140     values: list[float] = []
141     result = None
142     for index, objective in enumerate(objectives):
143         result = _solve_milp(
144             objective,
145             a_eq,
146             b_eq,
147             bounds,
148             integrality,
149             np.asarray(rows) if rows else None,
150             np.asarray(rhs) if rhs else None,
151         )
152         if not result.success:
153             raise RuntimeError(f"lexicographic MILP stage {index} failed: {result.message}")
154         value = float(np.dot(objective, result.x))
155         values.append(value)
156         rows.append(np.asarray(objective, dtype=float))
157         rhs.append(value + _lexicographic_slack(value, 10.0))
158     return result.x, result, values

```

```

158
159
160 def _validate(load_kw, pv_kw, price, storage: StorageParams, soc_initial, soc_terminal):
161     load_kw = np.asarray(load_kw, dtype=float)
162     pv_kw = np.asarray(pv_kw, dtype=float)
163     price = np.asarray(price, dtype=float)
164     if not (load_kw.shape == pv_kw.shape == price.shape):
165         raise ValueError("load/pv/price shape mismatch")
166     t = len(price)
167     if t == 0 or not np.isfinite(np.r_[load_kw, pv_kw, price]).all():
168         raise ValueError("invalid input")
169     if (load_kw < 0).any() or (pv_kw < 0).any() or (price <= 0).any():
170         raise ValueError("physical bounds violated")
171     if not storage.soc_min <= soc_initial <= storage.soc_max:
172         raise ValueError("initial SOC invalid")
173     if soc_terminal is not None and not storage.soc_min <= soc_terminal <= storage.soc_max:
174         raise ValueError("terminal SOC invalid")
175     return load_kw, pv_kw, price, t
176
177
178 def _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage: StorageParams, soc_initial)
179     :
180     """写入母线平衡与 SOC 递推两族等式约束。"""
181     g, ch, dis, soc, spill = _slices(t)
182     for i in range(t):
183         a_eq[i, g.start + i] = 1
184         a_eq[i, ch.start + i] = -1
185         a_eq[i, dis.start + i] = 1
186         a_eq[i, spill.start + i] = -1
187         b_eq[i] = (load_kw[i] - pv_kw[i]) * DT_HOURS
188         row = t + i
189         a_eq[row, soc.start + i] = 1
190         a_eq[row, ch.start + i] = -storage.eta_ch
191         a_eq[row, dis.start + i] = 1.0 / storage.eta_dis
192         if i == 0:
193             b_eq[row] = soc_initial
194         else:
195             a_eq[row, soc.start + i - 1] = -1
196
197 def solve_dispatch(
198     load_kw: np.ndarray,
199     pv_kw: np.ndarray,
200     price: np.ndarray,
201     *,
202     soc_initial: float,
203     soc_terminal: float | None,

```



```

204     storage: StorageParams = DEFAULT_STORAGE,
205 ) -> DispatchResult:
206     """白天/全天的确定性调度 LP:  $\min \sum p \cdot G$ , 词典序消除退化最优解."""
207     load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
208                                         soc_terminal)
209     g, ch, dis, soc, spill = _slices(t)
210     n = 5 * t
211     a_eq = np.zeros((2 * t + (soc_terminal is not None), n))
212     b_eq = np.zeros(a_eq.shape[0])
213     _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
214     if soc_terminal is not None:
215         a_eq[-1, soc.stop - 1] = 1
216         b_eq[-1] = soc_terminal
217     limit = storage.energy_limit
218     bounds = (
219         [(0, None)] * t
220         + [(0, limit)] * t
221         + [(0, limit)] * t
222         + [(storage.soc_min, storage.soc_max)] * t
223         + [(0, float(x) * DT_HOURS) for x in pv_kw]
224     )
225     primary_c = np.zeros(n)
226     primary_c[g] = price
227     throughput_c = np.zeros(n)
228     throughput_c[ch] = 1
229     throughput_c[dis] = 1
230     spill_c = np.zeros(n)
231     spill_c[spill] = 1
232     started = perf_counter()
233     x, tertiary, fell_back, values = _lexicographic(
234         [primary_c, throughput_c, spill_c], a_eq, b_eq, bounds
235     )
236     primary_value = values[0]
237     return DispatchResult(
238         x[g].copy(),
239         x[ch].copy(),
240         x[dis].copy(),
241         x[soc].copy(),
242         x[spill].copy(),
243         float(np.dot(price, x[g])),
244         primary_value,
245         float(x[spill].sum()),
246         float(x[ch].sum() + x[dis].sum()),
247         int(tertiary.status),
248         str(tertiary.message),
249         int(getattr(tertiary, "nit", -1)),
250         perf_counter() - started,

```

```

250         np.asarray(tertiary.eqlin.marginals, dtype=float),
251     )
252
253
254 @dataclass
255 class AdjustmentDispatchResult:
256     dispatch: DispatchResult
257     increase: np.ndarray
258     decrease: np.ndarray
259     settlement_objective: float
260     variable_objective: float
261     adjustment_mask: np.ndarray
262     settlement_mode: str
263     plan_purchase_constant: float
264     lexicographic_fallback: bool = False
265     throughput_objective: float = 0.0
266
267
268 def settlement_cost(plan_grid: np.ndarray, adjusted_grid: np.ndarray, price: np.ndarray,
269                    mode: str) -> float:
270     """按题意直接回代，不含紧急购电费。
271
272     replacement:  $C = p \cdot [\min(G_p, G_a) + 0.5 \cdot (G_p - G_a)^+ + 1.5 \cdot (G_a - G_p)^+]$ 
273     additive :  $C = p \cdot G_p + 0.5 \cdot p \cdot (G_p - G_a)^+ + 1.5 \cdot p \cdot (G_a - G_p)^+$ 
274     """
275     plan = np.asarray(plan_grid, dtype=float)
276     adjusted = np.asarray(adjusted_grid, dtype=float)
277     p = np.asarray(price, dtype=float)
278     increase = np.maximum(adjusted - plan, 0.0)
279     decrease = np.maximum(plan - adjusted, 0.0)
280     if mode == "replacement":
281         return float(np.dot(p, np.minimum(plan, adjusted) + 0.5 * decrease + 1.5 * increase)
282                        )
283     if mode == "additive":
284         return float(np.dot(p, plan) + np.dot(p, 0.5 * decrease + 1.5 * increase))
285     raise ValueError(f"unknown settlement mode: {mode}")
286
287
288 def solve_adjustment_dispatch(
289     load_kw: np.ndarray,
290     pv_kw: np.ndarray,
291     price: np.ndarray,
292     baseline_grid: np.ndarray,
293     adjustment_mask: np.ndarray,
294     *,
295     soc_initial: float,
296     soc_terminal: float | None,

```

```

295     settlement_mode: str = "replacement",
296     storage: StorageParams = DEFAULT_STORAGE,
297 ) -> AdjustmentDispatchResult:
298     """在给定基准计划下联合优化调整购电量与储能。
299
300     显式引入  $inc = (Ga - Gp)^+$  与  $dec = (Gp - Ga)^+$ , 满足  $Ga - Gp = inc - dec$ :
301     replacement 目标:  $\sum p \cdot (Ga + 0.5 \cdot inc + 0.5 \cdot dec)$  (等价于  $\min(Gp, Ga) + 0.5dec + 1.5$ 
302          $inc$ )
303     additive 目标:  $\sum p \cdot Gp + 0.5 \cdot p \cdot dec + 1.5 \cdot p \cdot inc$  ( $p \cdot Gp$  为常数, 最终报告时补
304         回)
305
306     """
307     if settlement_mode not in ("replacement", "additive"):
308         raise ValueError(f"unknown settlement mode: {settlement_mode}")
309     load_kw, pv_kw, price, t = _validate(load_kw, pv_kw, price, storage, soc_initial,
310         soc_terminal)
311     baseline = np.asarray(baseline_grid, dtype=float)
312     mask = np.asarray(adjustment_mask, dtype=bool)
313     if not (baseline.shape == mask.shape == (t,)):
314         raise ValueError("adjustment shape mismatch")
315
316     g, ch, dis, soc, spill = _slices(t)
317     inc = slice(5 * t, 6 * t)
318     dec = slice(6 * t, 7 * t)
319     # 题面字面口径 (additive) 作为主口径时, 连续 LP 的高维最优面可能出现
320     #  $C_t > 0, H_t > 0$  的内部耗散伪解。为使主结果严格满足  $C_t H_t = 0$ ,
321     # additive 模式增加二元变量  $z_t$ ; replacement 保留原 LP 作为敏感性对照。
322     use_milp = settlement_mode == "additive"
323     z = slice(7 * t, 8 * t) if use_milp else slice(7 * t, 7 * t)
324     n = 8 * t if use_milp else 7 * t
325
326     masked = np.flatnonzero(mask)
327     a_eq = np.zeros((2 * t + (soc_terminal is not None) + len(masked), n))
328     b_eq = np.zeros(a_eq.shape[0])
329     _energy_balance_rows(a_eq, b_eq, t, load_kw, pv_kw, storage, soc_initial)
330     if soc_terminal is not None:
331         last = a_eq.shape[0] - 1 - len(masked)
332         a_eq[last, soc.stop - 1] = 1
333         b_eq[last] = soc_terminal
334
335     limit = storage.energy_limit
336     bounds = (
337         [(0, None)] * t
338         + [(0, limit)] * t
339         + [(0, limit)] * t
340         + [(storage.soc_min, storage.soc_max)] * t
341         + [(0, float(x) * DT_HOURS) for x in pv_kw]
342         + [((0, None) if flag else (0, 0)) for flag in mask]

```

```

339         + [(0, None) if flag else (0, 0)] for flag in mask]
340         + [(0, 1)] * t if use_milp else []
341     )
342
343     # 等式约束:  $G_a - inc + dec = G_p$  (仅对允许调整的时段生效)
344     # 必须是等式: 若写成不等式, 当  $G_a < G_p$  时 LP 可取  $inc=dec=0$ , 从而漏付  $0.5p(G_p-G_a)$ 
345     # 的违约费用, 导致目标函数系统性低估向下调整成本、调度不再最优。
346     for k, i in enumerate(masked):
347         row = 2 * t + (1 if soc_terminal is not None else 0) + k
348         a_eq[row, g.start + i] = 1.0
349         a_eq[row, inc.start + i] = -1.0
350         a_eq[row, dec.start + i] = 1.0
351         b_eq[row] = baseline[i]
352     adjustment_rows: list[np.ndarray] = []
353     adjustment_rhs: list[float] = []
354
355     primary_c = np.zeros(n)
356     if settlement_mode == "replacement":
357         primary_c[g] = price
358         primary_c[inc] = np.where(mask, 0.5 * price, 0.0)
359         primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
360         constant = 0.0
361     else:
362         primary_c[inc] = np.where(mask, 1.5 * price, 0.0)
363         primary_c[dec] = np.where(mask, 0.5 * price, 0.0)
364         # additive 口径下  $G_a$  不进目标, 最优面维度较高;  $G_a$  的实际取值由母线平衡与
365         #  $inc/dec$  的等式关系共同决定, 退化解再由后续的吞吐量层消除, 无需人工微扰。
366         constant = float(np.sum(price[mask] * baseline[mask]))
367
368     throughput_c = np.zeros(n)
369     throughput_c[ch] = 1
370     throughput_c[dis] = 1
371     throughput_c[inc] = 1
372     throughput_c[dec] = 1
373     spill_c = np.zeros(n)
374     spill_c[spill] = 1
375     started = perf_counter()
376     if use_milp:
377         #  $C_t \leq Ebar * z_t, H_t \leq Ebar * (1 - z_t)$  精确表达充放电互斥。
378         mutual_rows = np.zeros((2 * t, n))
379         mutual_rhs = np.zeros(2 * t)
380         for i in range(t):
381             mutual_rows[2 * i, ch.start + i] = 1.0
382             mutual_rows[2 * i, z.start + i] = -limit
383             mutual_rows[2 * i + 1, dis.start + i] = 1.0
384             mutual_rows[2 * i + 1, z.start + i] = limit
385             mutual_rhs[2 * i + 1] = limit

```

```

386     integrality = np.zeros(n, dtype=int)
387     integrality[z] = 1
388     # 二元互斥已从可行域中彻底排除“边充边放”，因此主费用目标一次求解即可；
389     # replacement 的连续 LP 仍保留三层词典序以选择稳定顶点。
390     tertiary = _solve_milp(
391         primary_c,
392         a_eq,
393         b_eq,
394         bounds,
395         integrality,
396         mutual_rows,
397         mutual_rhs,
398     )
399     if not tertiary.success:
400         raise RuntimeError(f"adjustment MILP failed: {tertiary.message}")
401     x = tertiary.x
402     values = [
403         float(np.dot(primary_c, x)),
404         float(np.dot(throughput_c, x)),
405         float(np.dot(spill_c, x)),
406     ]
407     fell_back = False
408     else:
409         x, tertiary, fell_back, values = _lexicographic(
410             [primary_c, throughput_c, spill_c],
411             a_eq,
412             b_eq,
413             bounds,
414             adjustment_rows,
415             adjustment_rhs,
416         )
417     primary_value = values[0]
418     adjusted = x[g].copy()
419     increase_vec = x[inc].copy()
420     decrease_vec = x[dec].copy()
421     if settlement_mode == "replacement":
422         settlement = float(np.dot(primary_c, x))
423     else:
424         settlement = constant + float(np.dot(primary_c, x))
425     dispatch = DispatchResult(
426         adjusted,
427         x[ch].copy(),
428         x[dis].copy(),
429         x[soc].copy(),
430         x[spill].copy(),
431         settlement,
432         float(primary_value + constant),

```

```

433         float(x[spill].sum()),
434         float(x[ch].sum() + x[dis].sum()),
435         int(tertiary.status),
436         str(tertiary.message),
437         int(getattr(tertiary, "nit", getattr(tertiary, "mip_node_count", -1)) or 0),
438         perf_counter() - started,
439         (
440             np.zeros(a_eq.shape[0], dtype=float)
441             if use_milp
442             else np.asarray(tertiary.eqlin.marginals, dtype=float)
443         ),
444     )
445     return AdjustmentDispatchResult(
446         dispatch,
447         increase_vec,
448         decrease_vec,
449         settlement,
450         float(np.dot(primary_c, x)),
451         mask.copy(),
452         settlement_mode,
453         constant,
454         bool(fell_back),
455         float(values[1]) if len(values) > 1 else float("nan"),
456     )

```

B.4 光伏日前预测与风险定价 (forecast.py)

Listing 4:

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  from .io_data import DT_HOURS, SLOTS_PER_DAY
8
9  METHOD_WINDOWS = {"persistence": 1, "mean7": 7, "mean14": 14, "mean30": 30, "weighted14":
10                  14, "weighted30": 30}
11
12  QUANTILES = (0.80, 0.85, 0.90, 0.95)
13
14  @dataclass(frozen=True)
15  class ForecastDecision:
16      day_index: int
17      name: str

```

```

17     method: str
18     quantile: float
19     margin_kw: float
20     historical_score: float | None
21     history_days_used: int
22     forecast_kw: np.ndarray
23     candidates: dict[str, np.ndarray]
24     bases: dict[str, np.ndarray]
25     margins: dict[str, float]
26
27
28     class OnlinePVForecaster:
29         """光伏日前预测器，全部输入只来自第 d 天之前的实际光伏。
30
31         risk_mode = "asymmetric" (当前主方案)
32         在基预测上叠加历史残差分位数下修边际，候选按 4:1 非对称费用代理选型。
33         risk_mode = "point"
34         不做安全下修 (margin_kw = 0)，候选按对称 MAE 选型，用作对照。
35         """
36
37         def __init__(
38             self,
39             price: np.ndarray,
40             score_window: int = 30,
41             residual_window: int = 30,
42             risk_mode: str = "asymmetric",
43             prior_pv: np.ndarray | None = None,
44         ) -> None:
45             if risk_mode not in ("asymmetric", "point"):
46                 raise ValueError(f"unknown risk_mode: {risk_mode}")
47             self.price = np.asarray(price, dtype=float)
48             if self.price.shape != (SLOTS_PER_DAY,):
49                 raise ValueError("price must contain 144 slots")
50             self.prior_pv = None if prior_pv is None else np.asarray(prior_pv, dtype=float)
51             self.score_window = score_window
52             self.residual_window = residual_window
53             self.risk_mode = risk_mode
54             self.actual_history: list[np.ndarray] = []
55             self.residual_days: dict[str, list[np.ndarray]] = {m: [] for m in METHOD_WINDOWS}
56             self.score_days: dict[str, list[float]] = {self._name(m, q): [] for m in
57                 METHOD_WINDOWS for q in QUANTILES}
58             if risk_mode == "point":
59                 self.score_days = {m: [] for m in METHOD_WINDOWS}
60
61         @staticmethod
62         def _name(method: str, q: float) -> str:
63             return f"{method}_q{int(round(q * 100)):02d}"

```

```

63
64 def _base(self, method: str) -> np.ndarray:
65     if not self.actual_history:
66         # 尚无历史观测时使用附件 1 给出的典型日光伏预测作为先验
67         return np.zeros(SLOTS_PER_DAY) if self.prior_pv is None else np.clip(self.
            prior_pv, 0.0, None)
68     n = min(METHOD_WINDOWS[method], len(self.actual_history))
69     recent = np.stack(self.actual_history[-n:])
70     if method == "persistence":
71         return recent[-1].copy()
72     if method.startswith("weighted"):
73         return np.average(recent, axis=0, weights=np.arange(1, n + 1, dtype=float))
74     return recent.mean(axis=0)
75
76 def _margin(self, method: str, q: float) -> float:
77     days = self.residual_days[method][-self.residual_window :]
78     if not days:
79         return 0.0
80     residual = np.concatenate(days)
81     return 0.0 if residual.size == 0 else max(0.0, float(np.quantile(residual, q)))
82
83 def predict(self) -> ForecastDecision:
84     d = len(self.actual_history)
85     bases = {m: self._base(m) for m in METHOD_WINDOWS}
86     candidates: dict[str, np.ndarray] = {}
87     margins: dict[str, float] = {}
88     if self.risk_mode == "asymmetric":
89         for method, base in bases.items():
90             for q in QUANTILES:
91                 name = self._name(method, q)
92                 margins[name] = self._margin(method, q)
93                 candidates[name] = np.clip(base - margins[name], 0.0, None)
94             default = self._name("persistence", 0.80)
95             quantile = 0.80
96     else:
97         for method, base in bases.items():
98             margins[method] = 0.0
99             candidates[method] = np.clip(base, 0.0, None)
100             default = "persistence"
101             quantile = 0.0
102     eligible = [n for n, s in self.score_days.items() if s]
103     if d < 7 or not eligible:
104         selected = default
105         hist = None
106     else:
107         scores = {n: float(np.mean(self.score_days[n][-self.score_window :])) for n in
            eligible}

```



```

108         selected = min(scores, key=lambda n: (scores[n], n))
109         hist = scores[selected]
110         if self.risk_mode == "asymmetric":
111             method, q_text = selected.rsplitt("_q", 1)
112             q = int(q_text) / 100.0
113         else:
114             method = selected
115             q = 0.0
116         return ForecastDecision(
117             d,
118             selected,
119             method,
120             q if self.risk_mode == "asymmetric" else 0.0,
121             margins[selected],
122             hist,
123             d,
124             candidates[selected].copy(),
125             candidates,
126             bases,
127             margins,
128         )
129
130     def observe(self, decision: ForecastDecision, actual_kw: np.ndarray, price: np.ndarray |
131         None = None) -> dict[str, float]:
132         actual = np.asarray(actual_kw, dtype=float)
133         if actual.shape != (SLOTS_PER_DAY,) or decision.day_index != len(self.actual_history
134             ):
135             raise ValueError("forecast observation out of sequence")
136         score_price = self.price if price is None else np.asarray(price, dtype=float)
137         if score_price.shape != (SLOTS_PER_DAY,):
138             raise ValueError("score price must contain 144 slots")
139         result: dict[str, float] = {}
140         for name, forecast in decision.candidates.items():
141             error = forecast - actual
142             if self.risk_mode == "asymmetric":
143                 score = float(np.sum(score_price * (4 * np.maximum(error, 0) + np.maximum(-
144                     error, 0))) * DT_HOURS)
145             else:
146                 score = float(np.mean(np.abs(error)))
147             self.score_days[name].append(score)
148             result[name] = score
149         for method, base in decision.bases.items():
150             mask = (actual > 1e-9) | (base > 1e-9)
151             self.residual_days[method].append((base - actual)[mask].copy())
152         self.actual_history.append(actual.copy())
153         return result

```

B.5 因果负载预测 (load_forecast.py)

Listing 5:

```
1  """严格因果的负载预测模块。
2
3  信息集约束（本模块唯一允许读取的数据）：
4      * 第 d 天第 n 个决策点时点（n = 0/36/72/108），只允许使用
5          - 已经完整观测完的日期 d' < d 的附件 2 真实负载；
6          - 第 d 天已经执行完的时段 [0, n) 的真实负载；
7      * 任何时刻都不得读取第 d 天时段 >= n 的真实负载，也不得读取 d' > d 的数据。
8
9  预测器采用稳定、可解释的滚动历史方法，并在历史执行块上滚动在线选型
10     （选型同样只使用已经发生的误差）。
11 """
12 from __future__ import annotations
13
14 from dataclasses import dataclass
15
16 import numpy as np
17
18 from .io_data import DT_HOURS, SLOTS_PER_DAY
19
20 # 方法名 -> 回看天数 (weekday_mean 使用 4 个同 weekday 的样本)
21 METHOD_WINDOWS: dict[str, int] = {
22     "persistence": 1,
23     "mean7": 7,
24     "mean14": 14,
25     "mean30": 30,
26     "weighted14": 14,
27     "weighted30": 30,
28     "weekday_mean": 28,
29 }
30 BASE_METHODS = tuple(METHOD_WINDOWS)
31 NODE_SLOTS = (0, 36, 72, 108)
32
33
34 @dataclass(frozen=True)
35 class LoadForecastDecision:
36     """一次负载预测决策的完整留痕。"""
37
38     day_index: int
39     node: int
40     name: str # 选中的候选名 (含 _raw / _offset 后缀)
41     method: str # 基方法名
42     corrected: bool # 是否施加了日内水平偏移校正
43     offset_kw: float
```

```

44     historical_score: float | None
45     history_days_used: int
46     horizon: int
47     forecast_kw: np.ndarray
48     candidates: dict[str, np.ndarray]
49     weekday_of_target: int
50
51
52 def _daily_window_shape(history: list[np.ndarray], method: str, weekday: int) -> np.ndarray:
53     """由历史（严格早于目标日）给出目标日 144 个时段的基准预测形状。"""
54     if not history:
55         return np.zeros(SLOTS_PER_DAY, dtype=float)
56     if method == "weekday_mean":
57         same = [h for h, w in history if w == weekday]
58         if len(same) >= 2:
59             recent = np.stack(same[-4:])
60             return recent.mean(axis=0)
61         method = "mean7" # 同 weekday 样本不足时退化为 7 日均值
62     n = min(METHOD_WINDOWS[method], len(history))
63     recent = np.stack([h for h, _ in history[-n:]])
64     if method == "persistence":
65         return recent[-1].copy()
66     if method.startswith("weighted"):
67         weights = np.arange(1, n + 1, dtype=float)
68         return np.average(recent, axis=0, weights=weights)
69     return recent.mean(axis=0)
70
71
72 class OnlineLoadForecaster:
73     """因果在线负载预测器（滚动在线选型）。"""
74
75     def __init__(self, score_window: int = 30, prior_kw: np.ndarray | None = None) -> None:
76         self.score_window = score_window
77         # （负载曲线，weekday）只在日终写入，保证 d 日决策读不到 d 日数据
78         self.history: list[tuple[np.ndarray, int]] = []
79         # 附件 1 给出的典型日负载作为 1 月 1 日之前的先验（不占用真实历史日）
80         self.prior_days = 0
81         if prior_kw is not None:
82             prior = np.asarray(prior_kw, dtype=float)
83             if prior.shape != (SLOTS_PER_DAY,):
84                 raise ValueError("prior load must have 144 slots")
85             self.history.append((prior.copy(), -1))
86             self.prior_days = 1
87         # 第 d 天已执行的时段观测，长度 <= node
88         self.partial: np.ndarray = np.zeros(SLOTS_PER_DAY, dtype=float)
89         self.partial_len: int = 0
90         self.score_days: dict[tuple[int, str], list[float]] = {}

```

```

91
92 @property
93 def observed_days(self) -> int:
94     """真实历史天数（不含先验）。"""
95     return len(self.history) - self.prior_days
96
97 # ----- #
98 # 内部工具
99 # ----- #
100 def _target_weekday(self, day_index: int, day_offset: int) -> int:
101     """目标日期相对 2025-01-01 的星期（0=周一）。"""
102     return (day_index + day_offset) % 7
103
104 def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
    ndarray:
105     """给出 [d, node] 起 horizon 个时段的基准预测（可跨日）。"""
106     wd_today = self._target_weekday(day_index, 0)
107     shape_today = _daily_window_shape(self.history, method, wd_today)
108     head = shape_today[node:]
109     if len(head) >= horizon:
110         return head[:horizon].copy()
111     wd_next = self._target_weekday(day_index, 1)
112     shape_next = _daily_window_shape(self.history, method, wd_next)
113     tail = horizon - len(head)
114     while len(shape_next) < tail:
115         shape_next = np.concatenate([shape_next, shape_next])
116     return np.concatenate([head, shape_next[:tail]])
117
118 def _offset(self, method: str, day_index: int, node: int) -> float:
119     """用第 d 天已观测的 [node-k, node) 与基准预测之差估计水平偏移。"""
120     if node <= 0 or self.partial_len <= 0:
121         return 0.0
122     k = min(36, node, self.partial_len)
123     if k < 6:
124         return 0.0
125     observed = self.partial[node - k : node]
126     base = _daily_window_shape(self.history, method, self._target_weekday(day_index, 0))
        [node - k : node]
127     return float(np.mean(observed - base))
128
129 def _candidate_names(self, node: int) -> tuple[str, ...]:
130     if node == 0:
131         return BASE_METHODS
132     return tuple(f"{m}_{suffix}" for m in BASE_METHODS for suffix in ("raw", "offset"))
133
134 def _build(self, name: str, day_index: int, node: int, horizon: int) -> tuple[np.ndarray
    , str, bool, float]:

```

```

135         if name.endswith("_offset"):
136             method, corrected = name[: -len("_offset")], True
137         elif name.endswith("_raw"):
138             method, corrected = name[: -len("_raw")], False
139         else:
140             method, corrected = name, False
141         base = self._base_horizon(method, day_index, node, horizon)
142         offset = self._offset(method, day_index, node) if corrected else 0.0
143         return np.clip(base + offset, 0.0, None), method, corrected, offset
144
145         # ----- #
146         # 对外接口
147         # ----- #
148     def predict(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY) ->
        LoadForecastDecision:
149         """在第 d 天第 node 个时段之前生成未来 horizon 个时段的负载预测。"""
150         if node not in NODE_SLOTS:
151             raise ValueError(f"node must be one of {NODE_SLOTS}")
152         names = self._candidate_names(node)
153         candidates: dict[str, np.ndarray] = {}
154         meta: dict[str, tuple[str, bool, float]] = {}
155         for name in names:
156             forecast, method, corrected, offset = self._build(name, day_index, node, horizon
                )
157             candidates[name] = forecast
158             meta[name] = (method, corrected, offset)
159
160         eligible = [n for n in names if self.score_days.get((node, n))]
161         history_days = self._score_history_days(node)
162         default = "persistence" if node == 0 else "persistence_raw"
163         if history_days < 7 or not eligible:
164             selected, hist = default, None
165         else:
166             scores = {
167                 n: float(np.mean(self.score_days[(node, n)][-self.score_window :])) for n in
                    eligible
168             }
169             selected = min(scores, key=lambda n: (scores[n], n))
170             hist = scores[selected]
171         method, corrected, offset = meta[selected]
172         return LoadForecastDecision(
173             day_index=day_index,
174             node=node,
175             name=selected,
176             method=method,
177             corrected=corrected,
178             offset_kw=offset,

```

```

179         historical_score=hist,
180         history_days_used=self.observed_days,
181         horizon=horizon,
182         forecast_kw=candidates[selected].copy(),
183         candidates=candidates,
184         weekday_of_target=self._target_weekday(day_index, 0),
185     )
186
187     def _score_history_days(self, node: int) -> int:
188         return max((len(v) for (n, _), v in self.score_days.items() if n == node), default=0)
189
190     def observe_block(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray) -> dict[str, float]:
191         """执行块结束后回填误差，用于滚动在线选型。"""
192         actual = np.asarray(actual_block_kw, dtype=float)
193         if actual.ndim != 1:
194             raise ValueError("actual block must be 1-D")
195         out: dict[str, float] = {}
196         for name, forecast in decision.candidates.items():
197             segment = forecast[: len(actual)]
198             mae = float(np.mean(np.abs(segment - actual)))
199             self.score_days.setdefault((decision.node, name), []).append(mae)
200             out[name] = mae
201         return out
202
203     def observe_partial(self, node: int, actual_slots: np.ndarray) -> None:
204         """执行块完成后写入当天已观测前缀，供后续节点的水平校正使用。"""
205         if actual_slots.shape != (node,) and node != 0:
206             raise ValueError("partial block shape mismatch")
207         self.partial[:node] = actual_slots
208         self.partial_len = node
209
210     def commit_day(self, actual_full_day_kw: np.ndarray, day_index: int) -> None:
211         """日终把完整观测写入历史；此后该日才可被后续日期使用。"""
212         actual = np.asarray(actual_full_day_kw, dtype=float)
213         if actual.shape != (SLOTS_PER_DAY,):
214             raise ValueError("daily load must have 144 slots")
215         self.history.append((actual.copy(), self._target_weekday(day_index, 0)))
216         self.partial = np.zeros(SLOTS_PER_DAY, dtype=float)
217         self.partial_len = 0
218
219     # ----- #
220     def log_row(self, decision: LoadForecastDecision, actual_block_kw: np.ndarray, date_text: str) -> dict:
221         """任务书要求字段：date / decision_time / history_days_used / method / mae / rmse / 电量误差。"""

```

```

222     actual = np.asarray(actual_block_kw, dtype=float)
223     forecast = decision.forecast_kw[: len(actual)]
224     error = forecast - actual
225     slot = int(decision.node * 10) // 60
226     minute = (decision.node * 10) % 60
227     label = f"{slot:02d}:{minute:02d}"
228     return {
229         "date": date_text,
230         "decision_time": label,
231         "history_days_used": decision.history_days_used,
232         "candidate": decision.name,
233         "method": decision.method,
234         "offset_corrected": decision.corrected,
235         "offset_kw": decision.offset_kw,
236         "mae_kw": float(np.mean(np.abs(error))),
237         "rmse_kw": float(np.sqrt(np.mean(error**2))),
238         "daily_energy_error_kwh": float(np.sum(error) * DT_HOURS),
239         "block_energy_error_kwh": float(np.sum(error) * DT_HOURS),
240         "historical_score": decision.historical_score,
241     }

```

B.6 因果电价预测 (price_forecast.py)

Listing 6:

```

1  """严格因果的外网电价预测模块（用于问题四）。
2
3  信息集约束：
4      * 第 d 天第 n 个决策点，只允许使用附件 4 中 d' < d 的全部真实价格，
5        以及第 d 天已经过去时段 [0, n) 的真实价格；
6      * 不得读取第 d 天时段 >= n 的附件 4 真实价格。
7
8  价格与负载的形态差别：电价没有"日内水平漂移"的强物理机制，因此
9  候选只做基预测，不额外做偏移校正；但保留日内已观测前缀参与选型信息。
10 """
11 from __future__ import annotations
12
13 from dataclasses import dataclass
14
15 import numpy as np
16
17 from .io_data import DT_HOURS, SLOTS_PER_DAY
18 from .load_forecast import NODE_SLOTS
19
20 METHOD_WINDOWS: dict[str, int] = {
21     "persistence": 1,

```

```

22     "mean7": 7,
23     "mean14": 14,
24     "mean30": 30,
25     "weighted14": 14,
26     "weighted30": 30,
27     "weekday_mean": 28,
28 }
29 BASE_METHODS = tuple(METHOD_WINDOWS)
30
31
32 @dataclass(frozen=True)
33 class PriceForecastDecision:
34     day_index: int
35     node: int
36     name: str
37     method: str
38     historical_score: float | None
39     history_days_used: int
40     horizon: int
41     forecast_price: np.ndarray
42     candidates: dict[str, np.ndarray]
43
44
45 def _window_shape(history: list[tuple[np.ndarray, int]], method: str, weekday: int) -> np.
    ndarray:
46     if not history:
47         return np.full(SLOTS_PER_DAY, np.nan, dtype=float)
48     if method == "weekday_mean":
49         same = [h for h, w in history if w == weekday]
50         if len(same) >= 2:
51             return np.stack(same[-4:]).mean(axis=0)
52         method = "mean7"
53     n = min(METHOD_WINDOWS[method], len(history))
54     recent = np.stack([h for h, _ in history[-n:]])
55     if method == "persistence":
56         return recent[-1].copy()
57     if method.startswith("weighted"):
58         weights = np.arange(1, n + 1, dtype=float)
59         return np.average(recent, axis=0, weights=weights)
60     return recent.mean(axis=0)
61
62
63 class OnlinePriceForecaster:
64     """因果在线电价预测器。"""
65
66     def __init__(self, score_window: int = 30, prior_price: np.ndarray | None = None) ->
        None:

```



```

67     self.score_window = score_window
68     self.history: list[tuple[np.ndarray, int]] = []
69     self.prior_days = 0
70     if prior_price is not None:
71         prior = np.asarray(prior_price, dtype=float)
72         if prior.shape != (SLOTS_PER_DAY,):
73             raise ValueError("prior price must have 144 slots")
74         self.history.append((prior.copy(), -1))
75         self.prior_days = 1
76     self.partial_len: int = 0
77     self.score_days: dict[tuple[int, str], list[float]] = {}
78
79     @property
80     def observed_days(self) -> int:
81         return len(self.history) - self.prior_days
82
83     def _weekday(self, day_index: int, offset: int) -> int:
84         return (day_index + offset) % 7
85
86     def _base_horizon(self, method: str, day_index: int, node: int, horizon: int) -> np.
        ndarray:
87         head = _window_shape(self.history, method, self._weekday(day_index, 0))[node:]
88         if len(head) >= horizon:
89             return head[:horizon].copy()
90         tail_len = horizon - len(head)
91         tail = _window_shape(self.history, method, self._weekday(day_index, 1))
92         while len(tail) < tail_len:
93             tail = np.concatenate([tail, tail])
94         return np.concatenate([head, tail[:tail_len]])
95
96     def _fallback(self) -> np.ndarray:
97         """完全无历史时退化为最近一次已知价格水平。"""
98         if not self.history:
99             return np.full(SLOTS_PER_DAY, 0.5, dtype=float)
100         return self.history[-1][0].copy()
101
102     def predict_horizon(self, day_index: int, node: int = 0, horizon: int = SLOTS_PER_DAY)
        -> PriceForecastDecision:
103         if node not in NODE_SLOTS:
104             raise ValueError(f"node must be one of {NODE_SLOTS}")
105         candidates: dict[str, np.ndarray] = {}
106         for method in BASE_METHODS:
107             horizon_values = self._base_horizon(method, day_index, node, horizon)
108             if not np.isfinite(horizon_values).all():
109                 horizon_values = np.where(np.isfinite(horizon_values), horizon_values, self.
                    _fallback()[node % SLOTS_PER_DAY])
110             candidates[method] = np.clip(horizon_values, 1e-6, None)

```

```

111     eligible = [n for n in BASE_METHODS if self.score_days.get((node, n))]
112     history_days = max((len(v) for (n, _), v in self.score_days.items() if n == node),
113                        default=0)
114     if history_days < 7 or not eligible:
115         selected, hist = "persistence", None
116     else:
117         scores = {n: float(np.mean(self.score_days[(node, n)][-self.score_window :]))
118                  for n in eligible}
119         selected = min(scores, key=lambda n: (scores[n], n))
120         hist = scores[selected]
121     return PriceForecastDecision(
122         day_index=day_index,
123         node=node,
124         name=selected,
125         method=selected,
126         historical_score=hist,
127         history_days_used=self.observed_days,
128         horizon=horizon,
129         forecast_price=candidates[selected].copy(),
130         candidates=candidates,
131     )
132
133 def observe_block(self, decision: PriceForecastDecision, actual_block: np.ndarray) ->
134     dict[str, float]:
135     actual = np.asarray(actual_block, dtype=float)
136     out: dict[str, float] = {}
137     for name, forecast in decision.candidates.items():
138         segment = forecast[: len(actual)]
139         mae = float(np.mean(np.abs(segment - actual)))
140         self.score_days.setdefault((decision.node, name), []).append(mae)
141         out[name] = mae
142     return out
143
144 def observe_partial(self, node: int) -> None:
145     self.partial_len = node
146
147 def commit_day(self, actual_full_day: np.ndarray, day_index: int) -> None:
148     actual = np.asarray(actual_full_day, dtype=float)
149     if actual.shape != (SLOTS_PER_DAY,):
150         raise ValueError("daily price must have 144 slots")
151     self.history.append((actual.copy(), self._weekday(day_index, 0)))
152     self.partial_len = 0
153
154 def log_row(self, decision: PriceForecastDecision, actual_block: np.ndarray, date_text:
155     str) -> dict:
156     actual = np.asarray(actual_block, dtype=float)
157     forecast = decision.forecast_price[: len(actual)]

```

```

154     error = forecast - actual
155     slot = int(decision.node * 10) // 60
156     minute = (decision.node * 10) % 60
157     return {
158         "date": date_text,
159         "decision_time": f"{slot:02d}:{minute:02d}",
160         "history_days_used": decision.history_days_used,
161         "method": decision.method,
162         "price_mae": float(np.mean(np.abs(error))),
163         "price_rmse": float(np.sqrt(np.mean(error**2))),
164         "price_energy_weighted_error": float(
165             np.sum(np.abs(error) * actual) / max(np.sum(actual), 1e-9)
166         ),
167         "daily_price_energy_weighted_error": float(
168             np.sum(np.abs(error) * actual) * DT_HOURS
169         ),
170         "historical_score": decision.historical_score,
171     }

```

B.7 实时储能反馈与执行 (settle.py)

Listing 7:

```

1  from __future__ import annotations
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  from .io_data import DT_HOURS
8  from .lp_core import DispatchResult
9  from .params import DEFAULT_STORAGE, StorageParams
10
11  TOL = 1e-10
12
13
14  @dataclass(frozen=True)
15  class ExecutionResult:
16      grid: np.ndarray
17      charge: np.ndarray
18      discharge: np.ndarray
19      soc: np.ndarray
20      curtail: np.ndarray
21      emergency: np.ndarray
22
23

```

```

24 def execute_with_realtime_storage_feedback(
25     plan: DispatchResult,
26     load_kw: np.ndarray,
27     actual_pv_kw: np.ndarray,
28     *,
29     soc_initial: float,
30     storage: StorageParams = DEFAULT_STORAGE,
31 ) -> ExecutionResult:
32     """实时储能反馈 (realtime_feedback = ON) 。
33
34     真实光伏低于计划时: 先取消充电 → 再增加放电 → 最后紧急购电;
35     真实光伏高于计划时: 先减少放电 → 再增加充电 → 最后弃光。
36     每步只使用当前时刻已发生的真实光伏与当前 SOC, 不使用任何未来信息。
37
38     load = np.asarray(load_kw, dtype=float)
39     pv = np.asarray(actual_pv_kw, dtype=float)
40     n = len(load)
41     if pv.shape != load.shape or len(plan.grid) != n:
42         raise ValueError("execution shape mismatch")
43     limit = storage.energy_limit
44     c_out = np.zeros(n)
45     h_out = np.zeros(n)
46     soc = np.zeros(n)
47     spill = np.zeros(n)
48     emergency = np.zeros(n)
49     state = float(soc_initial)
50     for t in range(n):
51         c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
52             eta_ch))
53         h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *
54             storage.eta_dis))
55         net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
56         if net < -TOL:
57             deficit = -net
58             reduce = min(c, deficit)
59             c -= reduce
60             deficit -= reduce
61             max_h = max(0.0, (state + storage.eta_ch * c - storage.soc_min) * storage.
62                 eta_dis)
63             add = min(deficit, limit - h, max(0.0, max_h - h))
64             h += add
65             deficit -= add
66             emergency[t] = max(0.0, deficit)
67         elif net > TOL:
68             surplus = net
69             reduce = min(h, surplus)
70             h -= reduce

```

```

68         surplus -= reduce
69         max_c = max(0.0, (storage.soc_max - (state - h / storage.eta_dis)) / storage.
            eta_ch)
70         add = min(surplus, limit - c, max(0.0, max_c - c))
71         c += add
72         surplus -= add
73         spill[t] = max(0.0, surplus)
74         state = state + storage.eta_ch * c - h / storage.eta_dis
75         if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
76             raise RuntimeError(f"SOC invalid at {t}: {state}")
77         c_out[t] = c
78         h_out[t] = h
79         soc[t] = min(storage.soc_max, max(storage.soc_min, state))
80     return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
81
82
83 def execute_fixed_storage_plan(
84     plan: DispatchResult,
85     load_kw: np.ndarray,
86     actual_pv_kw: np.ndarray,
87     *,
88     soc_initial: float,
89     storage: StorageParams = DEFAULT_STORAGE,
90 ) -> ExecutionResult:
91     """固定储能计划执行 (realtime_feedback = OFF) 。
92
93     储能充放电严格等于计划值，只保留 SOC 物理边界的被动钳位：
94     真实不足部分全部进入 5 倍电价紧急购电，真实富余部分全部弃光。
95     该口径用于度量 10 min 实时储能 recourse 对总费用的贡献。
96     """
97     load = np.asarray(load_kw, dtype=float)
98     pv = np.asarray(actual_pv_kw, dtype=float)
99     n = len(load)
100     if pv.shape != load.shape or len(plan.grid) != n:
101         raise ValueError("execution shape mismatch")
102     limit = storage.energy_limit
103     c_out = np.zeros(n)
104     h_out = np.zeros(n)
105     soc = np.zeros(n)
106     spill = np.zeros(n)
107     emergency = np.zeros(n)
108     state = float(soc_initial)
109     for t in range(n):
110         # 被动钳位：计划值超出当前 SOC 可达范围时按边界截断（不主动做能量补偿）
111         c = min(float(plan.charge[t]), limit, max(0.0, (storage.soc_max - state) / storage.
            eta_ch))
112         h = min(float(plan.discharge[t]), limit, max(0.0, (state - storage.soc_min) *

```

```

        storage.eta_dis))
113     net = float(plan.grid[t] + pv[t] * DT_HOURS + h - load[t] * DT_HOURS - c)
114     if net < 0:
115         emergency[t] = -net
116     elif net > 0:
117         spill[t] = net
118     state = state + storage.eta_ch * c - h / storage.eta_dis
119     if not storage.soc_min - 1e-6 <= state <= storage.soc_max + 1e-6:
120         raise RuntimeError(f"SOC invalid at {t}: {state}")
121     c_out[t] = c
122     h_out[t] = h
123     soc[t] = min(storage.soc_max, max(storage.soc_min, state))
124     return ExecutionResult(plan.grid.copy(), c_out, h_out, soc, spill, emergency)
125
126
127 def execute_plan(
128     plan: DispatchResult,
129     load_kw: np.ndarray,
130     actual_pv_kw: np.ndarray,
131     *,
132     soc_initial: float,
133     realtime_feedback: bool = True,
134     storage: StorageParams = DEFAULT_STORAGE,
135 ) -> ExecutionResult:
136     if realtime_feedback:
137         return execute_with_realtime_storage_feedback(
138             plan, load_kw, actual_pv_kw, soc_initial=soc_initial, storage=storage
139         )
140     return execute_fixed_storage_plan(plan, load_kw, actual_pv_kw, soc_initial=soc_initial,
        storage=storage)

```

B.8 问题一求解 (q1.py)

Listing 8:

```

1  from __future__ import annotations
2  import csv,json,platform,sys
3  from pathlib import Path
4  import numpy as np
5  import scipy
6  import openpyxl
7  from .export import export_result1
8  from .io_data import DEFAULT_DATA_ROOT,DT_HOURS,read_attachment1,template_path
9  from .lp_core import ETA_CH,ETA_DIS,SOC_MAX,SOC_MIN,solve_dispatch
10
11 ROOT=Path(__file__).resolve().parents[1]; OUTPUT=ROOT/"outputs"/"q1"

```

```

12
13 def run_q1(*,data_root:Path=DEFAULT_DATA_ROOT)->dict:
14     data=read_attachment1(data_root)
15     result=solve_dispatch(data.load_kw,data.pv_kw,data.price,soc_initial=6000,soc_terminal
        =6000)
16     balance=result.grid+data.pv_kw*DT_HOURS+result.discharge-data.load_kw*DT_HOURS-result.
        charge-result.curtail
17     previous=np.r_[6000.0,result.soc[:-1]]
18     soc_res=result.soc-(previous+ETA_CH*result.charge-result.discharge/ETA_DIS)
19     no_storage_grid=np.maximum((data.load_kw-data.pv_kw)*DT_HOURS,0)
20     metrics={"cost_yuan":result.objective,"primary_cost_yuan":result.primary_objective,
21             "primary_cost_gap_yuan":result.objective-result.primary_objective,
22             "grid_energy_kwh":float(result.grid.sum()),"charge_energy_kwh":float(result.charge.
                sum()),
23             "discharge_energy_kwh":float(result.discharge.sum()),"curtailment_energy_kwh":float(
                result.curtail.sum()),
24             "no_storage_cost_yuan":float(np.dot(data.price,no_storage_grid)),
25             "savings_yuan":float(np.dot(data.price,no_storage_grid)-result.objective),
26             "savings_percent":float(100*(np.dot(data.price,no_storage_grid)-result.objective)/np
                .dot(data.price,no_storage_grid)),
27             "initial_soc_kwh":6000.0,"terminal_soc_kwh":float(result.soc[-1]),
28             "min_soc_kwh":float(result.soc.min()),"max_soc_kwh":float(result.soc.max()),
29             "max_balance_residual":float(np.max(np.abs(balance))),"max_soc_residual":float(np.
                max(np.abs(soc_res))),
30             "max_simultaneous_charge_discharge":float(np.max(np.minimum(result.charge,result.
                discharge))),
31             "solver_status":result.status,"solver_message":result.message,"iterations":result.
                nit,
32             "solve_seconds":result.solve_seconds}
33     metrics["passed"]=bool(metrics["max_balance_residual"]<1e-7 and metrics["
        max_soc_residual"]<1e-7 and
34         abs(result.soc[-1]-6000)<1e-7 and metrics["min_soc_kwh"]>=SOC_MIN-1e-7 and
35         metrics["max_soc_kwh"]<=SOC_MAX+1e-7 and metrics["max_simultaneous_charge_discharge"
        ]<1e-5)
36     OUTPUT.mkdir(parents=True,exist_ok=True)
37     with (OUTPUT/"solution.csv").open("w",newline="",encoding="utf-8-sig") as f:
38         fields=["slot","label","price","load_kw","pv_kw","grid_kwh","charge_kwh","
            discharge_kwh","soc_kwh","curtail_kwh"]
39         writer=csv.DictWriter(f,fieldnames=fields); writer.writeheader()
40         for t in range(144):
41             writer.writerow({"slot":t+1,"label":data.labels[t],"price":data.price[t],"
                load_kw":data.load_kw[t],
42                 "pv_kw":data.pv_kw[t],"grid_kwh":result.grid[t],"charge_kwh":result.charge[t]
                ],
43                 "discharge_kwh":result.discharge[t],"soc_kwh":result.soc[t],"curtail_kwh":
                result.curtail[t]})
44     check=export_result1(template_path("result1.xlsx",data_root),OUTPUT/"result1.xlsx",

```

```

        result)
45 metrics["xlsx_passed"]=check["passed"]
46 (OUTPUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),
        encoding="utf-8")
47 (OUTPUT/"xlsx_verification.json").write_text(json.dumps(check,ensure_ascii=False,indent
        =2),encoding="utf-8")
48 env={"python":sys.version,"platform":platform.platform(),"numpy":np.__version__,"scipy":
        scipy.__version__,
49         "openpyxl":openpyxl.__version__,"algorithm":"HiGHS lexicographic LP"}
50 (OUTPUT/"environment.json").write_text(json.dumps(env,ensure_ascii=False,indent=2),
        encoding="utf-8")
51 return {"metrics":metrics,"result":result}
52
53 if __name__=="__main__": print(json.dumps(run_q1()["metrics"],ensure_ascii=False,indent=2))

```

B.9 问题二求解 (q2.py)

Listing 9:

```

1 from __future__ import annotations
2
3 import csv
4 import json
5 import platform
6 import sys
7 from collections import Counter
8 from pathlib import Path
9
10 import numpy as np
11 import scipy
12
13 from .export import export_result2
14 from .forecast import QUANTILES, OnlinePVForecaster
15 from .io_data import DEFAULT_DATA_ROOT, DT_HOURS, read_attachment1, read_attachment2,
    template_path
16 from .load_forecast import OnlineLoadForecaster
17 from .lp_core import solve_dispatch
18 from .params import DEFAULT_STORAGE, StorageParams
19 from .price_forecast import OnlinePriceForecaster
20 from .settle import execute_plan
21
22 ROOT = Path(__file__).resolve().parents[1]
23 OUTPUT = ROOT / "outputs" / "q2"
24
25
26 def _verify_day(record: dict, storage: StorageParams) -> dict:

```



```

27     execution = record["execution"]
28     load = record["load_kw"]
29     pv = record["actual_pv_kw"]
30     balance = (
31         execution.grid
32         + pv * DT_HOURS
33         + execution.discharge
34         + execution.emergency
35         - load * DT_HOURS
36         - execution.charge
37         - execution.curtail
38     )
39     previous = np.r_[record["soc_initial"], execution.soc[:-1]]
40     soc_residual = execution.soc - (previous + storage.eta_ch * execution.charge - execution
41         .discharge / storage.eta_dis)
42     return {
43         "max_balance_residual": float(np.max(np.abs(balance))),
44         "max_soc_residual": float(np.max(np.abs(soc_residual))),
45         "min_soc": float(execution.soc.min()),
46         "max_soc": float(execution.soc.max()),
47         "max_simultaneous": float(np.max(np.minimum(execution.charge, execution.discharge)))
48     },
49 }
50 def _emergency_intervals(emergency: np.ndarray, threshold: float = 1e-7) -> int:
51     """统计紧急购电的连续时段块数（相邻时段合并为一个时间段）。"""
52     active = emergency > threshold
53     if not active.any():
54         return 0
55     return int(np.sum(active[1:] & ~active[:-1]) + (1 if active[0] else 0))
56
57
58 def run_q2(
59     *,
60     data_root: Path = DEFAULT_DATA_ROOT,
61     max_days: int = 365,
62     price_matrix: np.ndarray | None = None,
63     output_dir: Path | None = None,
64     template_name: str = "result2.xlsx",
65     write_outputs: bool = True,
66     load_mode: str = "causal",
67     pv_risk_mode: str = "asymmetric",
68     realtime_feedback: bool = True,
69     terminal_policy: str = "daily_cycle",
70     price_mode: str = "perfect",
71     storage: StorageParams = DEFAULT_STORAGE,

```

```

72     use_typical_prior: bool = True,
73     tag: str = "",
74 ) -> dict:
75     """问题二（及问题四对应问题二）日滚动调度。
76
77     信息集说明：
78         load_mode = "perfect" 计划使用当日真实负载（旧基准，含未来信息）
79         = "causal" 计划使用 OnlineLoadForecaster 的因果预测
80         price_mode = "perfect" 计划使用当日附件 4 真实电价
81         = "causal" 计划使用 OnlinePriceForecaster 的因果预测
82         pv_risk_mode 见 forecast.OnlinePVForecaster
83         realtime_feedback 见 settle.execute_plan
84         terminal_policy = "daily_cycle" 计划层 S144 = 当日实际 S0; "free" 不约束
85     """
86     if load_mode not in ("perfect", "causal"):
87         raise ValueError(f"unknown load_mode: {load_mode}")
88     if price_mode not in ("perfect", "causal"):
89         raise ValueError(f"unknown price_mode: {price_mode}")
90     if terminal_policy not in ("daily_cycle", "free"):
91         raise ValueError(f"unknown terminal_policy: {terminal_policy}")
92
93     day1 = read_attachment1(data_root)
94     annual = read_attachment2(data_root)
95     if max_days < 1 or max_days > 365:
96         raise ValueError("max_days must be 1..365")
97     prices = np.tile(day1.price, (365, 1)) if price_matrix is None else np.asarray(
98         price_matrix, dtype=float)
99     if prices.shape != (365, 144):
100         raise ValueError("price matrix must have shape (365,144)")
101     out_dir = OUTPUT if output_dir is None else Path(output_dir)
102
103     # 附件 1 的典型日曲线作为 2025-01-01 之前的先验（在论文假设中显式声明）；
104     # use_typical_prior=False 用于复现不含先验的 legacy 基准结果。
105     prior_pv = day1.pv_kw if use_typical_prior else None
106     prior_load = day1.load_kw if use_typical_prior else None
107     prior_price = day1.price if use_typical_prior else None
108     selector = OnlinePVForecaster(day1.price, risk_mode=pv_risk_mode, prior_pv=prior_pv)
109     load_forecaster = OnlineLoadForecaster(prior_kw=prior_load)
110     price_forecaster = OnlinePriceForecaster(prior_price=prior_price)
111
112     state = 6000.0
113     records: list[dict] = []
114     forecast_rows: list[dict] = []
115     load_rows: list[dict] = []
116     price_rows: list[dict] = []
117     verify_rows: list[dict] = []

```

```

118     for d in range(max_days):
119         initial = state
120
121         # ---- 决策时刻 0:00 可用信息 -----
122         load_decision = load_forecaster.predict(d, 0, 144)
123         pv_decision = selector.predict()
124         if pv_decision.history_days_used != d:
125             raise RuntimeError("PV forecast causality counter mismatch")
126         price_decision = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
            causal" else None
127
128         load_plan = annual.load_kw[d] if load_mode == "perfect" else load_decision.
            forecast_kw
129         price_plan = prices[d] if price_mode == "perfect" else price_decision.forecast_price
130         soc_terminal = initial if terminal_policy == "daily_cycle" else None
131
132         # ---- 日前计划优化（只使用上面确定的信息） -----
133         plan = solve_dispatch(
134             load_plan,
135             pv_decision.forecast_kw,
136             price_plan,
137             soc_initial=initial,
138             soc_terminal=soc_terminal,
139             storage=storage,
140         )
141
142         # ---- 真实执行（真实负载 / 真实光伏，实时储能反馈可开关） -----
143         execution = execute_plan(
144             plan,
145             annual.load_kw[d],
146             annual.pv_kw[d],
147             soc_initial=initial,
148             realtime_feedback=realtime_feedback,
149             storage=storage,
150         )
151
152         # ---- 用真实电价结算 -----
153         plan_cost = float(np.dot(prices[d], plan.grid))
154         emergency_cost = float(np.dot(5.0 * prices[d], execution.emergency))
155         state = float(execution.soc[-1])
156
157         record = {
158             "day_index": d,
159             "date": annual.dates[d],
160             "soc_initial": initial,
161             "plan": plan,
162             "execution": execution,

```

```

163         "load_kw": annual.load_kw[d],
164         "actual_pv_kw": annual.pv_kw[d],
165         "load_plan_kw": np.asarray(load_plan, dtype=float),
166         "pv_plan_kw": pv_decision.forecast_kw,
167         "price_plan": np.asarray(price_plan, dtype=float),
168         "plan_cost": plan_cost,
169         "emergency_cost": emergency_cost,
170         "total_cost": plan_cost + emergency_cost,
171         "load_decision": load_decision,
172         "pv_decision": pv_decision,
173         "adjusted_grid": execution.grid,
174     }
175     records.append(record)
176     verify_rows.append(_verify_day(record, storage))
177
178     # ---- 日志留痕 -----
179     forecast_row = {
180         "day_index": d,
181         "date": annual.dates[d].isoformat(),
182         "history_days_used": pv_decision.history_days_used,
183         "candidate": pv_decision.name,
184         "method": pv_decision.method,
185         "quantile": pv_decision.quantile,
186         "margin_kw": pv_decision.margin_kw,
187         "historical_score": pv_decision.historical_score,
188         "pv_forecast_mae_kw": float(np.mean(np.abs(pv_decision.forecast_kw - annual.
189             pv_kw[d]))),
189         "pv_forecast_rmse_kw": float(np.sqrt(np.mean((pv_decision.forecast_kw -
190             annual.pv_kw[d]) ** 2))),
190         "pv_daily_energy_error_kwh": float(np.sum(pv_decision.forecast_kw - annual.
191             pv_kw[d]) * DT_HOURS),
192     }
193     forecast_rows.append(forecast_row)
194     load_rows.append(load_forecaster.log_row(load_decision, annual.load_kw[d], annual.
195         dates[d].isoformat()))
196     if price_decision is not None:
197         price_rows.append(
198             price_forecaster.log_row(price_decision, prices[d], annual.dates[d].
199                 isoformat())
200         )
201
202     # ---- 观测回填（严格在决策之后） -----
203     candidate_scores = selector.observe(
204         pv_decision, annual.pv_kw[d], price=np.asarray(price_plan, dtype=float)
205     )
206     if pv_risk_mode == "asymmetric":
207         # 固定当日已选基方法，仅比较残差 Q0.80--Q0.95，形成可审计的样本外

```

```

205         # 风险代理。Q0.80(base-actual) 对应光伏输出的 0.20 分位点。
206         for quantile in QUANTILES:
207             name = OnlinePVForecaster._name(pv_decision.method, quantile)
208             forecast_row[f"risk_q{int(100 * quantile):02d}_yuan"] = candidate_scores[
                name]
209         load_forecaster.observe_block(load_decision, annual.load_kw[d])
210         load_forecaster.commit_day(annual.load_kw[d], d)
211         if price_decision is not None:
212             price_forecaster.observe_block(price_decision, prices[d])
213             price_forecaster.commit_day(prices[d], d)
214
215         delivery = records[31:] if max_days == 365 else []
216         min_soc = min(r["min_soc"] for r in verify_rows)
217         max_soc = max(r["max_soc"] for r in verify_rows)
218         max_balance = max(r["max_balance_residual"] for r in verify_rows)
219         max_soc_res = max(r["max_soc_residual"] for r in verify_rows)
220         max_sim = max(r["max_simultaneous"] for r in verify_rows)
221         continuity = (
222             max(abs(records[i]["soc_initial"] - records[i - 1]["execution"].soc[-1]) for i in
                range(1, len(records)))
223             if len(records) > 1
224             else 0.0
225         )
226
227         metrics = {
228             "case_tag": tag,
229             "days_simulated": max_days,
230             "warmup_days": 31 if max_days == 365 else None,
231             "delivery_days": len(delivery),
232             "load_mode": load_mode,
233             "pv_risk_mode": pv_risk_mode,
234             "price_mode": price_mode,
235             "realtime_feedback": bool(realtime_feedback),
236             "terminal_policy": terminal_policy,
237             "storage": storage.as_dict(),
238             "price_source": "attachment1" if price_matrix is None else "attachment4",
239             "load_information": (
240                 "Attachment 2 same-day true load curve (perfect-information benchmark)"
241                 if load_mode == "perfect"
242                 else "Online causal forecast from days strictly before d"
243             ),
244             "pv_information": (
245                 "Causal forecast from days strictly before d, "
246                 + ("historical residual quantile safety margin" if pv_risk_mode == "asymmetric"
                    else "no safety margin (point)")
247             ),
248             "price_information": (

```

```

249         "Attachment 4 same-day true price curve (perfect-information benchmark)"
250         if price_mode == "perfect"
251         else "Online causal forecast from days strictly before d"
252     ),
253     "realtime_storage_feedback": bool(realtime_feedback),
254     "terminal_policy_definition": (
255         "planned S144 equals actual S0 of the same day" if terminal_policy == "
256         daily_cycle" else "no terminal SOC constraint"
257     ),
258     "max_balance_residual": max_balance,
259     "max_soc_residual": max_soc_res,
260     "max_day_boundary_soc_gap": continuity,
261     "min_soc_kwh": min_soc,
262     "max_soc_kwh": max_soc,
263     "max_simultaneous_charge_discharge": max_sim,
264     "forecast_selection_counts": dict(Counter(row["candidate"] for row in forecast_rows)
265     ),
266     "load_selection_counts": dict(Counter(row["candidate"] for row in load_rows)),
267     "price_selection_counts": dict(Counter(row["method"] for row in price_rows) if
268     price_rows else None,
269 }
270 if delivery:
271     metrics.update(
272         {
273             "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
274             "plan_purchase_cost_yuan": float(sum(r["plan_cost"] for r in delivery)),
275             "adjustment_cost_yuan": 0.0,
276             "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
277             "plan_purchase_energy_kwh": float(sum(r["plan"].grid.sum() for r in delivery
278             )),
279             "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
280             delivery)),
281             "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
282             delivery)),
283             "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
284             delivery)),
285             "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
286             in delivery)),
287             "emergency_interval_count": int(
288                 sum(_emergency_intervals(r["execution"].emergency) for r in delivery)
289             ),
290             "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
291             delivery)),
292             "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
293             r in delivery)),
294             "storage_throughput_kwh": float(
295                 sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r

```

```

        in delivery)
286     ),
287     "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
288     "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows
        [31:]])),
289     "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows
        [31:]])),
290     "pv_forecast_mae_kw": float(np.mean([row["pv_forecast_mae_kw"] for row in
        forecast_rows[31:]])),
291     "pv_forecast_rmse_kw": float(np.mean([row["pv_forecast_rmse_kw"] for row in
        forecast_rows[31:]])),
292     "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows
        [31:]])) if price_rows else None,
293     "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in
        price_rows[31:]])) if price_rows else None,
294     "runtime_seconds": float(sum(r["plan"].solve_seconds for r in delivery)),
295 }
296 )
297 if pv_risk_mode == "asymmetric":
298     metrics["pv_residual_quantile_evaluation"] = {
299         f"Q{quantile:.2f}": {
300             "output_quantile": 1.0 - quantile,
301             "sample_out_risk_proxy_yuan": float(
302                 sum(row[f"risk_q{int(100 * quantile):02d}_yuan"] for row in
                    forecast_rows[31:])
303             ),
304             "selected_days": int(
305                 sum(abs(float(row["quantile"])) - quantile) < 1e-12 for row in
                    forecast_rows[31:])
306             ),
307         }
308         for quantile in QUANTILES
309     }
310 # 阈值说明: 能量残差用 1e-7 kWh, 同时充放电量用 1e-5 kWh。后者是数值噪声
311 # (约 1e-6 kWh) 与真实的线性松弛退化 (80~824 kWh) 之间的分界, 两者相差 7 个
312 # 数量级, 因此该阈值既能滤掉浮点噪声、又不会放过真正的"边充边放"伪解。
313 # (统一为与 q3.py 相同的阈值, 避免同一份代码里出现两套判据。)
314 metrics["passed"] = bool(
315     max_balance < 1e-7
316     and max_soc_res < 1e-7
317     and continuity < 1e-9
318     and min_soc >= storage.soc_min - 1e-7
319     and max_soc <= storage.soc_max + 1e-7
320     and max_sim < 1e-5
321 )
322
323 if write_outputs:

```

```

324 out_dir.mkdir(parents=True, exist_ok=True)
325 (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
    =2), encoding="utf-8")
326 _write_csv(out_dir / "forecast_log.csv", forecast_rows)
327 _write_csv(out_dir / "load_forecast_log.csv", load_rows)
328 if price_rows:
329     _write_csv(out_dir / "price_forecast_log.csv", price_rows)
330 if delivery:
331     fields = [
332         "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "pv_plan_kw", "
            price",
333         "grid_plan_kwh", "grid_actual_kwh", "charge_actual_kwh", "
            discharge_actual_kwh",
334         "soc_actual_kwh", "curtail_actual_kwh", "emergency_kwh",
335     ]
336     with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f
        :
337         writer = csv.DictWriter(f, fieldnames=fields)
338         writer.writeheader()
339         for r in delivery:
340             for t in range(144):
341                 writer.writerow(
342                     {
343                         "date": r["date"].isoformat(),
344                         "slot": t + 1,
345                         "load_kw": r["load_kw"][t],
346                         "actual_pv_kw": r["actual_pv_kw"][t],
347                         "load_plan_kw": r["load_plan_kw"][t],
348                         "pv_plan_kw": r["pv_plan_kw"][t],
349                         "price": prices[r["day_index"], t],
350                         "grid_plan_kwh": r["plan"].grid[t],
351                         "grid_actual_kwh": r["adjusted_grid"][t],
352                         "charge_actual_kwh": r["execution"].charge[t],
353                         "discharge_actual_kwh": r["execution"].discharge[t],
354                         "soc_actual_kwh": r["execution"].soc[t],
355                         "curtail_actual_kwh": r["execution"].curtail[t],
356                         "emergency_kwh": r["execution"].emergency[t],
357                     }
358                 )
359     daily_fields = [
360         "date", "soc_initial", "soc_terminal", "plan_cost", "emergency_cost", "
            total_cost",
361         "plan_energy", "emergency_energy", "curtailment_energy",
362     ]
363     _write_csv(
364         out_dir / "daily_metrics.csv",
365         [

```



```

366         {
367             "date": r["date"].isoformat(),
368             "soc_initial": r["soc_initial"],
369             "soc_terminal": r["execution"].soc[-1],
370             "plan_cost": r["plan_cost"],
371             "emergency_cost": r["emergency_cost"],
372             "total_cost": r["total_cost"],
373             "plan_energy": r["plan"].grid.sum(),
374             "emergency_energy": r["execution"].emergency.sum(),
375             "curtailment_energy": r["execution"].curtail.sum(),
376         }
377         for r in delivery
378     ],
379     daily_fields,
380 )
381 check = export_result2(template_path(template_name, data_root), out_dir /
382                        template_name, delivery)
383 (out_dir / "xlsx_verification.json").write_text(
384     json.dumps(check, ensure_ascii=False, indent=2), encoding="utf-8"
385 )
386 metrics["xlsx_passed"] = check["passed"]
387 (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
388         indent=2), encoding="utf-8")
389
390 environment = {
391     "python": sys.version,
392     "platform": platform.platform(),
393     "numpy": np.__version__,
394     "scipy": scipy.__version__,
395     "algorithm": "scipy.optimize.linprog(method=highs), lexicographic 3-pass LP",
396     "case_tag": tag,
397 }
398
399 if write_outputs:
400     (out_dir / "environment.json").write_text(json.dumps(environment, ensure_ascii=False
401         , indent=2), encoding="utf-8")
402
403 return {"metrics": metrics, "records": records, "forecast_rows": forecast_rows, "
404         load_rows": load_rows}
405
406
407 def _write_csv(path: Path, rows: list[dict], fields: list[str] | None = None) -> None:
408     if not rows:
409         return
410     fieldnames = fields if fields is not None else list(rows[0])
411     with path.open("w", newline="", encoding="utf-8-sig") as f:
412         writer = csv.DictWriter(f, fieldnames=fieldnames)
413         writer.writeheader()
414         writer.writerows(rows)

```

```

409
410
411 if __name__ == "__main__":
412     print(json.dumps(run_q2()["metrics"], ensure_ascii=False, indent=2))

```

B.10 问题三求解 (q3.py)

Listing 10:

```

1  from __future__ import annotations
2
3  import json
4  import platform
5  import sys
6  from pathlib import Path
7
8  import numpy as np
9  import scipy
10
11 from .export import export_result3
12 from .io_data import (
13     DEFAULT_DATA_ROOT,
14     DT_HOURS,
15     hourly_to_ten_min,
16     read_attachment1,
17     read_attachment2,
18     read_attachment3,
19     template_path,
20 )
21 from .load_forecast import OnlineLoadForecaster
22 from .lp_core import DispatchResult, settlement_cost, solve_adjustment_dispatch,
23     solve_dispatch
24 from .official_forecast import OnlineForecastBiasCalibrator
25 from .params import DEFAULT_STORAGE, StorageParams
26 from .price_forecast import OnlinePriceForecaster
27 from .q2 import _emergency_intervals, _write_csv
28 from .settle import ExecutionResult, execute_plan
29
30 ROOT = Path(__file__).resolve().parents[1]
31 OUTPUT = ROOT / "outputs" / "q3"
32 BLOCK = 36
33 ALL_NODES = (0, 36, 72, 108)
34
35 def _horizon(data: np.ndarray, day: int, start_slot: int, fallback: np.ndarray) -> np.
    ndarray:

```

```

36     flat = data.reshape(-1)
37     start = day * 144 + start_slot
38     values = flat[start : min(start + 144, len(flat))]
39     if len(values) < 144:
40         pieces = [values]
41         remaining = 144 - len(values)
42         while remaining > 0:
43             take = min(remaining, len(fallback))
44             pieces.append(fallback[:take])
45             remaining -= take
46         values = np.concatenate(pieces)
47     return np.asarray(values, dtype=float)
48
49
50 def _slice_dispatch(result: DispatchResult, start: int, stop: int) -> DispatchResult:
51     return DispatchResult(
52         result.grid[start:stop].copy(),
53         result.charge[start:stop].copy(),
54         result.discharge[start:stop].copy(),
55         result.soc[start:stop].copy(),
56         result.curtail[start:stop].copy(),
57         result.objective,
58         result.primary_objective,
59         float(result.curtail[start:stop].sum()),
60         float(result.charge[start:stop].sum() + result.discharge[start:stop].sum()),
61         result.status,
62         result.message,
63         result.nit,
64         result.solve_seconds,
65         result.equality_marginals.copy(),
66     )
67
68
69 def _replace_dispatch_tail(current: DispatchResult, horizon: DispatchResult, start: int) ->
    DispatchResult:
70     """把 horizon 的首段写入当前日计划的 start:144。
71
72     该函数是节点消融的关键：若后续节点未启用，最近一次有效计划仍保留在
73     current 中并继续执行，而不是回退到 0:00 基准计划。
74     """
75     count = 144 - start
76
77     def merged(name: str) -> np.ndarray:
78         values = np.asarray(getattr(current, name), dtype=float).copy()
79         values[start:] = np.asarray(getattr(horizon, name), dtype=float)[:count]
80     return values
81

```

```

82     charge = merged("charge")
83     discharge = merged("discharge")
84     curtail = merged("curtail")
85     return DispatchResult(
86         merged("grid"),
87         charge,
88         discharge,
89         merged("soc"),
90         curtail,
91         horizon.objective,
92         horizon.primary_objective,
93         float(curtail.sum()),
94         float(charge.sum() + discharge.sum()),
95         horizon.status,
96         horizon.message,
97         horizon.nit,
98         horizon.solve_seconds,
99         horizon.equality_marginals.copy(),
100     )
101
102
103 def _reversal_count(revisions: np.ndarray) -> int:
104     count = 0
105     for slot in range(144):
106         values = revisions[:, slot]
107         values = values[np.isfinite(values)]
108         signs = np.sign(np.diff(values))
109         signs = signs[np.abs(signs) > 0]
110         if len(signs) > 1 and np.any(signs[1:] * signs[:-1] < 0):
111             count += 1
112     return count
113
114
115 def _node_label(node: int) -> str:
116     return f"{node // 6:02d}:00"
117
118
119 def run_q3(
120     *,
121     data_root: Path = DEFAULT_DATA_ROOT,
122     max_days: int = 365,
123     downscale: str = "linear",
124     calibrate: bool = True,
125     update_nodes: tuple[int, ...] = ALL_NODES,
126     pv_update_nodes: tuple[int, ...] | None = None,
127     settlement_mode: str = "additive",
128     load_mode: str = "causal",

```

```

129     price_mode: str = "perfect",
130     realtime_feedback: bool = True,
131     terminal_policy: str = "daily_cycle",
132     price_matrix: np.ndarray | None = None,
133     storage: StorageParams = DEFAULT_STORAGE,
134     output_dir: Path | None = None,
135     template_name: str = "result3.xlsx",
136     write_outputs: bool = True,
137     use_typical_prior: bool = True,
138     tag: str = "",
139 ) -> dict:
140     """问题三（及问题四对应问题三）多节点滚动调度。
141
142     update_nodes 控制"允许在哪些时刻重新优化购电策略"; pv_update_nodes 控制这些
143     决策时刻是否使用新发布的官方光伏预报。常规 S0--S3 令二者相同；负载刷新
144     对照令 update_nodes=ALL_NODES、pv_update_nodes=(0,)，从而冻结 0:00 光伏预报：
145         S0 = (0,)           仅 0:00 预报
146         S1 = (0, 36)        +6:00
147         S2 = (0, 36, 72)    +12:00
148         S3 = (0, 36, 72, 108) +18:00
149     settlement_mode 决定 LP 目标函数本身：
150         replacement C = p[min(Gp,Ga) + 0.5(Gp-Ga)^+ + 1.5(Ga-Gp)^+]
151         additive    C = p * Gp + 0.5p(Gp-Ga)^+ + 1.5p(Ga-Gp)^+
152     两种口径**分别完整重新优化**，不是同一调度方案的事后换算式计费。
153     """
154     if settlement_mode not in ("replacement", "additive"):
155         raise ValueError(f"unknown settlement_mode: {settlement_mode}")
156     if load_mode not in ("perfect", "causal") or price_mode not in ("perfect", "causal"):
157         raise ValueError("unknown information mode")
158     nodes = tuple(sorted(int(n) for n in update_nodes))
159     if not nodes or nodes[0] != 0 or any(n not in ALL_NODES for n in nodes):
160         raise ValueError(f"update_nodes must be a prefix of {ALL_NODES}, got {update_nodes}"
161                             )
162     pv_nodes = nodes if pv_update_nodes is None else tuple(sorted(int(n) for n in
163                             pv_update_nodes))
164
165     if not pv_nodes or pv_nodes[0] != 0 or any(n not in nodes for n in pv_nodes):
166         raise ValueError(f"pv_update_nodes must contain 0 and be a subset of update_nodes,
167                             got {pv_nodes}")
168
169     typical = read_attachment1(data_root)
170     annual = read_attachment2(data_root)
171     forecasts = read_attachment3(data_root)
172     if annual.dates != forecasts.dates:
173         raise ValueError("attachment2/3 dates do not align")
174     if not 1 <= max_days <= 365:
175         raise ValueError("max_days must be 1..365")
176     prices = np.tile(typical.price, (365, 1)) if price_matrix is None else np.asarray(

```

```

        price_matrix, dtype=float)
173 if prices.shape != (365, 144):
174     raise ValueError("price matrix must have shape (365,144)")
175 out_dir = OUTPUT if output_dir is None else Path(output_dir)
176
177 state = 6000.0
178 records: list[dict] = []
179 verify: list[tuple] = []
180 load_rows: list[dict] = []
181 price_rows: list[dict] = []
182 total_reversals = 0
183 calibrator = OnlineForecastBiasCalibrator()
184 load_forecaster = OnlineLoadForecaster(prior_kw=typical.load_kw if use_typical_prior
    else None)
185 price_forecaster = OnlinePriceForecaster(prior_price=typical.price if use_typical_prior
    else None)
186
187 for d in range(max_days):
188     initial = state
189     node_state = initial
190     soc_terminal = initial if terminal_policy == "daily_cycle" else None
191
192     # ---- 0:00 基准计划 -----
193     load_decision0 = load_forecaster.predict(d, 0, 144)
194     price_decision0 = price_forecaster.predict_horizon(d, 0, 144) if price_mode == "
        causal" else None
195     load0 = annual.load_kw[d] if load_mode == "perfect" else load_decision0.forecast_kw
196     price0 = prices[d] if price_mode == "perfect" else price_decision0.forecast_price
197     raw_base = hourly_to_ten_min(forecasts.hourly_kw[d, 0], downscale)
198     calibration0 = calibrator.calibrate(0, raw_base)
199     base_pv = calibration0.forecast_kw if calibrate else raw_base
200     baseline = solve_dispatch(
201         load0, base_pv, price0, soc_initial=initial, soc_terminal=soc_terminal, storage=
            storage
202     )
203     active_plan = baseline
204     active_pv_day = np.asarray(base_pv, dtype=float).copy()
205     active_source_node = 0
206
207     adjusted = np.zeros(144)
208     charge = np.zeros(144)
209     discharge = np.zeros(144)
210     soc = np.zeros(144)
211     curtail = np.zeros(144)
212     emergency = np.zeros(144)
213     inc_acc = np.zeros(144)
214     dec_acc = np.zeros(144)

```

```

215     revisions = np.full((4, 144), np.nan)
216     revisions[0] = baseline.grid
217     plan_source_nodes = np.zeros(144, dtype=int)
218     block_forecast_mae: list[float] = []
219     calibration_margins: list[float] = []
220     calibration_quantiles: list = []
221     load_plan_day = np.asarray(load0, dtype=float).copy()
222
223     load_rows.append(load_forecaster.log_row(load_decision0, annual.load_kw[d, :36],
224         annual.dates[d].isoformat()))
225     if price_decision0 is not None:
226         price_rows.append(price_forecaster.log_row(price_decision0, prices[d, :36],
227             annual.dates[d].isoformat()))
228     node_load_decisions: dict[int, object] = {0: load_decision0}
229
230     for node_index, node in enumerate(ALL_NODES):
231         block_stop = node + BLOCK
232         if node_index == 0:
233             calibration = calibration0
234             block_plan = _slice_dispatch(baseline, 0, BLOCK)
235             block_forecast = base_pv[0:BLOCK]
236             block_inc = np.zeros(BLOCK)
237             block_dec = np.zeros(BLOCK)
238         elif node in nodes:
239             if node in pv_nodes:
240                 raw_horizon = hourly_to_ten_min(forecasts.hourly_kw[d, node_index],
241                     downscale)
242                 calibration = calibrator.calibrate(node_index, raw_horizon)
243                 forecast_horizon = calibration.forecast_kw if calibration else raw_horizon
244             else:
245                 # 负载刷新对照：允许重优化，但不引入该节点的新光伏预报。
246                 # 当日剩余部分沿用最近一次可用预报，跨午夜部分用附件 1 典型日先验补齐。
247                 calibration = None
248                 forecast_horizon = np.r_[active_pv_day[node:], typical.pv_kw[:node]]
249             if load_mode == "perfect":
250                 load_horizon = _horizon(annual.load_kw, d, node, typical.load_kw)
251             else:
252                 load_decision = load_forecaster.predict(d, node, 144)
253                 load_horizon = load_decision.forecast_kw
254                 load_plan_day[node:] = load_horizon[: 144 - node]
255                 node_load_decisions[node] = load_decision
256                 load_rows.append(
257                     load_forecaster.log_row(load_decision, annual.load_kw[d, node:
258                         block_stop], annual.dates[d].isoformat())
259                 )
260             if price_mode == "perfect":
261                 price_horizon = _horizon(prices, d, node, typical.price)

```

```

258         else:
259             price_decision = price_forecaster.predict_horizon(d, node, 144)
260             price_horizon = price_decision.forecast_price
261             price_rows.append(
262                 price_forecaster.log_row(price_decision, prices[d, node:block_stop],
263                                         annual.dates[d].isoformat())
264             )
265             baseline_horizon = np.r_[baseline.grid[node:], np.zeros(node)]
266             mask = np.r_[np.ones(144 - node, dtype=bool), np.zeros(node, dtype=bool)]
267             rolling = solve_adjustment_dispatch(
268                 load_horizon,
269                 forecast_horizon,
270                 price_horizon,
271                 baseline_horizon,
272                 mask,
273                 soc_initial=node_state,
274                 soc_terminal=None if terminal_policy == "free" else node_state,
275                 settlement_mode=settlement_mode,
276                 storage=storage,
277             )
278             horizon_plan = rolling.dispatch
279             active_plan = _replace_dispatch_tail(active_plan, horizon_plan, node)
280             active_pv_day[node:] = forecast_horizon[: 144 - node]
281             active_source_node = node
282             block_plan = _slice_dispatch(active_plan, node, block_stop)
283             revisions[node_index, node:] = active_plan.grid[node:]
284             block_forecast = active_pv_day[node:block_stop]
285             base_block = baseline.grid[node:block_stop]
286             block_inc = np.maximum(block_plan.grid - base_block, 0.0)
287             block_dec = np.maximum(base_block - block_plan.grid, 0.0)
288         else:
289             # 没有新决策时，最近一次有效计划持续生效。
290             calibration = None
291             block_plan = _slice_dispatch(active_plan, node, block_stop)
292             block_forecast = active_pv_day[node:block_stop]
293             base_block = baseline.grid[node:block_stop]
294             block_inc = np.maximum(block_plan.grid - base_block, 0.0)
295             block_dec = np.maximum(base_block - block_plan.grid, 0.0)
296
297             plan_source_nodes[node:block_stop] = active_source_node
298
299             execution = execute_plan(
300                 block_plan,
301                 annual.load_kw[d, node:block_stop],
302                 annual.pv_kw[d, node:block_stop],
303                 soc_initial=node_state,
304                 realtime_feedback=realtime_feedback,

```



```

304         storage=storage,
305     )
306     adjusted[node:block_stop] = execution.grid
307     charge[node:block_stop] = execution.charge
308     discharge[node:block_stop] = execution.discharge
309     soc[node:block_stop] = execution.soc
310     curtail[node:block_stop] = execution.curtail
311     emergency[node:block_stop] = execution.emergency
312     inc_acc[node:block_stop] = block_inc
313     dec_acc[node:block_stop] = block_dec
314
315     if calibration is not None:
316         calibrator.observe(calibration, annual.pv_kw[d, node:block_stop], prices[d,
317                                node:block_stop])
318         calibration_margins.append(calibration.margin_kw if calibrate else 0.0)
319         calibration_quantiles.append(calibration.quantile if calibrate else None)
320         block_forecast_mae.append(float(np.mean(np.abs(block_forecast - annual.pv_kw[d,
321                                node:block_stop]))))
322     node_state = float(execution.soc[-1])
323
324     # 供后续节点使用的已观测信息（严格在决策与执行之后）
325     if load_mode == "causal" and node in node_load_decisions:
326         load_forecaster.observe_block(node_load_decisions[node], annual.load_kw[d,
327                                node:block_stop])
328         load_forecaster.observe_partial(block_stop, annual.load_kw[d, :block_stop])
329
330     state = node_state
331     actual = ExecutionResult(adjusted.copy(), charge, discharge, soc, curtail, emergency
332                             )
333
334     gp = baseline.grid
335     ga = adjusted
336     increase = np.maximum(ga - gp, 0.0)
337     decrease = np.maximum(gp - ga, 0.0)
338     settlement = settlement_cost(gp, ga, prices[d], settlement_mode)
339     if settlement_mode == "replacement":
340         lp_side = float(np.dot(prices[d], ga + 0.5 * inc_acc + 0.5 * dec_acc))
341     else:
342         lp_side = float(np.dot(prices[d], gp) + np.dot(prices[d], 0.5 * dec_acc + 1.5 *
343                                inc_acc))
344     emergency_cost = float(np.dot(5 * prices[d], emergency))
345     reversals = _reversal_count(revisions)
346     total_reversals += reversals
347
348     record = {
349         "day_index": d,
350         "date": annual.dates[d],

```

```

346     "soc_initial": initial,
347     "baseline_plan": baseline,
348     "adjusted_grid": adjusted,
349     "execution": actual,
350     "load_kw": annual.load_kw[d],
351     "actual_pv_kw": annual.pv_kw[d],
352     "load_plan_kw": load_plan_day,
353     "revisions": revisions,
354     "plan_source_nodes": plan_source_nodes.copy(),
355     "baseline_cost": float(np.dot(prices[d], gp)),
356     "settlement_cost": settlement,
357     "emergency_cost": emergency_cost,
358     "total_cost": settlement + emergency_cost,
359     "adjustment_cost_yuan": settlement - float(np.dot(prices[d], gp)) if
        settlement_mode == "additive" else settlement - float(np.dot(prices[d], gp))
    ,
360     "linearization_gap": abs(settlement - lp_side),
361     "up_energy": float(increase.sum()),
362     "down_energy": float(decrease.sum()),
363     "reversal_slots": reversals,
364     "forecast_mae": float(np.mean(block_forecast_mae)),
365     "calibration_margins": calibration_margins,
366     "calibration_quantiles": calibration_quantiles,
367 }
368 records.append(record)
369
370 balance = ga + annual.pv_kw[d] * DT_HOURS + discharge + emergency - annual.load_kw[d
    ] * DT_HOURS - charge - curtail
371 previous = np.r_[initial, soc[:-1]]
372 soc_res = soc - (previous + storage.eta_ch * charge - discharge / storage.eta_dis)
373 verify.append(
374     (
375         float(np.max(np.abs(balance))),
376         float(np.max(np.abs(soc_res))),
377         float(soc.min()),
378         float(soc.max()),
379         float(np.max(np.minimum(charge, discharge))),
380     )
381 )
382
383 # ---- 日终观测回填 -----
384 if load_mode == "causal":
385     load_forecaster.commit_day(annual.load_kw[d], d)
386 if price_mode == "causal":
387     price_forecaster.commit_day(prices[d], d)
388 else:
389     price_forecaster.commit_day(prices[d], d)

```

```

390
391 delivery = records[31:] if max_days == 365 else []
392 metrics = {
393     "case_tag": tag,
394     "days_simulated": max_days,
395     "warmup_days": 31 if max_days == 365 else None,
396     "delivery_days": len(delivery),
397     "downscale_method": downscale,
398     "calibration_enabled": calibrate,
399     "update_nodes": list(nodes),
400     "update_nodes_label": "S" + str(len(nodes) - 1),
401     "pv_update_nodes": list(pv_nodes),
402     "settlement_mode": settlement_mode,
403     "load_mode": load_mode,
404     "price_mode": price_mode,
405     "realtime_feedback": bool(realtime_feedback),
406     "terminal_policy": terminal_policy,
407     "storage": storage.as_dict(),
408     "price_source": "attachment1" if price_matrix is None else "attachment4",
409     "load_information": (
410         "Attachment 2 same-day true load curve (perfect-information benchmark)"
411         if load_mode == "perfect"
412         else "Online causal forecast from days strictly before d"
413     ),
414     "pv_information": (
415         "Attachment 3 rolling forecasts with causal issue-specific calibration"
416         if calibrate
417         else "raw Attachment 3 forecasts"
418     )
419     + f"; newly issued forecasts used at nodes {list(pv_nodes)}; actual PV used only
        after block execution",
420     "price_information": (
421         "Attachment 4 same-day true price curve (perfect-information benchmark)"
422         if price_mode == "perfect"
423         else "Online causal price forecast from days strictly before d"
424     ),
425     "max_balance_residual": max(v[0] for v in verify),
426     "max_soc_residual": max(v[1] for v in verify),
427     "min_soc_kwh": min(v[2] for v in verify),
428     "max_soc_kwh": max(v[3] for v in verify),
429     "max_simultaneous_charge_discharge": max(v[4] for v in verify),
430     "replacement_settlement_formula": "min(Gp,Ga) + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
431     "additive_settlement_formula": "Gp + 0.5*(Gp-Ga)^+ + 1.5*(Ga-Gp)^+",
432     "cross_midnight": "24h optimize, execute current 6h block only; carry SOC only",
433 }
434 if delivery:
435     metrics.update(

```

```

436     {
437         "total_cost_yuan": float(sum(r["total_cost"] for r in delivery)),
438         "baseline_purchase_cost_yuan": float(sum(r["baseline_cost"] for r in
439             delivery)),
440         "adjustment_settlement_cost_yuan": float(sum(r["settlement_cost"] for r in
441             delivery)),
442         "adjustment_cost_yuan": float(sum(r["adjustment_cost_yuan"] for r in
443             delivery)),
444         "emergency_cost_yuan": float(sum(r["emergency_cost"] for r in delivery)),
445         "plan_purchase_energy_kwh": float(sum(r["baseline_plan"].grid.sum() for r in
446             delivery)),
447         "adjusted_purchase_energy_kwh": float(sum(r["adjusted_grid"].sum() for r in
448             delivery)),
449         "emergency_energy_kwh": float(sum(r["execution"].emergency.sum() for r in
450             delivery)),
451         "curtailment_energy_kwh": float(sum(r["execution"].curtail.sum() for r in
452             delivery)),
453         "up_adjustment_energy_kwh": float(sum(r["up_energy"] for r in delivery)),
454         "down_adjustment_energy_kwh": float(sum(r["down_energy"] for r in delivery))
455         ,
456         "days_with_emergency": int(sum(r["execution"].emergency.sum() > 1e-7 for r
457             in delivery)),
458         "emergency_interval_count": int(sum(_emergency_intervals(r["execution"].
459             emergency) for r in delivery)),
460         "storage_charge_energy_kwh": float(sum(r["execution"].charge.sum() for r in
461             delivery)),
462         "storage_discharge_energy_kwh": float(sum(r["execution"].discharge.sum() for
463             r in delivery)),
464         "storage_throughput_kwh": float(
465             sum(r["execution"].charge.sum() + r["execution"].discharge.sum() for r
466                 in delivery)
467         ),
468         "ending_soc_kwh": float(delivery[-1]["execution"].soc[-1]),
469         "direction_reversal_slots": int(sum(r["reversal_slots"] for r in delivery)),
470         "max_linearization_gap": float(max(r["linearization_gap"] for r in delivery)
471             ),
472         "executed_block_forecast_mae_kw": float(np.mean([r["forecast_mae"] for r in
473             delivery])),
474         "load_forecast_mae_kw": float(np.mean([row["mae_kw"] for row in load_rows
475             [31:]])) if load_rows else None,
476         "load_forecast_rmse_kw": float(np.mean([row["rmse_kw"] for row in load_rows
477             [31:]])) if load_rows else None,
478         "price_forecast_mae": float(np.mean([row["price_mae"] for row in price_rows
479             [31:]])) if price_rows else None,
480         "price_forecast_rmse": float(np.mean([row["price_rmse"] for row in
481             price_rows[31:]])) if price_rows else None,
482         "runtime_seconds": float(

```

```

464         sum(r["baseline_plan"].solve_seconds for r in delivery)
465     ),
466 }
467 )
468 metrics["passed"] = bool(
469     metrics["max_balance_residual"] < 1e-7
470     and metrics["max_soc_residual"] < 1e-7
471     and metrics["min_soc_kwh"] >= storage.soc_min - 1e-7
472     and metrics["max_soc_kwh"] <= storage.soc_max + 1e-7
473     and metrics["max_simultaneous_charge_discharge"] < 1e-5
474     and (not delivery or metrics["max_linearization_gap"] < 1e-6)
475 )
476
477 if write_outputs:
478     out_dir.mkdir(parents=True, exist_ok=True)
479     (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False, indent
480                                         =2), encoding="utf-8")
481     _write_csv(out_dir / "load_forecast_log.csv", load_rows)
482     if price_rows:
483         _write_csv(out_dir / "price_forecast_log.csv", price_rows)
484     if delivery:
485         fields = [
486             "date", "slot", "load_kw", "actual_pv_kw", "load_plan_kw", "price", "
487             baseline_grid_kwh",
488             "adjusted_grid_kwh", "charge_actual_kwh", "discharge_actual_kwh", "
489             soc_actual_kwh",
490             "curtail_actual_kwh", "emergency_kwh", "revision_0", "revision_6", "
491             revision_12", "revision_18",
492         ]
493         with (out_dir / "solution.csv").open("w", newline="", encoding="utf-8-sig") as f
494             :
495             import csv as _csv
496
497             writer = _csv.DictWriter(f, fieldnames=fields)
498             writer.writeheader()
499             for r in delivery:
500                 for t in range(144):
501                     rev = r["revisions"][t, t]
502                     writer.writerow(
503                         {
504                             "date": r["date"].isoformat(),
505                             "slot": t + 1,
506                             "load_kw": r["load_kw"][t],
507                             "actual_pv_kw": r["actual_pv_kw"][t],
508                             "load_plan_kw": r["load_plan_kw"][t],
509                             "price": prices[r["day_index"], t],
510                             "baseline_grid_kwh": r["baseline_plan"].grid[t],

```

```

506         "adjusted_grid_kwh": r["adjusted_grid"][t],
507         "charge_actual_kwh": r["execution"].charge[t],
508         "discharge_actual_kwh": r["execution"].discharge[t],
509         "soc_actual_kwh": r["execution"].soc[t],
510         "curtail_actual_kwh": r["execution"].curtail[t],
511         "emergency_kwh": r["execution"].emergency[t],
512         "revision_0": rev[0],
513         "revision_6": rev[1],
514         "revision_12": rev[2],
515         "revision_18": rev[3],
516     }
517 )
518 fields_d = [
519     "date", "soc_initial", "soc_terminal", "baseline_cost", "settlement_cost", "
520     emergency_cost",
521     "total_cost", "up_energy", "down_energy", "reversal_slots", "forecast_mae",
522 ]
523 _write_csv(
524     out_dir / "daily_metrics.csv",
525     [
526         {
527             "date": r["date"].isoformat(),
528             "soc_initial": r["soc_initial"],
529             "soc_terminal": r["execution"].soc[-1],
530             **{k: r[k] for k in fields_d[3:]},
531         }
532     ],
533     for r in delivery
534 )
535 check = export_result3(template_path(template_name, data_root), out_dir /
536     template_name, delivery)
537 (out_dir / "xlsx_verification.json").write_text(json.dumps(check, ensure_ascii=
538     False, indent=2), encoding="utf-8")
539 metrics["xlsx_passed"] = check["passed"]
540 (out_dir / "metrics.json").write_text(json.dumps(metrics, ensure_ascii=False,
541     indent=2), encoding="utf-8")
542 env = {
543     "python": sys.version,
544     "platform": platform.platform(),
545     "numpy": np.__version__,
546     "scipy": scipy.__version__,
547     "algorithm": (
548         "HiGHS MILP with binary charge/discharge exclusivity and explicit (inc, dec)
549         split"
550         if settlement_mode == "additive"
551         else "HiGHS LP with explicit (inc, dec) split"
552     )
553 }

```

```

548         ),
549         "case_tag": tag,
550     }
551     (out_dir / "environment.json").write_text(json.dumps(env, ensure_ascii=False, indent
552         =2), encoding="utf-8")
553     return {"metrics": metrics, "records": records}
554
555 if __name__ == "__main__":
556     print(json.dumps(run_q3()["metrics"], ensure_ascii=False, indent=2))

```

B.11 问题四求解 (q4.py)

Listing 11:

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  from .io_data import DEFAULT_DATA_ROOT, read_attachment4
7  from .q2 import run_q2
8  from .q3 import run_q3
9
10 ROOT = Path(__file__).resolve().parents[1]
11 OUTPUT = ROOT / "outputs"
12
13
14 def run_q4(
15     *,
16     data_root: Path = DEFAULT_DATA_ROOT,
17     max_days: int = 365,
18     price_mode_q2: str = "perfect",
19     price_mode_q3: str = "perfect",
20     load_mode: str = "causal",
21     settlement_mode: str = "additive",
22     update_nodes: tuple[int, ...] = (0, 36, 72, 108),
23     realtime_feedback: bool = True,
24     write_outputs: bool = True,
25     tag_prefix: str = "",
26 ) -> dict:
27     """问题四：把固定典型日电价替换为附件 4 的波动电价，分别重算问题二、问题三。
28
29     price_mode = "perfect" 优化时已知当日真实电价 (perfect_price benchmark) ;
30     = "causal" 只用决策时刻之前已发生的电价滚动预测未来电价。
31
32     """

```

```

32     annual_price = read_attachment4(data_root)
33     prices = annual_price.price
34
35     suffix = f"-{tag_prefix}" if tag_prefix else ""
36     out2 = OUTPUT / f"q4-2{suffix}"
37     out3 = OUTPUT / f"q4-3{suffix}"
38     q4_2 = run_q2(
39         data_root=data_root,
40         max_days=max_days,
41         price_matrix=prices,
42         output_dir=out2,
43         template_name="result4-2.xlsx",
44         write_outputs=write_outputs,
45         load_mode=load_mode,
46         price_mode=price_mode_q2,
47         realtime_feedback=realtime_feedback,
48         tag=f"q4-2 {tag_prefix}".strip(),
49     )
50     q4_3 = run_q3(
51         data_root=data_root,
52         max_days=max_days,
53         price_matrix=prices,
54         output_dir=out3,
55         template_name="result4-3.xlsx",
56         write_outputs=write_outputs,
57         load_mode=load_mode,
58         price_mode=price_mode_q3,
59         settlement_mode=settlement_mode,
60         update_nodes=update_nodes,
61         realtime_feedback=realtime_feedback,
62         tag=f"q4-3 {tag_prefix}".strip(),
63     )
64
65     c2 = q4_2["metrics"].get("total_cost_yuan")
66     c3 = q4_3["metrics"].get("total_cost_yuan")
67     comparison = {
68         "q4_2_total_cost_yuan": c2,
69         "q4_3_total_cost_yuan": c3,
70         "rolling_saving_yuan": None if (c2 is None or c3 is None) else c2 - c3,
71         "rolling_saving_percent": None if (c2 is None or c3 is None) else 100 * (c2 - c3) /
72             c2,
73         "q4_2_emergency_energy_kwh": q4_2["metrics"].get("emergency_energy_kwh"),
74         "q4_3_emergency_energy_kwh": q4_3["metrics"].get("emergency_energy_kwh"),
75         "q4_2_price_mode": price_mode_q2,
76         "q4_3_price_mode": price_mode_q3,
77         "q4_2_price_mae": q4_2["metrics"].get("price_forecast_mae"),
78         "q4_3_price_mae": q4_3["metrics"].get("price_forecast_mae"),

```



```

78     "q4_2_passed": bool(q4_2["metrics"]["passed"]),
79     "q4_3_passed": bool(q4_3["metrics"]["passed"]),
80 }
81 comparison["passed"] = comparison["q4_2_passed"] and comparison["q4_3_passed"]
82 if write_outputs:
83     out2.mkdir(parents=True, exist_ok=True)
84     (out2 / "q4_comparison.json").write_text(
85         json.dumps(comparison, ensure_ascii=False, indent=2), encoding="utf-8"
86     )
87     return {"q4_2": q4_2, "q4_3": q4_3, "comparison": comparison}
88
89
90 if __name__ == "__main__":
91     print(json.dumps(run_q4()["comparison"], ensure_ascii=False, indent=2))

```

B.12 官方结果文件导出 (export.py)

Listing 12:

```

1  from __future__ import annotations
2  from copy import copy
3  from datetime import datetime, time
4  from pathlib import Path
5  from shutil import copy2
6  import numpy as np
7  import openpyxl
8  from .lp_core import DispatchResult
9
10 def export_result1(template: Path, destination: Path, result: DispatchResult) -> dict:
11     destination.parent.mkdir(parents=True, exist_ok=True); copy2(template, destination)
12     wb=openpyxl.load_workbook(destination); plan, storage=wb.worksheets[:2]
13     for row, value in enumerate(result.grid, start=2):
14         plan.cell(row, 2, float(value)); plan.cell(row, 2).number_format="0.0000"
15     for block in range(6):
16         lo, hi=block*24, (block+1)*24
17         storage.cell(block+2, 2, float(result.charge[lo:hi].sum()))
18         storage.cell(block+2, 3, float(result.discharge[lo:hi].sum()))
19     storage.cell(2, 5, 6000.0); storage.cell(3, 5, float(result.soc[-1])); wb.save(destination)
20     check=openpyxl.load_workbook(destination, data_only=True, read_only=True)
21     values=np.asarray([check.worksheets[0].cell(i, 2).value for i in range(2, 146)], dtype=
22                       float)
23     diff=float(np.max(np.abs(values-result.grid)))
24     return {"rows": len(values), "max_plan_roundtrip_diff": diff, "passed": bool(diff<1e-9)}
25
26 def _copy_row_style(ws, source_row: int, target_row: int, max_col: int) -> None:
27     for col in range(1, max_col+1):

```

```

27         src=ws.cell(source_row,col); dst=ws.cell(target_row,col)
28         if src.has_style: dst._style=copy(src._style)
29         dst.alignment=copy(src.alignment); dst.protection=copy(src.protection)
30     ws.row_dimensions[target_row].height=ws.row_dimensions[source_row].height
31
32 def _segments(values:np.ndarray,tolerance:float=1e-7)->list[tuple[int,int,float]]:
33     result=[]; start=None
34     for idx,flag in enumerate(np.r_[np.asarray(values)>tolerance,False]):
35         if flag and start is None: start=idx
36         elif not flag and start is not None:
37             result.append((start,idx,float(np.sum(values[start:idx])))); start=None
38     return result
39
40 def _slot_time(slot:int)->str:
41     minutes=slot*10
42     return "24:00" if minutes==1440 else f"{minutes//60}:{minutes%60:02d}"
43
44 def export_result2(template:Path,destination:Path,days:list[dict])->dict:
45     if len(days)!=334: raise ValueError(f"result2 requires 334 days, got {len(days)}")
46     destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
47     wb=openpyxl.load_workbook(destination); plan_ws,storage_ws,emergency_ws=wb.worksheets
48     [:3]
49     for idx,day in enumerate(days):
50         row=idx+2; plan=day["plan"]
51         plan_ws.cell(row,1,datetime.combine(day["date"],time()))
52         for slot,value in enumerate(plan.grid,start=2):
53             plan_ws.cell(row,slot,float(value)); plan_ws.cell(row,slot).number_format="
54                 0.0000"
55             plan_ws.cell(row,146,float(np.sum(plan.grid))); plan_ws.cell(row,147,float(day["
56                 plan_cost"]))
57             plan_ws.cell(row,146).number_format=plan_ws.cell(row,147).number_format="0.0000"
58     target=1+6*len(days)
59     if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
60         storage_ws.max_row)
61     periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
62     for idx,day in enumerate(days):
63         actual=day["execution"]
64         for block in range(6):
65             row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
66             storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None
67                 )
68             storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
69             storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
70             storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
71             storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
72                 )
73             storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.

```

```

        soc[-1]) if block==1 else None))
68         for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
69     if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
70     rows=[]
71     for day in days:
72         segments=_segments(day["execution"].emergency)
73         if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
74         else:
75             for k,(start,end,total) in enumerate(segments):
76                 rows.append((datetime.combine(day["date"],time()) if k==0 else None,
77                             f"{_slot_time(start)}-{_slot_time(end)}",total))
78     target_e=1+len(rows)
79     if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
                        =target_e-emergency_ws.max_row)
80     for row,(date_value,period,total) in enumerate(rows,start=2):
81         _copy_row_style(emergency_ws,2,row,3)
82         emergency_ws.cell(row,1,date_value); emergency_ws.cell(row,2,period)
83         emergency_ws.cell(row,3,float(total)); emergency_ws.cell(row,3).number_format="
            0.0000"
84     if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
        max_row-target_e)
85     wb.save(destination)
86     check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
87     plan_rows=list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col
        =145,values_only=True))
88     values=np.asarray(plan_rows,dtype=float)
89     expected=np.stack([d["plan"].grid for d in days]); diff=float(np.max(np.abs(values-
        expected)))
90     last_row=3+6*(len(days)-1)
91     last_soc=float(next(check.worksheets[1].iter_rows(min_row=last_row,max_row=last_row,
        min_col=6,max_col=6,values_only=True))[0])
92     return {"delivery_days":334,"plan_rows":values.shape[0],"plan_slots":values.shape[1],
93            "storage_rows":check.worksheets[1].max_row-1,"emergency_rows":check.worksheets
        [2].max_row-1,
94            "max_plan_roundtrip_diff":diff,"last_terminal_soc":last_soc,
95            "passed":bool(values.shape==(334,144) and diff<1e-9 and check.worksheets[1].
        max_row-1==2004)}
96
97
98 def export_result3(template:Path,destination:Path,days:list[dict])>->dict:
99     if len(days)!=334: raise ValueError(f"result3 requires 334 days, got {len(days)}")
100     destination.parent.mkdir(parents=True,exist_ok=True); copy2(template,destination)
101     wb=openpyxl.load_workbook(destination); plan_ws,adjust_ws,storage_ws,emergency_ws=wb.
        worksheets[:4]
102     for idx,day in enumerate(days):
103         row=idx+2; plan=day["baseline_plan"]; adjusted=day["adjusted_grid"]
104         for ws,values,cost in ((plan_ws,plan.grid,day["baseline_cost"]), (adjust_ws,adjusted,

```

```

        day["settlement_cost"])):
105     ws.cell(row,1,datetime.combine(day["date"],time()))
106     for slot,value in enumerate(values,start=2):
107         ws.cell(row,slot,float(value)); ws.cell(row,slot).number_format="0.0000"
108     ws.cell(row,146,float(np.sum(values))); ws.cell(row,147,float(cost))
109     ws.cell(row,146).number_format=ws.cell(row,147).number_format="0.0000"
110 target=1+6*len(days)
111 if storage_ws.max_row<target: storage_ws.insert_rows(storage_ws.max_row+1,amount=target-
    storage_ws.max_row)
112 periods=("0:00-4:00","4:00-8:00","8:00-12:00","12:00-16:00","16:00-20:00","20:00-24:00")
113 for idx,day in enumerate(days):
114     actual=day["execution"]
115     for block in range(6):
116         row=2+idx*6+block; _copy_row_style(storage_ws,2+block,row,6)
117         storage_ws.cell(row,1,datetime.combine(day["date"],time()) if block==0 else None
            )
118         storage_ws.cell(row,2,periods[block]); lo,hi=block*24,(block+1)*24
119         storage_ws.cell(row,3,float(actual.charge[lo:hi].sum()))
120         storage_ws.cell(row,4,float(actual.discharge[lo:hi].sum()))
121         storage_ws.cell(row,5,time(0,0) if block==0 else ("24:00" if block==1 else None)
            )
122         storage_ws.cell(row,6,float(day["soc_initial"]) if block==0 else (float(actual.
            soc[-1]) if block==1 else None))
123         for col in (3,4,6): storage_ws.cell(row,col).number_format="0.0000"
124 if storage_ws.max_row>target: storage_ws.delete_rows(target+1,storage_ws.max_row-target)
125 rows=[]
126 for day in days:
127     segments=_segments(day["execution"].emergency)
128     if not segments: rows.append((datetime.combine(day["date"],time()),"无",0.0))
129     else:
130         for k,(start,end,total) in enumerate(segments):
131             rows.append((datetime.combine(day["date"],time()) if k==0 else None,
132                 f"{_slot_time(start)}-{_slot_time(end)}",total))
133 target_e=1+len(rows)
134 if emergency_ws.max_row<target_e: emergency_ws.insert_rows(emergency_ws.max_row+1,amount
    =target_e-emergency_ws.max_row)
135 for row,(date_value,period,total) in enumerate(rows,start=2):
136     _copy_row_style(emergency_ws,2,row,3); emergency_ws.cell(row,1,date_value)
137     emergency_ws.cell(row,2,period); emergency_ws.cell(row,3,float(total))
138     emergency_ws.cell(row,3).number_format="0.0000"
139 if emergency_ws.max_row>target_e: emergency_ws.delete_rows(target_e+1,emergency_ws.
    max_row-target_e)
140 wb.save(destination)
141 check=openpyxl.load_workbook(destination,data_only=True,read_only=True)
142 plan_values=np.asarray(list(check.worksheets[0].iter_rows(min_row=2,max_row=335,min_col
    =2,max_col=145,values_only=True)),dtype=float)
143 adjust_values=np.asarray(list(check.worksheets[1].iter_rows(min_row=2,max_row=335,

```

```

min_col=2,max_col=145,values_only=True)),dtype=float)
144 expected_plan=np.stack([d["baseline_plan"].grid for d in days]); expected_adjust=np.
stack([d["adjusted_grid"] for d in days])
145 return {"delivery_days":334,"plan_shape":list(plan_values.shape),"adjust_shape":list(
adjust_values.shape),
146         "storage_rows":check.worksheets[2].max_row-1,"emergency_rows":check.worksheets
[3].max_row-1,
147         "max_plan_roundtrip_diff":float(np.max(np.abs(plan_values-expected_plan))),
148         "max_adjust_roundtrip_diff":float(np.max(np.abs(adjust_values-expected_adjust)))
,
149         "passed":bool(plan_values.shape==(334,144) and adjust_values.shape==(334,144)
and
150                             check.worksheets[2].max_row-1==2004 and
151                             np.max(np.abs(plan_values-expected_plan))<1e-9 and np.max(np.abs(
adjust_values-expected_adjust))<1e-9)}

```

B.13 模型审计与消融实验总控 (ablation.py)

Listing 13:

```

1  """V4 模型审计总控: Q2 原因拆解 / Q3 节点消融与结算口径 / Q4 价格边界 / 敏感性。
2
3  所有变体产物统一写入 MathModel/outputs/variants/model_audit_v4/, 不覆盖 V3 结果。
4  """
5  from __future__ import annotations
6
7  import json
8  import time
9  from pathlib import Path
10
11  import numpy as np
12
13  from .io_data import DEFAULT_DATA_ROOT, read_attachment1, read_attachment4
14  from .params import DEFAULT_STORAGE, EFFICIENCY_VARIANTS
15  from .q1 import run_q1
16  from .q2 import run_q2
17  from .q3 import run_q3
18
19  ROOT = Path(__file__).resolve().parents[1]
20  VARIANTS = ROOT / "outputs" / "variants" / "model_audit_v4"
21
22  # Q2 原因拆解: 只改变一个因素
23  Q2_CASES = {
24      "A_perfectload_risk_feedback": dict(load_mode="perfect", pv_risk_mode="asymmetric",
25                                         realtime_feedback=True),
26      "B_causalload_risk_feedback": dict(load_mode="causal", pv_risk_mode="asymmetric",

```

```

        realtime_feedback=True),
26     "C_causalload_point_feedback": dict(load_mode="causal", pv_risk_mode="point",
        realtime_feedback=True),
27     "D_causalload_risk_nofeedback": dict(load_mode="causal", pv_risk_mode="asymmetric",
        realtime_feedback=False),
28 }
29
30 Q3_MAIN_CASES = {
31     "S0_0": dict(update_nodes=(0,)),
32     "S1_0_6": dict(update_nodes=(0, 36)),
33     "S2_0_6_12": dict(update_nodes=(0, 36, 72)),
34     "S3_0_6_12_18": dict(update_nodes=(0, 36, 72, 108)),
35 }
36 Q3_CONTROL_CASES = {
37     # 三个成对控制臂：先只加入该节点的负载刷新，再加入同节点新光伏预报。
38     "C1_load6": dict(update_nodes=(0, 36), pv_update_nodes=(0,)),
39     "C2_load12": dict(update_nodes=(0, 36, 72), pv_update_nodes=(0, 36)),
40     "C3_load18": dict(update_nodes=(0, 36, 72, 108), pv_update_nodes=(0, 36, 72)),
41 }
42 Q3_CASES = {**Q3_MAIN_CASES, **Q3_CONTROL_CASES}
43
44 Q4_CASES = {
45     "q42_perfect_price": ("q2", dict(price_mode="perfect")),
46     "q42_causal_price": ("q2", dict(price_mode="causal")),
47     "q43_perfect_price": ("q3", dict(price_mode="perfect")),
48     "q43_causal_price": ("q3", dict(price_mode="causal")),
49 }
50
51 SEASONS = {
52     "spring": (3, 4, 5),
53     "summer": (6, 7, 8),
54     "autumn": (9, 10, 11),
55     "winter": (12, 1, 2),
56 }
57
58
59 def _season(month: int) -> str:
60     for name, months in SEASONS.items():
61         if month in months:
62             return name
63     raise ValueError(month)
64
65
66 def run_q2_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
67     out = VARIANTS / "q2"
68     summary = {}
69     for name, kwargs in Q2_CASES.items():

```

```

70         started = time.perf_counter()
71         result = run_q2(
72             data_root=data_root,
73             max_days=max_days,
74             output_dir=out / name,
75             template_name="result2.xlsx",
76             write_outputs=True,
77             tag=name,
78             **kwargs,
79         )
80         metrics = result["metrics"]
81         metrics["wall_seconds"] = time.perf_counter() - started
82         (out / name / "metrics.json").write_text(
83             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
84         )
85         summary[name] = metrics
86         print(f"[Q2] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('emergency_energy_kwh')}")
87     _dump(out / "summary.json", summary)
88     return summary
89
90
91 def run_q3_ablation(
92     *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "additive"
93 ) -> dict:
94     # 两种结算口径的输出必须分目录，否则后跑的一轮会覆盖前一轮的
95     # metrics.json 与官方 result3.xlsx，导致论文与提交文件口径不一致。
96     out = VARIANTS / f"q3_{settlement_mode}"
97     summary = {}
98     for name, kwargs in Q3_CASES.items():
99         started = time.perf_counter()
100         result = run_q3(
101             data_root=data_root,
102             max_days=max_days,
103             output_dir=out / name,
104             template_name="result3.xlsx",
105             write_outputs=True,
106             settlement_mode=settlement_mode,
107             tag=name,
108             **kwargs,
109         )
110         metrics = result["metrics"]
111         metrics["wall_seconds"] = time.perf_counter() - started
112         (out / name / "metrics.json").write_text(
113             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
114         )

```

```

115         # 季节分解
116         (out / name / "by_season.csv").write_text(_season_table(result["records"]), encoding
            ="utf-8-sig")
117         summary[name] = metrics
118         print(f"[Q3-{settlement_mode}] {name}: total={metrics.get('total_cost_yuan')}")
119     _dump(out / "summary.json", summary)
120     return summary
121
122
123 def run_q3_controls(
124     *, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT, settlement_mode: str = "
        additive"
125 ) -> dict:
126     """只补跑成对控制臂，并合并进既有 summary.json。"""
127     out = VARIANTS / f"q3_{settlement_mode}"
128     summary_path = out / "summary.json"
129     summary = json.loads(summary_path.read_text(encoding="utf-8")) if summary_path.exists()
        else {}
130     for name, kwargs in Q3_CONTROL_CASES.items():
131         started = time.perf_counter()
132         result = run_q3(
133             data_root=data_root,
134             max_days=max_days,
135             output_dir=out / name,
136             template_name="result3.xlsx",
137             write_outputs=True,
138             settlement_mode=settlement_mode,
139             tag=name,
140             **kwargs,
141         )
142         metrics = result["metrics"]
143         metrics["wall_seconds"] = time.perf_counter() - started
144         (out / name / "metrics.json").write_text(
145             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
146         )
147         (out / name / "by_season.csv").write_text(
148             _season_table(result["records"]), encoding="utf-8-sig"
149         )
150         summary[name] = metrics
151         print(f"[Q3-{settlement_mode}-control] {name}: total={metrics.get('total_cost_yuan')}
            ")
152     _dump(summary_path, summary)
153     return summary
154
155
156 def _season_table(records: list[dict]) -> str:
157     delivery = records[31:]

```



```

158     rows = ["season,days,total_cost_yuan,baseline_cost_yuan,emergency_energy_kwh,
159             up_energy_kwh,down_energy_kwh,curtailment_kwh"]
160     for season in ("spring", "summer", "autumn", "winter"):
161         subset = [r for r in delivery if _season(r["date"].month) == season]
162         if not subset:
163             continue
164         rows.append(
165             ", ".join(
166                 str(x)
167                 for x in [
168                     season,
169                     len(subset),
170                     round(sum(r["total_cost"] for r in subset), 4),
171                     round(sum(r["baseline_cost"] for r in subset), 4),
172                     round(sum(r["execution"].emergency.sum() for r in subset), 4),
173                     round(sum(r["up_energy"] for r in subset), 4),
174                     round(sum(r["down_energy"] for r in subset), 4),
175                     round(sum(r["execution"].curtail.sum() for r in subset), 4),
176                 ]
177             )
178         )
179     return "\n".join(rows) + "\n"
180
181 def run_q4_ablation(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
182     out = VARIANTS / "q4"
183     prices = read_attachment4(data_root).price
184     summary = {}
185     for name, (kind, kwargs) in Q4_CASES.items():
186         started = time.perf_counter()
187         if kind == "q2":
188             result = run_q2(
189                 data_root=data_root,
190                 max_days=max_days,
191                 price_matrix=prices,
192                 output_dir=out / name,
193                 template_name="result4-2.xlsx",
194                 write_outputs=True,
195                 load_mode="causal",
196                 tag=name,
197                 **kwargs,
198             )
199         else:
200             result = run_q3(
201                 data_root=data_root,
202                 max_days=max_days,
203                 price_matrix=prices,

```

```

204         output_dir=out / name,
205         template_name="result4-3.xlsx",
206         write_outputs=True,
207         load_mode="causal",
208         settlement_mode="additive",
209         update_nodes=(0, 36, 72, 108),
210         tag=name,
211         **kwargs,
212     )
213     metrics = result["metrics"]
214     metrics["wall_seconds"] = time.perf_counter() - started
215     (out / name / "metrics.json").write_text(
216         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
217     )
218     summary[name] = metrics
219     print(f"[Q4] {name}: total={metrics.get('total_cost_yuan')} emergency={metrics.get('emergency_energy_kwh')}")
220     _dump(out / "summary.json", summary)
221     return summary
222
223
224 def run_sensitivity(*, max_days: int = 365, data_root: Path = DEFAULT_DATA_ROOT) -> dict:
225     out = VARIANTS / "sensitivity"
226     summary = {"terminal": {}, "efficiency": {}}
227     for policy in ("daily_cycle", "free"):
228         started = time.perf_counter()
229         result = run_q2(
230             data_root=data_root,
231             max_days=max_days,
232             output_dir=out / "terminal" / policy,
233             template_name="result2.xlsx",
234             write_outputs=True,
235             load_mode="causal",
236             pv_risk_mode="asymmetric",
237             realtime_feedback=True,
238             terminal_policy=policy,
239             tag=f"terminal_{policy}",
240         )
241         metrics = result["metrics"]
242         metrics["wall_seconds"] = time.perf_counter() - started
243         daily_soc = [float(r["execution"].soc[-1]) for r in result["records"][31:]]
244         metrics["daily_boundary_soc"] = {
245             "min": float(np.min(daily_soc)),
246             "max": float(np.max(daily_soc)),
247             "mean": float(np.mean(daily_soc)),
248             "std": float(np.std(daily_soc)),
249         }

```

```

250         (out / "terminal" / policy / "metrics.json").write_text(
251             json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
252         )
253     summary["terminal"][policy] = metrics
254     print(f"[P1-terminal] {policy}: total={metrics.get('total_cost_yuan')} endSOC={
        metrics.get('ending_soc_kwh')}")
255 for name, storage in EFFICIENCY_VARIANTS.items():
256     started = time.perf_counter()
257     result = run_q2(
258         data_root=data_root,
259         max_days=max_days,
260         output_dir=out / "efficiency" / name,
261         template_name="result2.xlsx",
262         write_outputs=True,
263         load_mode="causal",
264         pv_risk_mode="asymmetric",
265         realtime_feedback=True,
266         storage=storage,
267         tag=f"efficiency_{name}",
268     )
269     metrics = result["metrics"]
270     metrics["wall_seconds"] = time.perf_counter() - started
271     (out / "efficiency" / name / "metrics.json").write_text(
272         json.dumps(metrics, ensure_ascii=False, indent=2), encoding="utf-8"
273     )
274     summary["efficiency"][name] = metrics
275     print(f"[P1-efficiency] {name}: total={metrics.get('total_cost_yuan')}")
276 _dump(out / "summary.json", summary)
277 return summary
278
279
280 def _dump(path: Path, payload: dict) -> None:
281     path.parent.mkdir(parents=True, exist_ok=True)
282     path.write_text(json.dumps(payload, ensure_ascii=False, indent=2, default=str), encoding
        ="utf-8")
283
284
285 def main() -> None:
286     """按 V4 采用顺序重跑全部年度对照；该过程耗时较长。"""
287     run_q1()
288     run_q2_ablation()
289     run_q3_ablation(settlement_mode="additive")
290     run_q3_ablation(settlement_mode="replacement")
291     run_q4_ablation()
292     run_sensitivity()
293
294

```

```

295 if __name__ == "__main__":
296     main()

```

B.14 官方光伏预报处理 (official_forecast.py)

Listing 14:

```

1  from __future__ import annotations
2  from dataclasses import dataclass
3  import numpy as np
4  from .io_data import DT_HOURS
5
6  QUANTILES=(0.80,0.85,0.90,0.95)
7
8  @dataclass(frozen=True)
9  class CalibrationDecision:
10     issue_index:int; quantile:float; margin_kw:float; history_days_used:int
11     raw_forecast_kw:np.ndarray; forecast_kw:np.ndarray; candidates:dict[float,np.ndarray]
12     historical_score:float|None
13
14  class OnlineForecastBiasCalibrator:
15     """Issue-specific causal bias correction for official 24 h PV forecasts."""
16     def __init__(self,residual_window:int=30,score_window:int=30):
17         self.residual_window=residual_window; self.score_window=score_window
18         self.residual_days=[[] for _ in range(4)]
19         self.score_days=[{q:[] for q in QUANTILES} for _ in range(4)]
20     def calibrate(self,issue_index:int,raw_forecast_kw:np.ndarray)->CalibrationDecision:
21         raw=np.asarray(raw_forecast_kw,dtype=float)
22         days=self.residual_days[issue_index][-self.residual_window:]
23         residual=np.concatenate(days) if days else np.array([])
24         margins={q:(max(0.0,float(np.quantile(residual,q))) if residual.size else 0.0) for q
25                  in QUANTILES}
26         candidates={q:np.clip(raw-margins[q],0.0,None) for q in QUANTILES}
27         history_count=len(self.residual_days[issue_index])
28         if history_count<7: q=0.80; hist=None
29         else:
30             scores={x:float(np.mean(self.score_days[issue_index][x][-self.score_window:]))
31                    for x in QUANTILES}
32             q=min(scores,key=lambda x:(scores[x],x)); hist=scores[q]
33         return CalibrationDecision(issue_index,q,margins[q],history_count,raw.copy(),
34                                   candidates[q],candidates,hist)
35
36     def observe(self,decision:CalibrationDecision,actual_block_kw:np.ndarray,price_block:np.
37                ndarray)->None:
38         actual=np.asarray(actual_block_kw,dtype=float); price=np.asarray(price_block,dtype=
39                                float)
40         if actual.shape!=(36,) or price.shape!=(36,): raise ValueError("calibration block

```

```

        must contain 36 slots")
35     issue=decision.issue_index
36     if decision.history_days_used!=len(self.residual_days[issue]): raise ValueError("
        stale calibration decision")
37     for q,forecast in decision.candidates.items():
38         error=forecast[:36]-actual
39         score=float(np.sum(price*(4*np.maximum(error,0)+np.maximum(-error,0))*DT_HOURS)
40         self.score_days[issue][q].append(score)
41     raw_error=decision.raw_forecast_kw[:36]-actual
42     mask=(decision.raw_forecast_kw[:36]>1e-9)|(actual>1e-9)
43     self.residual_days[issue].append(raw_error[mask].copy())

```

B.15 指定日期论文表格生成 (paper_tables.py)

Listing 15:

```

1  """从 V4 最终逐时输出自动生成题面指定日期的 LaTeX 表格。"""
2  from __future__ import annotations
3
4  import csv
5  from collections import defaultdict
6  from pathlib import Path
7
8  import numpy as np
9  import openpyxl
10
11  ROOT = Path(__file__).resolve().parents[1]
12  VARIANTS = ROOT / "outputs" / "variants" / "model_audit_v4"
13  GENERATED = ROOT / "paper" / "generated"
14  TEMPLATE = ROOT.parent / "CUMCM2026Problems" / "C题" / "附件" / "附件5" / "result2.xlsx"
15  DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
16  INTERVALS = ("10:00-10:10", "12:00-12:10", "14:00-14:10", "16:00-16:10", "18:00-18:10", "
    20:00-20:10")
17  PERIODS = ("0:00--4:00", "4:00--8:00", "8:00--12:00", "12:00--16:00", "16:00--20:00", "
    20:00--24:00")
18
19  SOURCES = {
20      "q2": VARIANTS / "q2" / "B_causalload_risk_feedback",
21      "q3": VARIANTS / "q3_additive" / "S3_0_6_12_18",
22      "q42": VARIANTS / "q4" / "q42_perfect_price",
23      "q43": VARIANTS / "q4" / "q43_perfect_price",
24  }
25
26
27  def _rows(path: Path) -> list[dict[str, str]]:
28      with path.open(encoding="utf-8-sig", newline="") as handle:

```

```

29         return list(csv.DictReader(handle))
30
31
32 def _slot_indices() -> dict[str, int]:
33     workbook = openpyxl.load_workbook(TEMPLATE, data_only=True, read_only=True)
34     headers = list(next(workbook.worksheets[0].iter_rows(min_row=1, max_row=1, values_only=
35         True)))
36     workbook.close()
37     # 官方模板第 2 列对应逐时数组下标 0; 以模板表头定位, 避免手工时标偏移。
38     return {label: headers.index(label) - 1 for label in INTERVALS}
39
40 def emergency_segments(values: np.ndarray, tol: float = 1e-7) -> list[tuple[str, float]]:
41     output: list[tuple[str, float]] = []
42     start = None
43
44     def stamp(slot: int) -> str:
45         minutes = slot * 10
46         return "24:00" if minutes == 1440 else f"{minutes // 60}:{minutes % 60:02d}"
47
48     for index, active in enumerate(np.r_[np.asarray(values, dtype=float) > tol, False]):
49         if active and start is None:
50             start = index
51         elif not active and start is not None:
52             output.append((f"{stamp(start)}--{stamp(index)}", float(np.sum(values[start:
53                 index]))))
54             start = None
55     return output
56
57 def _dataset(key: str) -> tuple[dict[str, list[dict[str, str]]], dict[str, dict[str, str]]]:
58     grouped: dict[str, list[dict[str, str]]] = defaultdict(list)
59     for row in _rows(SOURCES[key] / "solution.csv"):
60         grouped[row["date"]].append(row)
61     daily = {row["date"]: row for row in _rows(SOURCES[key] / "daily_metrics.csv")}
62     missing = [date for date in DATES if date not in grouped or date not in daily]
63     if missing:
64         raise ValueError(f"{key} 缺少指定日期: {missing}")
65     return grouped, daily
66
67
68 def _purchase_table(key: str, caption: str, label: str, adjusted: bool) -> str:
69     grouped, daily = _dataset(key)
70     indices = _slot_indices()
71     lines = [
72         r"\begin{table}[htbp]",
73         r"\centering\zihao{-5}",

```

```

74         rf"\caption{{{caption}}}",
75         rf"\label{{{label}}}",
76         r"\resizebox{\textwidth}{!}{%",
77     ]
78     if adjusted:
79         lines += [
80             r"\begin{tabular}{c c r r r r r r r}",
81             r"\toprule",
82             r"日期 & 口径 & 10:00 & 12:00 & 14:00 & 16:00 & 18:00 & 20:00 & 全天/kWh & 费用/元 \\",
83             r"\midrule",
84         ]
85     else:
86         lines += [
87             r"\begin{tabular}{c r r r r r r r r}",
88             r"\toprule",
89             r"日期 & 10:00 & 12:00 & 14:00 & 16:00 & 18:00 & 20:00 & 全天/kWh & 计划费/元 \\",
90             r"\midrule",
91         ]
92     for date in DATES:
93         day = grouped[date]
94         drow = daily[date]
95         if adjusted:
96             for row_index, (name, column, cost_field) in enumerate(
97                 (("计划", "baseline_grid_kwh", "baseline_cost"), ("调整", "adjusted_grid_kwh", "settlement_cost"))
98             ):
99                 values = [float(day[indices[item]][column]) for item in INTERVALS]
100                 total = sum(float(row[column]) for row in day)
101                 date_cell = rf"\multirow{{{2}}}{{{*}}}{{{date}}}" if row_index == 0 else ""
102                 lines.append(
103                     f"{date_cell} & {name} & " + " & ".join(f"{value:.2f}" for value in values)
104                     + f" & {total:.2f} & {float(drow[cost_field]):.2f} \\\\"
105                 )
106         else:
107             values = [float(day[indices[item]]["grid_plan_kwh"]) for item in INTERVALS]
108             total = sum(float(row["grid_plan_kwh"]) for row in day)
109             lines.append(
110                 f"{date} & " + " & ".join(f"{value:.2f}" for value in values)
111                 + f" & {total:.2f} & {float(drow['plan_cost']):.2f} \\\\"
112             )
113     lines += [r"\bottomrule", r"\end{tabular}", r"\end{table}"]
114     return "\n".join(lines)
115
116

```

```

117 def _storage_table(key: str, caption: str, label: str) -> str:
118     grouped, daily = _dataset(key)
119     lines = [
120         r"\begin{table}[htbp]",
121         r"\centering\zihao{-5}",
122         rf"\caption{{{caption}}}",
123         rf"\label{{{label}}}",
124         r"\resizebox{\textwidth}{!}{%",
125         r"\begin{tabular}{c r r r r r r r}",
126         r"\toprule",
127         r"日期 & \multicolumn{6}{c}{各 4 h 时段充电量/放电量 (kWh) } & \${0:00}$ & \${0:00}$ \\",
128         r"\cmidrule(lr){2-7}",
129         "日期 & " + " & ".join(PERIODS) + r" & kWh & kWh \\",
130         r"\midrule",
131     ]
132     for date in DATES:
133         day = grouped[date]
134         charge = np.asarray([float(row["charge_actual_kwh"]) for row in day])
135         discharge = np.asarray([float(row["discharge_actual_kwh"]) for row in day])
136         pairs = [f"{charge[i * 24:(i + 1) * 24].sum():.2f}/{discharge[i * 24:(i + 1) * 24].sum():.2f}" for i in range(6)]
137         lines.append(
138             f"{date} & " + " & ".join(pairs)
139             + f" & {float(daily[date]['soc_initial']):.2f} & {float(daily[date]['soc_terminal']):.2f} \\\\"
140         )
141     lines += [r"\bottomrule", r"\end{tabular}}", r"\end{table}"]
142     return "\n".join(lines)
143
144
145 def _emergency_table(key: str, caption: str, label: str) -> str:
146     grouped, daily = _dataset(key)
147     lines = [
148         r"\begin{table}[htbp]",
149         r"\centering\zihao{-5}",
150         rf"\caption{{{caption}}}",
151         rf"\label{{{label}}}",
152         r"\begin{tabularx}{0.98\textwidth}{c >{\raggedright\arraybackslash}X r r}",
153         r"\toprule",
154         r"日期 & 紧急购电时段及对应电量/kWh & 合计/kWh & 当日总费用/元 \\",
155         r"\midrule",
156     ]
157     for date in DATES:
158         day = grouped[date]
159         segments = emergency_segments(np.asarray([float(row["emergency_kwh"]) for row in day
160 ]))

```



```

160     description = "无" if not segments else "; ".join(f"{period} ({value:.2f}) " for
161         period, value in segments)
162     total = sum(value for _, value in segments)
163     lines.append(f"{date} & {description} & {total:.2f} & {float(daily[date]['total_cost
164         ']):.2f} \\\ \"")
165
166
167     lines += [r"\bottomrule", r"\end{tabularx}", r"\end{table}"]
168     return "\n".join(lines)
169
170
171 def _bundle(key: str, title: str, prefix: str, adjusted: bool) -> str:
172     return "\n\n".join(
173         (
174             _purchase_table(key, f"{title}指定日期购电结果", f"tab:{prefix}-purchase",
175                 adjusted),
176             _storage_table(key, f"{title}指定日期储能结果", f"tab:{prefix}-storage"),
177             _emergency_table(key, f"{title}指定日期紧急购电结果", f"tab:{prefix}-emergency")
178         ),
179         '\n\n'
180     )
181
182 def build() -> None:
183     GENERATED.mkdir(parents=True, exist_ok=True)
184     outputs = {
185         "q2_required_tables.tex": _bundle("q2", "问题二", "q2-required", False),
186         "q3_required_tables.tex": _bundle("q3", "问题三", "q3-required", True),
187         "q4_required_tables.tex": (
188             _bundle("q42", "问题四对应问题二", "q42-required", False)
189             + "\n\n"
190             + _bundle("q43", "问题四对应问题三", "q43-required", True)
191         ),
192     }
193
194     for name, content in outputs.items():
195         path = GENERATED / name
196         path.write_text("% 由 MathModel.solve.paper_tables 自动生成，勿手工修改。\\n" +
197             content, encoding="utf-8")
198         print("[table]", path)
199
200 if __name__ == "__main__":
201     build()

```

B.16 问题二结果验证 (validate_q2.py)

Listing 16:

```

1  from __future__ import annotations
2  import csv,json
3  from pathlib import Path
4  import numpy as np
5  import openpyxl
6
7  ROOT=Path(__file__).resolve().parents[1]
8  OUT=ROOT/"outputs"/"q2"
9
10 def validate()->dict:
11     with (OUT/"solution.csv").open(encoding="utf-8-sig",newline="") as f:
12         rows=list(csv.DictReader(f))
13         if len(rows)!=334*144: raise ValueError(f"solution rows: {len(rows)}")
14         grid=np.asarray([float(r["grid_plan_kwh"]) for r in rows]).reshape(334,144)
15         charge=np.asarray([float(r["charge_actual_kwh"]) for r in rows]).reshape(334,144)
16         discharge=np.asarray([float(r["discharge_actual_kwh"]) for r in rows]).reshape(334,144)
17         emergency=np.asarray([float(r["emergency_kwh"]) for r in rows]).reshape(334,144)
18         with (OUT/"daily_metrics.csv").open(encoding="utf-8-sig",newline="") as f:
19             daily=list(csv.DictReader(f))
20             wb=openpyxl.load_workbook(OUT/"result2.xlsx",data_only=True,read_only=True)
21             template=openpyxl.load_workbook(ROOT.parent/"CUMCM2026Problems"/"C题"/"附件"/"附件5"/
                "result2.xlsx",
22                                     data_only=True,read_only=True)
23             plan_rows=list(wb.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col=147,
                values_only=True))
24             plan_values=np.asarray([r[:144] for r in plan_rows],dtype=float)
25             total_values=np.asarray([r[144] for r in plan_rows],dtype=float)
26             cost_values=np.asarray([r[145] for r in plan_rows],dtype=float)
27             storage_rows=list(wb.worksheets[1].iter_rows(min_row=2,max_row=2005,min_col=1,max_col=6,
                values_only=True))
28             storage_charge=np.asarray([float(r[2]) for r in storage_rows]).reshape(334,6)
29             storage_discharge=np.asarray([float(r[3]) for r in storage_rows]).reshape(334,6)
30             storage_initial=np.asarray([float(storage_rows[d*6][5]) for d in range(334)])
31             storage_terminal=np.asarray([float(storage_rows[d*6+1][5]) for d in range(334)])
32             expected_charge=charge.reshape(334,6,24).sum(axis=2)
33             expected_discharge=discharge.reshape(334,6,24).sum(axis=2)
34             emergency_rows=list(wb.worksheets[2].iter_rows(min_row=2,min_col=1,max_col=3,values_only
                =True))
35             emergency_sheet_total=float(sum(float(r[2] or 0) for r in emergency_rows))
36             expected_initial=np.asarray([float(r["soc_initial"]) for r in daily])
37             expected_terminal=np.asarray([float(r["soc_terminal"]) for r in daily])
38             expected_cost=np.asarray([float(r["plan_cost"]) for r in daily])
39             result={
40                 "sheet_names_preserved":wb.sheetnames==template.sheetnames,
41                 "solution_rows":len(rows),"plan_shape":list(plan_values.shape),"storage_rows":len(
                    storage_rows),

```

```

42     "max_plan_diff":float(np.max(np.abs(plan_values-grid))),
43     "max_plan_total_diff":float(np.max(np.abs(total_values-grid.sum(axis=1)))),
44     "max_plan_cost_diff":float(np.max(np.abs(cost_values-expected_cost))),
45     "max_storage_charge_diff":float(np.max(np.abs(storage_charge-expected_charge))),
46     "max_storage_discharge_diff":float(np.max(np.abs(storage_discharge-
        expected_discharge))),
47     "max_storage_initial_soc_diff":float(np.max(np.abs(storage_initial-expected_initial)
        )),
48     "max_storage_terminal_soc_diff":float(np.max(np.abs(storage_terminal-
        expected_terminal))),
49     "emergency_sheet_total_kwh":emergency_sheet_total,
50     "emergency_csv_total_kwh":float(emergency.sum()),
51     "emergency_total_diff":abs(emergency_sheet_total-float(emergency.sum())),
52 }
53 result["passed"]=bool(result["sheet_names_preserved"] and result["plan_shape"
    ]==[334,144] and
54     len(storage_rows)==2004 and max(result[k] for k in result if k.startswith("max_")<1
        e-7 and
55     result["emergency_total_diff"]<1e-7)
56 (OUT/"xlsx_verification.json").write_text(json.dumps(result,ensure_ascii=False,indent=2)
    ,encoding="utf-8")
57 metrics=json.loads((OUT/"metrics.json").read_text(encoding="utf-8"))
58 metrics["xlsx_passed"]=result["passed"]
59 (OUT/"metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,indent=2),encoding
    ="utf-8")
60 return result
61
62 if __name__=="__main__": print(json.dumps(validate(),ensure_ascii=False,indent=2))

```

B.17 全部方案物理验证（validate_all.py）

Listing 17:

```

1  from __future__ import annotations
2  import csv,json
3  from pathlib import Path
4  import numpy as np
5  import openpyxl
6
7  ROOT=Path(__file__).resolve().parents[1]
8  DATA=ROOT.parent/"CUMCM2026Problems"/"C题"/"附件"/"附件5"
9
10 def _csv(path:Path)->list[dict]:
11     with path.open(encoding="utf-8-sig",newline="") as f: return list(csv.DictReader(f))
12
13 def _maxdiff(a,b)->float: return float(np.max(np.abs(np.asarray(a,dtype=float)-np.asarray(b,

```

```

dtype=float))))
14
15 def validate_q1()->dict:
16     folder=ROOT/"outputs"/"q1"; rows=_csv(folder/"solution.csv")
17     wb=openpyxl.load_workbook(folder/"result1.xlsx",data_only=True,read_only=True)
18     template=openpyxl.load_workbook(DATA/"result1.xlsx",data_only=True,read_only=True)
19     grid=np.asarray([float(r["grid_kwh"]) for r in rows])
20     xgrid=np.asarray(list(wb.worksheets[0].iter_rows(min_row=2,max_row=145,min_col=2,max_col
        =2,values_only=True)),dtype=float).ravel()
21     result={"rows":len(rows),"sheet_names_preserved":wb.sheetnames==template.sheetnames,
22             "max_grid_diff":_maxdiff(grid,xgrid)}
23     result["passed"]=bool(len(rows)==144 and result["sheet_names_preserved"] and result["
        max_grid_diff"]<1e-7)
24     return result
25
26 def validate_annual(folder:Path,xlsx_name:str,kind:str)->dict:
27     rows=_csv(folder/"solution.csv"); daily=_csv(folder/"daily_metrics.csv")
28     if len(rows)!=334*144 or len(daily)!=334: raise ValueError(f"{folder}: row count
        mismatch")
29     wb=openpyxl.load_workbook(folder/xlsx_name,data_only=True,read_only=True)
30     template=openpyxl.load_workbook(DATA/xlsx_name,data_only=True,read_only=True)
31     has_adjust=kind=="q3"; storage_index=2 if has_adjust else 1; emergency_index=3 if
        has_adjust else 2
32     plan_field="baseline_grid_kwh" if has_adjust else "grid_plan_kwh"
33     plan=np.asarray([float(r[plan_field]) for r in rows]).reshape(334,144)
34     charge=np.asarray([float(r["charge_actual_kwh"]) for r in rows]).reshape(334,144)
35     discharge=np.asarray([float(r["discharge_actual_kwh"]) for r in rows]).reshape(334,144)
36     emergency=np.asarray([float(r["emergency_kwh"]) for r in rows]).reshape(334,144)
37     plan_rows=list(wb.worksheets[0].iter_rows(min_row=2,max_row=335,min_col=2,max_col=147,
        values_only=True))
38     xplan=np.asarray([r[:144] for r in plan_rows],dtype=float)
39     xttotal=np.asarray([r[144] for r in plan_rows],dtype=float)
40     xcost=np.asarray([r[145] for r in plan_rows],dtype=float)
41     expected_plan_cost=np.asarray([float(r["baseline_cost"] if has_adjust else r["plan_cost"
        ]) for r in daily])
42     result={"solution_rows":len(rows),"daily_rows":len(daily),"sheet_names_preserved":wb.
        sheetnames==template.sheetnames,
43             "max_plan_diff":_maxdiff(xplan,plan),"max_plan_total_diff":_maxdiff(xttotal,plan.
        sum(axis=1)),
44             "max_plan_cost_diff":_maxdiff(xcost,expected_plan_cost)}
45     if has_adjust:
46         adjusted=np.asarray([float(r["adjusted_grid_kwh"]) for r in rows]).reshape(334,144)
47         adjust_rows=list(wb.worksheets[1].iter_rows(min_row=2,max_row=335,min_col=2,max_col
            =147,values_only=True))
48         xadjust=np.asarray([r[:144] for r in adjust_rows],dtype=float)
49         xa_total=np.asarray([r[144] for r in adjust_rows],dtype=float)
50         xa_cost=np.asarray([r[145] for r in adjust_rows],dtype=float)

```

```

51     expected_cost=np.asarray([float(r["settlement_cost"]) for r in daily])
52     result.update({"max_adjust_diff":_maxdiff(xadjust,adjusted),
53                  "max_adjust_total_diff":_maxdiff(xa_total,adjusted.sum(axis=1)),
54                  "max_adjust_cost_diff":_maxdiff(xa_cost,expected_cost)})
55     storage_rows=list(wb.worksheets[storage_index].iter_rows(min_row=2,max_row=2005,min_col
56                     =1,max_col=6,values_only=True))
57     xcharge=np.asarray([float(r[2]) for r in storage_rows]).reshape(334,6)
58     xdischarge=np.asarray([float(r[3]) for r in storage_rows]).reshape(334,6)
59     xs0=np.asarray([float(storage_rows[d*6][5]) for d in range(334)])
60     xs1=np.asarray([float(storage_rows[d*6+1][5]) for d in range(334)])
61     result.update({"storage_rows":len(storage_rows),
62                  "max_storage_charge_diff":_maxdiff(xcharge,charge.reshape(334,6,24).sum(axis=2)),
63                  "max_storage_discharge_diff":_maxdiff(xdischarge,discharge.reshape(334,6,24).sum(
64                      axis=2)),
65                  "max_storage_initial_diff":_maxdiff(xs0,[float(r["soc_initial"]) for r in daily]),
66                  "max_storage_terminal_diff":_maxdiff(xs1,[float(r["soc_terminal"]) for r in daily])
67                      })
68     erows=list(wb.worksheets[emergency_index].iter_rows(min_row=2,min_col=1,max_col=3,
69               values_only=True))
70     x_emergency=sum(float(r[2] or 0) for r in erows)
71     result.update({"emergency_rows":len(erows),"emergency_sheet_total":x_emergency,
72                  "emergency_csv_total":float(emergency.sum()),
73                  "emergency_total_diff":abs(x_emergency-float(emergency.sum()))})
74     numerical=[v for k,v in result.items() if k.startswith("max_") or k=="
75                emergency_total_diff"]
76     result["passed"]=bool(result["sheet_names_preserved"] and len(storage_rows)==2004 and
77                           max(numerical)<1e-7)
78     return result
79
80 def validate_all()->dict:
81     cases={
82         "q1":validate_q1(),
83         "q2":validate_annual(ROOT/"outputs"/"q2","result2.xlsx","q2"),
84         "q3_main":validate_annual(ROOT/"outputs"/"q3","result3.xlsx","q3"),
85         "q3_raw_step":validate_annual(ROOT/"outputs"/"q3"/"variants"/"raw_step","result3.
86             xlsx","q3"),
87         "q3_calibrated_step":validate_annual(ROOT/"outputs"/"q3"/"variants"/"calibrated_step
88             ", "result3.xlsx","q3"),
89         "q4_2":validate_annual(ROOT/"outputs"/"q4-2","result4-2.xlsx","q2"),
90         "q4_3":validate_annual(ROOT/"outputs"/"q4-3","result4-3.xlsx","q3"),
91     }
92     result={"cases":cases,"passed":all(v["passed"] for v in cases.values())}
93     (ROOT/"outputs"/"verification_summary.json").write_text(json.dumps(result,ensure_ascii=
94         False,indent=2),encoding="utf-8")
95     return result
96
97 if __name__=="__main__": print(json.dumps(validate_all(),ensure_ascii=False,indent=2))

```

B.18 复现清单生成 (build_manifest.py)

Listing 18:

```
1 from __future__ import annotations
2 import hashlib, json
3 from datetime import datetime
4 from pathlib import Path
5
6 ROOT=Path(__file__).resolve().parents[1]
7 OUTPUTS=ROOT/"outputs"
8 VARIANTS={
9     "q1":{"path":"q1", "parameters":{"question":1}},
10    "q2_fixed_online":{"path":"q2", "parameters":{"question":2, "price":"attachment1", "
11        forecast":"strictly_online"}},
12    "q3_main_linear_calibrated":{"path":"q3", "parameters":{"question":3, "downscale":"linear"
13        , "calibration":True, "price":"attachment1"}},
14    "q3_raw_step":{"path":"q3/variants/raw_step", "parameters":{"question":3, "downscale":"
15        step", "calibration":False, "price":"attachment1"}},
16    "q3_calibrated_step":{"path":"q3/variants/calibrated_step", "parameters":{"question":3, "
17        downscale":"step", "calibration":True, "price":"attachment1"}},
18    "q4_2_dynamic":{"path":"q4-2", "parameters":{"question":"4-2", "price":"attachment4", "
19        forecast":"strictly_online"}},
20    "q4_3_dynamic":{"path":"q4-3", "parameters":{"question":"4-3", "price":"attachment4", "
21        downscale":"linear", "calibration":True}},
22 }
23
24 def digest(path:Path)->str:
25     h=hashlib.sha256()
26     with path.open("rb") as f:
27         for block in iter(lambda:f.read(1024*1024),b""): h.update(block)
28     return h.hexdigest()
29
30 def build()->dict:
31     result={"generated_at":datetime.now().astimezone().isoformat(), "authority_root":str(ROOT
32         ),
33         "main":{"q1":"q1", "q2":"q2_fixed_online", "q3":"q3_main_linear_calibrated",
34             "q4_2":"q4_2_dynamic", "q4_3":"q4_3_dynamic"}, "variants":{}, "source_files
35             ":{}}
36     for name, spec in VARIANTS.items():
37         folder=OUTPUTS/spec["path"]; files={}
38         for path in sorted(folder.glob("*")):
39             if path.is_file():
40                 files[path.name]={"bytes":path.stat().st_size, "sha256":digest(path)}
```

```

33     metrics_path=folder/"metrics.json"
34     metrics=json.loads(metrics_path.read_text(encoding="utf-8")) if metrics_path.exists
    () else None
35     result["variants"][name]={ "relative_path":spec["path"], "parameters":spec["parameters
    "],
36                                     "metrics":metrics, "files":files}
37     for path in sorted((ROOT/"solve").glob("*.py")):
38         result["source_files"][str(path.relative_to(ROOT))]={ "bytes":path.stat().st_size, "
    sha256":digest(path)}
39     for pattern in ("reports/*.md", "tests/*.py", "README.md", "requirements.txt", "run_all.ps1"
    ):
40         for path in sorted(ROOT.glob(pattern)):
41             if path.is_file(): result["source_files"][str(path.relative_to(ROOT))]={ "bytes":
    path.stat().st_size, "sha256":digest(path)}
42     result["figures"]={}
43     for path in sorted((ROOT/"figures").rglob("*.pdf")):
44         result["figures"][str(path.relative_to(ROOT))]={ "bytes":path.stat().st_size, "sha256"
    :digest(path)}
45     result["governance_outputs"]={}
46     for name in ("verification_summary.json", "q4_comparison.json"):
47         path=OUTPUTS/name
48         if path.exists(): result["governance_outputs"][name]={ "bytes":path.stat().st_size, "
    sha256":digest(path)}
49     (OUTPUTS/"MANIFEST.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),
    encoding="utf-8")
50     return result
51
52 if __name__=="__main__":
53     manifest=build(); print(json.dumps({"variants":list(manifest["variants"]),
54         "source_files":len(manifest["source_files"]), "manifest":str(OUTPUTS/"MANIFEST.json")
    },ensure_ascii=False,indent=2))

```

B.19 科研绘图公共样式 (plot_style.py)

Listing 19:

```

1  """论文配图统一样式。
2
3  中文用 SimSun（宋体）、西文与数字用 Times New Roman，保证 LaTeX 与 Word 端均无方框；
4  统一调色板、字号、线宽与导出 DPI，所有图共用同一套视觉语言。
5  """
6  from __future__ import annotations
7
8  from pathlib import Path
9
10 import matplotlib

```

```

11
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14
15 FIGURES = Path(__file__).resolve().parents[1] / "paper" / "figures"
16 FIGURES.mkdir(parents=True, exist_ok=True)
17
18 # 主色板：低饱和、印刷友好、色盲可区分
19 PALETTE = {
20     "navy": "#1F3B73",
21     "blue": "#3E6FB0",
22     "sky": "#7FB2E5",
23     "teal": "#2E8B84",
24     "green": "#5FA05A",
25     "olive": "#9BAF3B",
26     "gold": "#D8A22B",
27     "orange": "#E07B39",
28     "red": "#C0392B",
29     "rose": "#D46A8C",
30     "purple": "#7A5BA6",
31     "gray": "#7F8C99",
32     "lightgray": "#D5DBE1",
33     "ink": "#1A2028",
34 }
35 CYCLE = [PALETTE[k] for k in ("navy", "orange", "teal", "red", "purple", "gold", "sky", "
    green")]
36
37 # 四问/四场景的固定语义色，全文一致
38 SCENARIO_COLORS = {
39     "Q1": PALETTE["navy"],
40     "Q2": PALETTE["orange"],
41     "Q3": PALETTE["teal"],
42     "Q4-2": PALETTE["red"],
43     "Q4-3": PALETTE["purple"],
44     "legacy": PALETTE["gray"],
45 }
46
47
48 def apply_style() -> None:
49     plt.rcParams.update(
50         {
51             "font.family": "sans-serif",
52             "font.sans-serif": ["SimSun", "Times New Roman", "DejaVu Sans"],
53             "font.serif": ["SimSun", "Times New Roman"],
54             "axes.unicode_minus": False,
55             "font.size": 11,
56             "axes.titlesize": 12.5,

```



```

57         "axes.labelsize": 11.5,
58         "axes.titleweight": "bold",
59         "legend.fontsize": 10,
60         "xtick.labelsize": 10,
61         "ytick.labelsize": 10,
62         "axes.linewidth": 0.9,
63         "axes.edgecolor": PALETTE["ink"],
64         "axes.labelcolor": PALETTE["ink"],
65         "text.color": PALETTE["ink"],
66         "xtick.color": PALETTE["ink"],
67         "ytick.color": PALETTE["ink"],
68         "axes.grid": True,
69         "grid.color": PALETTE["lightgray"],
70         "grid.linewidth": 0.6,
71         "grid.alpha": 0.75,
72         "axes.axisbelow": True,
73         "figure.dpi": 130,
74         "savefig.dpi": 330,
75         "savefig.bbox": "tight",
76         "savefig.facecolor": "white",
77         "figure.facecolor": "white",
78         "axes.spines.top": False,
79         "axes.spines.right": False,
80         "legend.frameon": False,
81         "lines.linewidth": 1.8,
82         "lines.markersize": 4.5,
83     }
84 )
85
86
87 def panel_label(ax, text: str, dx: float = -0.085, dy: float = 1.06) -> None:
88     """在子图左上角标注 (a)/(b)/(c)/(d)。"""
89     ax.text(dx, dy, text, transform=ax.transAxes, fontsize=12.5, fontweight="bold",
90           va="bottom", ha="left", color=PALETTE["ink"])
91
92
93 def save(fig, name: str, *, pdf: bool = True) -> Path:
94     """保存 PNG（论文插图）与可选 PDF（矢量备份），返回 PNG 路径。"""
95     png = FIGURES / f"{name}.png"
96     fig.savefig(png)
97     if pdf:
98         fig.savefig(FIGURES / f"{name}.pdf")
99     plt.close(fig)
100     return png
101
102
103 def wan(value: float) -> float:

```

```

104     """元 → 万元。"""
105     return value / 1e4

```

B.20 数据总览绘图 (fig_data.py)

Listing 20:

```

1  """数据层配图：附件结构总览、净负荷与光伏渗透、预报误差诊断、相关性。
2
3  所有图都只依赖附件原始数据，可在求解前独立复现。
4  """
5  from __future__ import annotations
6
7  import sys
8  from pathlib import Path
9
10 import numpy as np
11
12 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
13
14 from MathModel.solve.io_data import ( # noqa: E402
15     DEFAULT_DATA_ROOT,
16     DT_HOURS,
17     hourly_to_ten_min,
18     read_attachment1,
19     read_attachment2,
20     read_attachment3,
21     read_attachment4,
22 )
23 from MathModel.solve.plot_style import PALETTE, apply_style, panel_label, save # noqa: E402
24
25 apply_style()
26 import matplotlib.dates as mdates # noqa: E402
27 import matplotlib.pyplot as plt # noqa: E402
28 import seaborn as sns # noqa: E402
29 from matplotlib.colors import TwoSlopeNorm # noqa: E402
30
31 MONTH_EDGES = np.cumsum([0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31])
32 MONTH_LABELS = [f"{m}月" for m in range(1, 13)]
33 DAYS_PER_MONTH = np.diff(MONTH_EDGES)
34
35
36 def fig_data_overview(root: Path = DEFAULT_DATA_ROOT) -> None:
37     """(a) 典型日曲线 (b) 全年负载热力图 (c) 全年光伏热力图 (d) 全年电价热力图。"""
38     day1 = read_attachment1(root)
39     annual = read_attachment2(root)

```

```

40 price = read_attachment4(root)
41
42 fig = plt.figure(figsize=(13.6, 8.2))
43 gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.22)
44
45 ax = fig.add_subplot(gs[0, 0])
46 hours = (np.arange(144) + 1) / 6.0
47 ax.plot(hours, day1.load_kw / 1e3, color=PALETTE["navy"], label="小区负载")
48 ax.plot(hours, day1.pv_kw / 1e3, color=PALETTE["gold"], label="光伏发电预测")
49 ax.set_xlabel("时刻 / h")
50 ax.set_ylabel("功率 / MW")
51 ax.set_title("典型日负载与光伏出力的错配")
52 ax.set_xlim(0, 24)
53 ax.set_xticks(range(0, 25, 4))
54 ax2 = ax.twinx()
55 ax2.plot(hours, day1.price, color=PALETTE["red"], linestyle="--", linewidth=1.5, label="
    外网电价")
56 ax2.set_ylabel("电价 / (元·kWh$^{-1}$)", color=PALETTE["red"])
57 ax2.tick_params(axis="y", colors=PALETTE["red"])
58 ax2.grid(False)
59 ax2.spines["right"].set_visible(True)
60 handles1, labels1 = ax.get_legend_handles_labels()
61 handles2, labels2 = ax2.get_legend_handles_labels()
62 ax.legend(handles1 + handles2, labels1 + labels2, loc="upper left", ncol=1, fontsize=9)
63 panel_label(ax, "(a)")
64
65 for idx, (data, title, cmap, unit) in enumerate(
66     [
67         (annual.load_kw, "全年负载分布", "YlGnBu", "kW"),
68         (annual.pv_kw, "全年光伏出力分布", "YlOrBr", "kW"),
69         (price.price, "全年实时电价分布", "RdYlBu_r", "元/kWh"),
70     ]
71 ):
72     ax = fig.add_subplot(gs[(1 + idx) // 2, (1 + idx) % 2])
73     im = ax.imshow(
74         data / (1e3 if unit == "kW" else 1),
75         aspect="auto",
76         cmap=cmap,
77         interpolation="nearest",
78         extent=[0, 24, 365, 1],
79     )
80     ax.set_xlabel("时刻 / h")
81     ax.set_ylabel("日期 (第 n 天)")
82     ax.set_title(title)
83     ax.set_xticks(range(0, 25, 4))
84     ax.grid(False)
85     cb = fig.colorbar(im, ax=ax, pad=0.02)

```

```

86         cb.set_label("MW" if unit == "kW" else "元/kWh", fontsize=9)
87         cb.ax.tick_params(labelsize=8.5)
88         panel_label(ax, "%s" % "bcd"[idx])
89
90     save(fig, "data_overview")
91
92
93 def fig_netload_season(root: Path = DEFAULT_DATA_ROOT) -> None:
94     """(a) 净负荷月度箱线图 (b) 光伏渗透率月度分布 (c) 光伏富余时段占比。"""
95     annual = read_attachment2(root)
96     net = annual.load_kw - annual.pv_kw
97     month_of_day = np.repeat(np.arange(12), DAYS_PER_MONTH)
98
99     fig = plt.figure(figsize=(13.6, 4.4))
100     gs = fig.add_gridspec(1, 3, wspace=0.28)
101
102     ax = fig.add_subplot(gs[0, 0])
103     data = [net[month_of_day == m].ravel() / 1e3 for m in range(12)]
104     bp = ax.boxplot(data, patch_artist=True, widths=0.62, showfliers=False,
105                     medianprops=dict(color=PALETTE["ink"], linewidth=1.3),
106                     whiskerprops=dict(color=PALETTE["gray"]),
107                     capprops=dict(color=PALETTE["gray"]),
108                     boxprops=dict(facecolor=PALETTE["sky"], edgecolor=PALETTE["navy"],
109                                   linewidth=0.9))
109     ax.axhline(0, color=PALETTE["red"], linewidth=1.1, linestyle="--")
110     ax.set_xticks(range(1, 13))
111     ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
112     ax.set_xlabel("月份")
113     ax.set_ylabel("净负荷 / MW")
114     ax.set_title("净负荷的月度分布")
115     panel_label(ax, "(a)")
116
117     ax = fig.add_subplot(gs[0, 1])
118     penetration = annual.pv_kw / np.maximum(annual.load_kw, 1e-9)
119     data = [np.clip(penetration[month_of_day == m].ravel(), 0, 3) for m in range(12)]
120     parts = ax.violinplot(data, showmedians=True, widths=0.85)
121     for body in parts["bodies"]:
122         body.set_facecolor(PALETTE["gold"])
123         body.set_edgecolor(PALETTE["orange"])
124         body.set_alpha(0.75)
125     for key in ("cmidians", "cbars", "cmins", "cmaxes"):
126         parts[key].set_color(PALETTE["ink"])
127         parts[key].set_linewidth(1.1)
128     ax.axhline(1.0, color=PALETTE["red"], linestyle="--", linewidth=1.1)
129     ax.set_xticks(range(1, 13))
130     ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
131     ax.set_xlabel("月份")

```

```

132 ax.set_ylabel("光伏/负载 比值")
133 ax.set_title("光伏渗透率的月度分布")
134 ax.set_ylim(0, 3)
135 panel_label(ax, "(b)")
136
137 ax = fig.add_subplot(gs[0, 2])
138 surplus_slots = (annual.pv_kw - annual.load_kw > 0).sum(axis=1) # 每天富余时段数
139 big_surplus = (annual.pv_kw - annual.load_kw > 5000).sum(axis=1)
140 hours_surplus = np.array([surplus_slots[month_of_day == m].mean() * 10 / 60 for m in
    range(12)])
141 hours_big = np.array([big_surplus[month_of_day == m].mean() * 10 / 60 for m in range(12)
    ])
142 x = np.arange(12)
143 ax.bar(x - 0.2, hours_surplus, width=0.4, color=PALETTE["green"], label="光伏富余时段")
144 ax.bar(x + 0.2, hours_big, width=0.4, color=PALETTE["red"], label="富余超 5000 kW 时段")
145 ax.set_xticks(x)
146 ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
147 ax.set_xlabel("月份")
148 ax.set_ylabel("日均时长 / h")
149 ax.set_title("光伏富余的月度强度")
150 ax.legend(fontsize=9)
151 panel_label(ax, "(c)")
152
153 save(fig, "netload_season")
154
155
156 def fig_forecast_diagnosis(root: Path = DEFAULT_DATA_ROOT) -> None:
157     """附件 3 官方预报的误差诊断: (a) 分发布时刻误差雨云 (b) 月度 MAE/偏度 (c) 降尺度方法
    对比。"""
158     annual = read_attachment2(root)
159     forecasts = read_attachment3(root)
160     month_of_day = np.repeat(np.arange(12), DAYS_PER_MONTH)
161
162     err = np.zeros_like(forecasts.hourly_kw)
163     for node in range(4):
164         for hour in range(24):
165             target_slot = (node * 36 + (hour + 1) * 6 - 1) % 144
166             target_day = np.arange(365) + (node * 36 + (hour + 1) * 6 - 1) // 144
167             valid = target_day < 365
168             err[valid, node, hour] = forecasts.hourly_kw[valid, node, hour] - annual.pv_kw[
                target_day[valid], target_slot]
169
170     fig = plt.figure(figsize=(13.6, 4.6))
171     gs = fig.add_gridspec(1, 3, wspace=0.28)
172
173     ax = fig.add_subplot(gs[0, 0])
174     labels = ["0:00", "6:00", "12:00", "18:00"]

```

```

175 colors = [PALETTE["navy"], PALETTE["orange"], PALETTE["teal"], PALETTE["purple"]]
176 for n, (label, color) in enumerate(zip(labels, colors)):
177     values = np.sort(np.abs(err[:, n, :].ravel()))
178     values = np.maximum(values, 1e-3)
179     share = np.arange(1, values.size + 1) / values.size
180     step = max(1, values.size // 4000)
181     ax.plot(values[:, :step], share[:, :step], color=color, linewidth=2.1, label=label)
182     p95 = np.percentile(values, 95)
183     ax.plot([p95], [0.95], marker="o", color=color, markersize=6,
184             markeredgecolor="white", markeredgewidth=1.0, zorder=5)
185     ax.axhline(0.95, color=PALETTE["gray"], linestyle=":", linewidth=1.0)
186     ax.text(1.6, 0.955, "$P_{95}$", fontsize=9, color=PALETTE["gray"])
187     ax.set_xscale("log")
188     ax.set_xlim(1, 6000)
189     ax.set_ylim(0, 1.0)
190     ax.set_xlabel("绝对预报误差 / kW (对数轴)")
191     ax.set_ylabel("经验累积分布 ECDF")
192     ax.set_title("分发布时刻的预报误差累积分布")
193     ax.legend(fontsize=9, loc="lower right", title="发布时刻", title_fontsize=9)
194     panel_label(ax, "(a)")
195
196 ax = fig.add_subplot(gs[0, 1])
197 mae = [np.mean(np.abs(err[month_of_day == m])) for m in range(12)]
198 bias = [np.mean(err[month_of_day == m]) for m in range(12)]
199 x = np.arange(12)
200 ax.bar(x, mae, color=PALETTE["sky"], edgecolor=PALETTE["navy"], linewidth=0.7, label="
    MAE")
201 ax.plot(x, bias, color=PALETTE["red"], marker="o", markersize=4.5, label="平均偏差")
202 ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
203 ax.set_xticks(x)
204 ax.set_xticklabels([f"{m}" for m in range(1, 13)], fontsize=9)
205 ax.set_xlabel("月份")
206 ax.set_ylabel("误差 / kW")
207 ax.set_title("预报误差的月度演变")
208 ax.legend(fontsize=9)
209 panel_label(ax, "(b)")
210
211 ax = fig.add_subplot(gs[0, 2])
212 # 第 n 个发布时刻的预报降尺度后, 索引 0..35 对应当天时段 [36n, 36n+36),
213 # 这正是该节点后要执行的 6 h 块, 因此必须比较 forecast[0:36] 与该块的真实光伏。
214 raw_step, raw_linear = [], []
215 for d in range(365):
216     for n in range(4):
217         lo = n * 36
218         hi = lo + 36
219         actual = annual.pv_kw[d, lo:hi]
220         raw_step.append(np.mean(np.abs(hourly_to_ten_min(forecasts.hourly_kw[d, n], "

```

```

        step"))[:36] - actual)))
221     raw_linear.append(np.mean(np.abs(hourly_to_ten_min(forecasts.hourly_kw[d, n], "
        linear"))[:36] - actual)))
222     mae_step, mae_linear = float(np.mean(raw_step)), float(np.mean(raw_linear))
223     bars = ax.bar(["阶梯保持", "线性插值"], [mae_step, mae_linear],
224                  color=[PALETTE["gray"], PALETTE["teal"]], width=0.5,
225                  edgecolor=PALETTE["ink"], linewidth=0.8)
226     for bar, value in zip(bars, [mae_step, mae_linear]):
227         ax.text(bar.get_x() + bar.get_width() / 2, value * 1.02, f"{value:.1f} kW",
228                ha="center", va="bottom", fontsize=10, fontweight="bold")
229     ax.set_ylabel("执行块 MAE / kW")
230     ax.set_title("小时预报降尺度方法的精度对比")
231     ax.set_ylim(0, max(mae_step, mae_linear) * 1.25)
232     panel_label(ax, "(c)")
233
234     save(fig, "forecast_diagnosis")
235     print(f"[fig] 降尺度执行块 MAE: 阶梯={mae_step:.4f} kW, 线性={mae_linear:.4f} kW")
236
237
238 def fig_correlation(root: Path = DEFAULT_DATA_ROOT) -> None:
239     """(a) 相关系数热力图 (b) 负载—净负荷联合分布。"""
240     annual = read_attachment2(root)
241     price = read_attachment4(root)
242     load = annual.load_kw.ravel()
243     pv = annual.pv_kw.ravel()
244     net = load - pv
245     p = price.price.ravel()
246     slot_phase = np.tile(np.arange(144) / 144.0, 365)
247
248     frame = {
249         "小区负载": load, "光伏出力": pv, "净负荷": net,
250         "实时电价": p, "日内相位": slot_phase,
251     }
252     keys = list(frame)
253     matrix = np.corrcoef(np.stack([frame[k] for k in keys]))
254     matrix = matrix / np.sqrt(np.outer(np.diag(matrix), np.diag(matrix)))
255
256     fig = plt.figure(figsize=(13.0, 4.8))
257     gs = fig.add_gridspec(1, 2, width_ratios=[1.0, 1.15], wspace=0.3)
258
259     ax = fig.add_subplot(gs[0, 0])
260     im = ax.imshow(matrix, cmap="RdBu_r", vmin=-1, vmax=1)
261     ax.set_xticks(range(len(keys)))
262     ax.set_xticklabels(keys, rotation=32, ha="right", fontsize=9)
263     ax.set_yticks(range(len(keys)))
264     ax.set_yticklabels(keys, fontsize=9)
265     for i in range(len(keys)):

```

```

266         for j in range(len(keys)):
267             ax.text(j, i, f"{matrix[i, j]:.2f}", ha="center", va="center", fontsize=9.5,
268                     color="white" if abs(matrix[i, j]) > 0.55 else PALETTE["ink"])
269     ax.grid(False)
270     ax.set_title("关键变量的相关系数矩阵")
271     cb = fig.colorbar(im, ax=ax, pad=0.02, shrink=0.9)
272     cb.set_label("Pearson 相关系数", fontsize=9)
273     panel_label(ax, "(a)")
274
275     ax = fig.add_subplot(gs[0, 1])
276     sample = np.random.default_rng(0).choice(load.size, size=min(50000, load.size), replace=
277         False)
278     hb = ax.hexbin(load[sample] / 1e3, net[sample] / 1e3, gridsize=48, cmap="viridis",
279                   bins="log", mincnt=1, linewidths=0)
280     ax.axhline(0, color=PALETTE["red"], linestyle="--", linewidth=1.2)
281     ax.set_xlabel("小区负载 / MW")
282     ax.set_ylabel("净负荷 / MW")
283     ax.set_title("负载—净负荷联合分布（含零线）")
284     cb = fig.colorbar(hb, ax=ax, pad=0.02, shrink=0.9)
285     cb.set_label("时段数（对数）", fontsize=9)
286     panel_label(ax, "(b)")
287
288     save(fig, "correlation_structure")
289
290 def main() -> None:
291     root = DEFAULT_DATA_ROOT
292     fig_data_overview(root)
293     fig_netload_season(root)
294     fig_forecast_diagnosis(root)
295     fig_correlation(root)
296     print("[fig] 数据层配图完成 →", (Path(__file__).resolve().parents[1] / "paper" / "
297         figures"))
298
299 if __name__ == "__main__":
300     main()

```

B.21 问题一绘图（fig_q1.py）

Listing 21:

```

1  """问题一调度结果可视化（含无储能基准对照）。"""
2  from __future__ import annotations
3
4  import sys

```



```

5 from pathlib import Path
6
7 import numpy as np
8 import pandas as pd
9
10 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
11
12 from MathModel.solve.io_data import DT_HOURS, read_attachment1 # noqa: E402
13 from MathModel.solve.plot_style import PALETTE, apply_style, panel_label, save # noqa: E402
14
15 apply_style()
16 import matplotlib.pyplot as plt # noqa: E402
17
18 ROOT = Path(__file__).resolve().parents[1]
19
20
21 def main() -> None:
22     frame = pd.read_csv(ROOT / "outputs" / "q1" / "solution.csv", encoding="utf-8-sig")
23     day1 = read_attachment1()
24     hours = frame["slot"].to_numpy() / 6.0
25
26     fig = plt.figure(figsize=(13.6, 7.6))
27     gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.24)
28
29     ax = fig.add_subplot(gs[0, 0])
30     ax.plot(hours, day1.load_kw / 1e3, color=PALETTE["navy"], linewidth=1.9, label="小区负载")
31     ax.plot(hours, day1.pv_kw / 1e3, color=PALETTE["gold"], linewidth=1.9, label="光伏预测出力")
32     ax.fill_between(hours, day1.pv_kw / 1e3, day1.load_kw / 1e3,
33                     where=(day1.pv_kw < day1.load_kw), color=PALETTE["sky"], alpha=0.35,
34                     label="净负荷缺口")
35     ax.fill_between(hours, day1.pv_kw / 1e3, day1.load_kw / 1e3,
36                     where=(day1.pv_kw >= day1.load_kw), color=PALETTE["green"], alpha=0.32,
37                     label="光伏富余")
38     ax.set_xlabel("时刻 / h")
39     ax.set_ylabel("功率 / MW")
40     ax.set_title("典型日净负荷结构")
41     ax.set_xlim(0, 24)
42     ax.legend(fontsize=8.5, ncol=2)
43     panel_label(ax, "(a)")
44
45     ax = fig.add_subplot(gs[0, 1])
46     no_storage = np.maximum((day1.load_kw - day1.pv_kw) * DT_HOURS, 0)
47     ax.plot(hours, frame["grid_kwh"] * 6 / 1e3, color=PALETTE["navy"], linewidth=1.9, label="含储能最优购电")
48     ax.plot(hours, no_storage * 6 / 1e3, color=PALETTE["gray"], linewidth=1.6, linestyle="--")

```

```

    ", label="无储能基准购电")
47 ax.fill_between(hours, frame["grid_kwh"] * 6 / 1e3, no_storage * 6 / 1e3,
48                  where=(no_storage > frame["grid_kwh"]), color=PALETTE["teal"], alpha
                    =0.25, label="储能削减的购电")
49 ax.set_xlabel("时刻 / h")
50 ax.set_ylabel("购电功率 / MW")
51 ax.set_title("储能对购电曲线的削峰填谷")
52 ax.set_xlim(0, 24)
53 ax.legend(fontsize=8.5)
54 panel_label(ax, "(b)")
55
56 ax = fig.add_subplot(gs[1, 0])
57 ax.bar(hours, frame["charge_kwh"], width=0.13, color=PALETTE["purple"],
58        edgecolor=PALETTE["ink"], linewidth=0.5, label="充电量")
59 ax.bar(hours, -frame["discharge_kwh"], width=0.13, color=PALETTE["teal"],
60        edgecolor=PALETTE["ink"], linewidth=0.5, label="放电量")
61 ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
62 ax.axhline(833.33, color=PALETTE["red"], linestyle=":", linewidth=1.0)
63 ax.axhline(-833.33, color=PALETTE["red"], linestyle=":", linewidth=1.0)
64 ax.text(0.2, 860, "单时段功率上限 833.33 kWh", fontsize=8, color=PALETTE["red"])
65 ax.set_xlabel("时刻 / h")
66 ax.set_ylabel("电量 / kWh (10 min)")
67 ax.set_title("储能充放电计划")
68 ax.set_xlim(0, 24)
69 ax.legend(fontsize=9)
70 panel_label(ax, "(c)")
71
72 ax = fig.add_subplot(gs[1, 1])
73 ax.plot(hours, frame["soc_kwh"] / 1e3, color=PALETTE["navy"], linewidth=2.2)
74 ax.fill_between(hours, frame["soc_kwh"] / 1e3, 1.2, color=PALETTE["sky"], alpha=0.3)
75 ax.axhline(1.2, color=PALETTE["red"], linestyle="--", linewidth=1.1)
76 ax.axhline(10.8, color=PALETTE["red"], linestyle="--", linewidth=1.1)
77 ax.axhline(6.0, color=PALETTE["gray"], linestyle=":", linewidth=1.0)
78 ax.text(0.2, 1.35, "下限 1200 kWh", fontsize=8.5, color=PALETTE["red"])
79 ax.text(0.2, 10.95, "上限 10800 kWh", fontsize=8.5, color=PALETTE["red"])
80 ax.text(0.2, 6.15, "首末储电量 6000 kWh", fontsize=8.5, color=PALETTE["gray"])
81 ax.set_xlabel("时刻 / h")
82 ax.set_ylabel("储电量 / MWh")
83 ax.set_title("储电量日内轨迹 (首末闭合)")
84 ax.set_xlim(0, 24)
85 ax.set_ylim(0.5, 12.2)
86 panel_label(ax, "(d)")
87
88 save(fig, "q1_dispatch")
89 print("[fig] q1_dispatch 完成")
90
91

```

```

92 if __name__ == "__main__":
93     main()

```

B.22 结果与消融绘图 (fig_results.py)

Listing 22:

```

1  """结果层配图：原因拆解、预报价值、价格信息价值、结算口径、灵敏度、调度全景。"""
2  from __future__ import annotations
3
4  import json
5  import sys
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11 sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
12
13 from MathModel.solve.plot_style import ( # noqa: E402
14     CYCLE,
15     PALETTE,
16     SCENARIO_COLORS,
17     apply_style,
18     panel_label,
19     save,
20 )
21
22 apply_style()
23 import matplotlib.pyplot as plt # noqa: E402
24
25 ROOT = Path(__file__).resolve().parents[1]
26 VARIANTS = ROOT / "outputs" / "variants" / "model_audit_v4"
27 FIGS = ROOT / "paper" / "figures"
28
29 Q2_ORDER = ["A_perfectload_risk_feedback", "B_causalload_risk_feedback",
30             "C_causalload_point_feedback", "D_causalload_risk_nofeedback"]
31 Q2_LABEL = {
32     "A_perfectload_risk_feedback": "A 完美负载\n风险边际•有反馈",
33     "B_causalload_risk_feedback": "B 因果负载\n风险边际•有反馈",
34     "C_causalload_point_feedback": "C 因果负载\n点预测•有反馈",
35     "D_causalload_risk_nofeedback": "D 因果负载\n风险边际•无反馈",
36 }
37 Q3_ORDER = ["S0_0", "S1_0_6", "S2_0_6_12", "S3_0_6_12_18"]
38 Q3_LABEL = {"S0_0": "S0\n仅0:00", "S1_0_6": "S1\n+6:00", "S2_0_6_12": "S2\n+12:00", "
39             S3_0_6_12_18": "S3\n+18:00"}

```

```

39 Q4_ORDER = ["q42_perfect_price", "q42_causal_price", "q43_perfect_price", "q43_causal_price"
40 ]
41 Q4_LABEL = {"q42_perfect_price": "Q4-2\n附件4直用", "q42_causal_price": "Q4-2\n因果扩展",
42             "q43_perfect_price": "Q4-3\n附件4直用", "q43_causal_price": "Q4-3\n因果扩展"}
43
44 def load(path: Path) -> dict:
45     return json.loads(path.read_text(encoding="utf-8"))
46
47
48 def load_group(*parts: str) -> dict:
49     """读取 ablation 写出的分组汇总文件；最后一段是文件名主干（不含 .json）。
50
51     load_group("q2", "summary")          -> outputs/variants/.../q2/summary.json
52     load_group("q3_additive", "summary")  -> outputs/variants/.../q3/summary_additive.json
53     """
54     *dirs, stem = parts
55     return load(VARIANTS.joinpath(*dirs) / f"{stem}.json")
56
57
58
59
60
61 def _bar_labels(ax, bars, values, fmt="{:.2f}", dy=0.012, fontsize=9.5):
62     span = ax.get_ylim()[1] - ax.get_ylim()[0]
63     for bar, value in zip(bars, values):
64         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + span * dy,
65                 fmt.format(value), ha="center", va="bottom", fontsize=fontsize, fontweight="
66                     bold")
67
68 # ----- #
69 def fig_q2_attribution() -> None:
70     """(a) 四组总费用与增量 (b) 紧急购电 (c) 弃光 (d) 费用归因瀑布。"""
71     summary = load_group("q2", "summary")
72     totals = [summary[k]["total_cost_yuan"] / 1e4 for k in Q2_ORDER]
73     emergencies = [summary[k]["emergency_energy_kwh"] / 1e3 for k in Q2_ORDER]
74     curtails = [summary[k]["curtailment_energy_kwh"] / 1e3 for k in Q2_ORDER]
75     c = {k: summary[k]["total_cost_yuan"] / 1e4 for k in Q2_ORDER}
76
77     fig = plt.figure(figsize=(13.8, 8.6))
78     gs = fig.add_gridspec(2, 2, hspace=0.48, wspace=0.26)
79
80     ax = fig.add_subplot(gs[0, 0])
81     colors = [PALETTE["blue"], PALETTE["teal"], PALETTE["yellow"] if "yellow" in PALETTE
82               else PALETTE["gold"], PALETTE["red"]]
83     bars = ax.bar(range(4), totals, color=colors, edgecolor=PALETTE["ink"], linewidth=0.8,

```

```

        width=0.62)
83     ax.set_xticks(range(4))
84     ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
85     ax.set_ylabel("全年总费用 / 万元")
86     ax.set_title("问题二四种信息集/机制组合的总费用")
87     ax.set_ylim(0, max(totals) * 1.16)
88     _bar_labels(ax, bars, totals, "{:.1f}")
89     panel_label(ax, "(a)")
90
91     ax = fig.add_subplot(gs[0, 1])
92     x = np.arange(4)
93     bars = ax.bar(x - 0.19, emergencies, width=0.38, color=PALETTE["orange"],
94                  edgecolor=PALETTE["ink"], linewidth=0.7, label="紧急购电")
95     bars2 = ax.bar(x + 0.19, curtails, width=0.38, color=PALETTE["sky"],
96                  edgecolor=PALETTE["ink"], linewidth=0.7, label="弃光")
97     ax.set_xticks(x)
98     ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
99     ax.set_ylabel("电量 / MWh")
100    ax.set_title("紧急购电与弃光电量")
101    ax.legend(fontsize=9)
102    _bar_labels(ax, bars, emergencies, "{:.0f}")
103    panel_label(ax, "(b)")
104
105    ax = fig.add_subplot(gs[1, 0])
106    # 三个因素都是"去掉它 / 不用它会使费用上升多少", 因此全部为正的费用增量,
107    # 直接对比它们的量级即可看出哪一个是原结果偏低的主因。
108    delta_load = c["B_causalload_risk_feedback"] - c["A_perfectload_risk_feedback"]
109    delta_pv = c["C_causalload_point_feedback"] - c["B_causalload_risk_feedback"]
110    delta_fb = c["D_causalload_risk_nofeedback"] - c["B_causalload_risk_feedback"]
111    labels = ["完美负载信息\n$\Delta C_{load}$", "去掉光伏风险边际\n$\Delta C_{PVrisk}$",
112             "去掉实时储能反馈\n$\Delta C_{feedback}$"]
113    values = [delta_load, delta_pv, delta_fb]
114    bars = ax.bar(range(3), values, width=0.55,
115                 color=[PALETTE["red"], PALETTE["orange"], PALETTE["gold"]],
116                 edgecolor=PALETTE["ink"], linewidth=0.8)
117    _bar_labels(ax, bars, values, "{:+.1f}")
118    for i, value in enumerate(values):
119        share = 100 * value / c["B_causalload_risk_feedback"]
120        ax.text(i, value * 0.5, f"占因果基准\n{share:.1f}%", ha="center", va="center",
121               fontsize=9, color="white", fontweight="bold")
122    ax.set_xticks(range(3))
123    ax.set_xticklabels(labels, fontsize=8.8)
124    ax.set_ylabel("费用增量 / 万元")
125    ax.set_title("三项机制各自的费用贡献 (万元)")
126    ax.set_ylim(0, max(values) * 1.24)
127    panel_label(ax, "(c)")

```

```

128 ax = fig.add_subplot(gs[1, 1])
129 rows = []
130 for key in Q2_ORDER:
131     m = summary[key]
132     rows.append([m["plan_purchase_cost_yuan"] / 1e4, m["emergency_cost_yuan"] / 1e4])
133 rows = np.asarray(rows)
134 ax.bar(range(4), rows[:, 0], width=0.62, color=PALETTE["navy"], label="计划购电费",
135         edgecolor=PALETTE["ink"], linewidth=0.8)
136 ax.bar(range(4), rows[:, 1], bottom=rows[:, 0], width=0.62, color=PALETTE["orange"],
137         label="紧急购电费", edgecolor=PALETTE["ink"], linewidth=0.8)
138 for i, (plan, em) in enumerate(rows):
139     ax.text(i, plan / 2, f"{plan:.0f}", ha="center", va="center", color="white",
140             fontsize=9.5, fontweight="bold")
141     ax.text(i, plan + em + max(rows[:, 0]) * 0.012, f"{plan + em:.1f}", ha="center", va=
142             "bottom",
143             fontsize=9.5, fontweight="bold")
144 ax.set_xticks(range(4))
145 ax.set_xticklabels([Q2_LABEL[k] for k in Q2_ORDER], fontsize=9)
146 ax.set_ylabel("费用 / 万元")
147 ax.set_title("费用构成: 计划购电与紧急购电")
148 ax.set_ylim(0, (rows.sum(axis=1).max()) * 1.16)
149 ax.legend(fontsize=9)
150 panel_label(ax, "(d)")
151
152
153 def fig_q3_forecast_value() -> None:
154     """(a) S0 - S3 总费用与增量价值 (b) 季节分解 (c) 调整电量 (d) 紧急购电与弃光。"""
155     summary = load_group("q3_additive", "summary")
156     costs = np.array([summary[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER])
157
158     fig = plt.figure(figsize=(13.8, 8.6))
159     gs = fig.add_gridspec(2, 2, hspace=0.46, wspace=0.26)
160
161     ax = fig.add_subplot(gs[0, 0])
162     ax.plot(range(4), costs, marker="o", color=PALETTE["navy"], markersize=7, linewidth=2.2)
163     for i, value in enumerate(costs):
164         ax.annotate(f"{value:.1f}", (i, value), textcoords="offset points", xytext=(0, 10),
165                   ha="center", fontsize=10, fontweight="bold")
166     ax.set_xticks(range(4))
167     ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
168     ax.set_xlabel("可用的决策更新节点")
169     ax.set_ylabel("全年总费用 / 万元")
170     ax.set_title("新增决策节点带来的费用下降")
171     ax.set_ylim(costs.min() * 0.965, costs.max() * 1.022)
172     panel_label(ax, "(a)")

```

```

173
174 ax = fig.add_subplot(gs[0, 1])
175 controls = [summary["C1_load6"]["total_cost_yuan"], summary["C2_load12"]["
    total_cost_yuan"],
176             summary["C3_load18"]["total_cost_yuan"]]
177 load_value = np.array([summary[Q3_ORDER[i]]["total_cost_yuan"] - controls[i] for i in
    range(3)]) / 1e4
178 pv_value = np.array([controls[i] - summary[Q3_ORDER[i + 1]]["total_cost_yuan"] for i in
    range(3)]) / 1e4
179 labels = ["$V_6$", "$V_{12}$", "$V_{18}$"]
180 bars = ax.bar(labels, load_value, color=PALETTE["orange"],
181             edgecolor=PALETTE["ink"], linewidth=0.8, width=0.55, label="负载观测刷新")
182 ax.bar(labels, pv_value, bottom=load_value, color=PALETTE["teal"],
183             edgecolor=PALETTE["ink"], linewidth=0.8, width=0.55, label="光伏预报刷新")
184 values = load_value + pv_value
185 ax.axhline(0, color=PALETTE["ink"], linewidth=0.9)
186 for index, value in enumerate(values):
187     ax.text(index, value + max(values) * 0.025, f"{value:+.1f}", ha="center", va="bottom
        ",
188             fontsize=9.5, fontweight="bold")
189 ax.set_ylabel("增量价值 / 万元")
190 ax.set_title("节点价值的配对分解")
191 ax.set_ylim(min(0, min(values)) - 12, max(values) * 1.24)
192 ax.legend(fontsize=8.5)
193 panel_label(ax, "(b)")
194
195 ax = fig.add_subplot(gs[1, 0])
196 by_season = []
197 for key in Q3_ORDER:
198     frame = pd.read_csv(VARIANTS / "q3_additive" / key / "by_season.csv", encoding="utf
        -8-sig")
199     frame = frame.set_index("season").reindex(["spring", "summer", "autumn", "winter"])
200     by_season.append(frame)
201 seasons = ["spring", "summer", "autumn", "winter"]
202 names = ["春季", "夏季", "秋季", "冬季"]
203 matrix = np.array([[f.loc[s, "total_cost_yuan"] / 1e4 for s in seasons] for f in
    by_season])
204 im = ax.imshow(matrix, cmap="YlGnBu", aspect="auto")
205 ax.set_xticks(range(4))
206 ax.set_xticklabels(names)
207 ax.set_yticks(range(4))
208 ax.set_yticklabels([Q3_LABEL[k].replace("\n", " ") for k in Q3_ORDER], fontsize=9)
209 for i in range(4):
210     for j in range(4):
211         ax.text(j, i, f"{matrix[i, j]:.0f}", ha="center", va="center", fontsize=9.5,
212                 color="white" if matrix[i, j] > matrix.mean() else PALETTE["ink"])
213 ax.grid(False)

```

```

214 ax.set_title("季节维度的费用下降 (万元)")
215 cb = fig.colorbar(im, ax=ax, pad=0.02, shrink=0.92)
216 cb.set_label("万元", fontsize=9)
217 panel_label(ax, "(c)")
218
219 ax = fig.add_subplot(gs[1, 1])
220 up = [summary[k]["up_adjustment_energy_kwh"] / 1e3 for k in Q3_ORDER]
221 down = [summary[k]["down_adjustment_energy_kwh"] / 1e3 for k in Q3_ORDER]
222 em = [summary[k]["emergency_energy_kwh"] / 1e3 for k in Q3_ORDER]
223 x = np.arange(4)
224 ax.bar(x - 0.21, up, width=0.4, color=PALETTE["red"], label="向上调整",
225        edgecolor=PALETTE["ink"], linewidth=0.7)
226 ax.bar(x + 0.21, np.array(down) * -1, width=0.4, color=PALETTE["green"], label="向下调整",
227        edgecolor=PALETTE["ink"], linewidth=0.7)
228 ax.plot(x, em, color=PALETTE["orange"], marker="s", markersize=6, linewidth=2.0, label="
    紧急购电")
229 ax.axhline(0, color=PALETTE["ink"], linewidth=0.9)
230 ax.set_xticks(x)
231 ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
232 ax.set_ylabel("电量 / MWh")
233 ax.set_title("调整电量与紧急购电")
234 ax.legend(fontsize=9, ncol=1)
235 panel_label(ax, "(d)")
236
237 save(fig, "q3_forecast_value")
238
239
240 def fig_q3_settlement() -> None:
241     """两种结算口径分别重新优化的对比。"""
242     repl = load_group("q3_replacement", "summary")
243     addi = load_group("q3_additive", "summary")
244
245     fig, axes = plt.subplots(1, 3, figsize=(13.8, 4.4))
246     plt.subplots_adjust(wspace=0.3)
247     ax = axes[0]
248     x = np.arange(4)
249     r = [repl[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER]
250     a = [addi[k]["total_cost_yuan"] / 1e4 for k in Q3_ORDER]
251     ax.bar(x - 0.2, r, width=0.4, color=PALETTE["navy"], label="replacement 口径",
252           edgecolor=PALETTE["ink"], linewidth=0.8)
253     ax.bar(x + 0.2, a, width=0.4, color=PALETTE["gold"], label="additive 口径",
254           edgecolor=PALETTE["ink"], linewidth=0.8)
255     ax.set_xticks(x)
256     ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
257     ax.set_ylabel("全年总费用 / 万元")
258     ax.set_title("两种结算口径分别重优化")

```



```

259     ax.set_ylim(0, max(a) * 1.16)
260     ax.legend(fontsize=8.5)
261     panel_label(ax, "(a)")
262
263     ax = axes[1]
264     gap = [a[i] - r[i] for i in range(4)]
265     bars = ax.bar(x, gap, width=0.55, color=PALETTE["rose"], edgecolor=PALETTE["ink"],
266                  linewidth=0.8)
267     _bar_labels(ax, bars, gap, "{:+.2f}")
268     ax.set_xticks(x)
269     ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
270     ax.set_ylabel("additive - replacement / 万元")
271     ax.set_title("口径差异（同一优化问题的两种目标）")
272     ax.set_ylim(0, max(gap) * 1.25)
273     panel_label(ax, "(b)")
274
275     ax = axes[2]
276     for label, data, color in (("replacement", r, PALETTE["navy"]), ("additive", a, PALETTE[
277         "gold"])):
278         ax.plot(range(4), data, marker="o", color=color, linewidth=2.0, markersize=6.5,
279               label=label)
280     ax.set_xticks(range(4))
281     ax.set_xticklabels([Q3_LABEL[k] for k in Q3_ORDER], fontsize=9)
282     ax.set_ylabel("全年总费用 / 万元")
283     ax.set_title("预报时刻价值的口径稳健性")
284     ax.legend(fontsize=9)
285     panel_label(ax, "(c)")
286     save(fig, "q3_settlement")
287
288
289 def fig_q4_price_value() -> None:
290     """(a) 四组总费用 (b) 价格信息价值 (c) 价格预测误差。"""
291     summary = load_group("q4", "summary")
292     costs = np.array([summary[k]["total_cost_yuan"] / 1e4 for k in Q4_ORDER])
293
294     fig = plt.figure(figsize=(13.8, 4.6))
295     gs = fig.add_gridspec(1, 3, wspace=0.3)
296
297     ax = fig.add_subplot(gs[0, 0])
298     colors = [PALETTE["sky"], PALETTE["red"], PALETTE["teal"], PALETTE["purple"]]
299     bars = ax.bar(range(4), costs, color=colors, edgecolor=PALETTE["ink"], linewidth=0.8,
300                  width=0.6)
301     ax.set_xticks(range(4))
302     ax.set_xticklabels([Q4_LABEL[k] for k in Q4_ORDER], fontsize=8.5)
303     ax.set_ylabel("全年总费用 / 万元")
304     ax.set_title("波动电价下的四组结果")
305     ax.set_ylim(0, costs.max() * 1.16)

```

```

302     _bar_labels(ax, bars, costs, "{:.1f}")
303     panel_label(ax, "(a)")
304
305     ax = fig.add_subplot(gs[0, 1])
306     value_q2 = costs[1] - costs[0]
307     value_q3 = costs[3] - costs[2]
308     value_roll_perfect = costs[0] - costs[2]
309     value_roll_causal = costs[1] - costs[3]
310     labels = ["Q4-2\n发布信息差额", "Q4-3\n发布信息差额", "附件4直用下\n滚动调整价值", "因果
        扩展下\n滚动调整价值"]
311     values = [value_q2, value_q3, value_roll_perfect, value_roll_causal]
312     bars = ax.bar(range(4), values,
313                  color=[PALETTE["red"], PALETTE["purple"], PALETTE["teal"], PALETTE["green"]
314                        ],
315                  edgecolor=PALETTE["ink"], linewidth=0.8, width=0.6)
316     _bar_labels(ax, bars, values, "{:+.1f}")
317     ax.set_xticks(range(4))
318     ax.set_xticklabels(labels, fontsize=8.5)
319     ax.set_ylabel("费用差额 / 万元")
320     ax.set_title("价格信息与滚动调整的价值")
321     ax.set_ylim(0, max(values) * 1.28)
322     panel_label(ax, "(b)")
323
324     ax = fig.add_subplot(gs[0, 2])
325     frame = pd.read_csv(VARIANTS / "q4" / "q43_causal_price" / "price_forecast_log.csv",
326                        encoding="utf-8-sig")
327     frame["hour"] = frame["decision_time"].str.slice(0, 2).astype(int)
328     grouped = frame.groupby("hour")["price_mae"].mean()
329     hours = grouped.index.values
330     ax.bar(hours, grouped.values, color=PALETTE["purple"], edgecolor=PALETTE["ink"],
331           linewidth=0.7, width=0.6)
332     ax.set_xticks(hours)
333     ax.set_xticklabels([f"{h}" for h in hours])
334     ax.set_xlabel("决策时刻 / h")
335     ax.set_ylabel("电价预测 MAE / (元·kWh$^{-1}$)")
336     ax.set_title("因果电价预测的分时精度")
337     panel_label(ax, "(c)")
338     save(fig, "q4_price_value")
339
340 def fig_sensitivity() -> None:
341     """(a) 终端 SOC 策略 (b) 储能效率口径 (c) 日边界 SOC 分布。"""
342     sens = load_group("sensitivity", "summary")
343     terminal = sens["terminal"]
344     efficiency = sens["efficiency"]
345
346     fig = plt.figure(figsize=(13.8, 4.6))

```

```

345 gs = fig.add_gridspec(1, 3, wspace=0.3)
346
347 ax = fig.add_subplot(gs[0, 0])
348 policies = ["daily_cycle", "free"]
349 names = ["daily_cycle\n(计划 S$_1$$_4$$_4$=S$_0$)", "free\n(无终端约束)"]
350 totals = [terminal[p]["total_cost_yuan"] / 1e4 for p in policies]
351 bars = ax.bar(range(2), totals, color=[PALETTE["navy"], PALETTE["gold"]],
352             edgecolor=PALETTE["ink"], linewidth=0.8, width=0.5)
353 _bar_labels(ax, bars, totals, "{:.1f}")
354 ax.set_xticks(range(2))
355 ax.set_xticklabels(names, fontsize=9)
356 ax.set_ylabel("全年总费用 / 万元")
357 ax.set_title("终端储电量策略敏感性")
358 ax.set_ylim(0, max(totals) * 1.16)
359 panel_label(ax, "(a)")
360
361 ax = fig.add_subplot(gs[0, 1])
362 keys = list(efficiency)
363 short = ["E1\n$\eta_c=\eta_d=0.90$", "E2\n$\eta_c=\eta_d=\sqrt{0.9}$"]
364 totals = [efficiency[k]["total_cost_yuan"] / 1e4 for k in keys]
365 throughput = [efficiency[k]["storage_throughput_kwh"] / 1e3 for k in keys]
366 bars = ax.bar(range(len(keys)), totals, color=[PALETTE["teal"], PALETTE["rose"]],
367             edgecolor=PALETTE["ink"], linewidth=0.8, width=0.5)
368 _bar_labels(ax, bars, totals, "{:.1f}")
369 for i, value in enumerate(throughput):
370     ax.text(i, totals[i] * 0.5, f"吞吐\n{value:.0f} MWh", ha="center", va="center",
371           fontsize=9, color="white", fontweight="bold")
372 ax.set_xticks(range(len(keys)))
373 ax.set_xticklabels(short, fontsize=9)
374 ax.set_ylabel("全年总费用 / 万元")
375 ax.set_title("储能效率口径敏感性")
376 ax.set_ylim(0, max(totals) * 1.16)
377 panel_label(ax, "(b)")
378
379 ax = fig.add_subplot(gs[0, 2])
380 data = [terminal[p]["daily_boundary_soc"]["mean"] for p in policies]
381 x = np.arange(2)
382 ax.bar(x, data, color=[PALETTE["navy"], PALETTE["gold"]], edgecolor=PALETTE["ink"],
383       linewidth=0.8, width=0.5, label="日均日末储电量")
384 for i, p in enumerate(policies):
385     info = terminal[p]["daily_boundary_soc"]
386     ax.errorbar(i, info["mean"], yerr=[[info["mean"] - info["min"]], [info["max"] - info
387           ["mean"]]],
388               fmt="none", ecolor=PALETTE["ink"], elinewidth=1.4, capsize=7)
389     ax.text(i, info["max"] + 200, f"std={info['std']:.0f}", ha="center", fontsize=9)
389 ax.set_xticks(x)
390 ax.set_xticklabels(names, fontsize=9)

```

```

391 ax.set_ylabel("日末储电量 / kWh")
392 ax.set_title("日边界储电量的分布")
393 ax.set_ylim(0, 12500)
394 panel_label(ax, "(c)")
395 save(fig, "sensitivity")
396
397
398 def fig_dispatch_overview(day_label: str = "2025-06-21", scenario: str = "q3_additive/
    S3_0_6_12_18") -> None:
399     """典型日调度全景：(a) 功率平衡 (b) 购电 (c) 储电量 (d) 偏差电量。"""
400     path = VARIANTS / scenario / "solution.csv"
401     frame = pd.read_csv(path, encoding="utf-8-sig")
402     frame = frame[frame["date"] == day_label].reset_index(drop=True)
403     if frame.empty:
404         raise ValueError(f"{day_label} not found in {path}")
405     hours = (frame["slot"].to_numpy()) / 6.0
406
407     fig = plt.figure(figsize=(13.8, 8.4))
408     gs = fig.add_gridspec(2, 2, hspace=0.42, wspace=0.24)
409
410     ax = fig.add_subplot(gs[0, 0])
411     ax.fill_between(hours, 0, frame["load_kw"] / 1e3, color=PALETTE["sky"], alpha=0.55,
        label="小区负载")
412     ax.plot(hours, frame["actual_pv_kw"] / 1e3, color=PALETTE["gold"], linewidth=1.9, label=
        "实际光伏")
413     ax.plot(hours, frame["discharge_actual_kwh"] * 6 / 1e3, color=PALETTE["teal"], linewidth
        =1.5, label="储能放电功率")
414     ax.plot(hours, -frame["charge_actual_kwh"] * 6 / 1e3, color=PALETTE["purple"], linewidth
        =1.5, label="储能充电功率")
415     ax.axhline(0, color=PALETTE["ink"], linewidth=0.8)
416     ax.set_xlabel("时刻 / h")
417     ax.set_ylabel("功率 / MW")
418     ax.set_title(f"{day_label} 功率平衡与储能充放电")
419     ax.set_xlim(0, 24)
420     ax.legend(fontsize=8.5, ncol=2)
421     panel_label(ax, "(a)")
422
423     ax = fig.add_subplot(gs[0, 1])
424     ax.plot(hours, frame["baseline_grid_kwh"] * 6 / 1e3, color=PALETTE["gray"], linewidth
        =1.6,
425         linestyle="--", label="0:00 基准计划 $G^{\{p\}}$")
426     ax.plot(hours, frame["adjusted_grid_kwh"] * 6 / 1e3, color=PALETTE["navy"], linewidth
        =1.9,
427         label="最终调整购电 $G^{\{a\}}$")
428     ax.fill_between(hours, frame["baseline_grid_kwh"] * 6 / 1e3, frame["adjusted_grid_kwh"]
        * 6 / 1e3,
429         where=(frame["adjusted_grid_kwh"] > frame["baseline_grid_kwh"]),

```

```

430         color=PALETTE["red"], alpha=0.28, label="向上调整")
431     ax.fill_between(hours, frame["baseline_grid_kwh"] * 6 / 1e3, frame["adjusted_grid_kwh"]
432                     * 6 / 1e3,
433                     where=(frame["adjusted_grid_kwh"] <= frame["baseline_grid_kwh"]),
434                     color=PALETTE["green"], alpha=0.28, label="向下调整")
435     ax.set_xlabel("时刻 / h")
436     ax.set_ylabel("购电功率 / MW")
437     ax.set_title("计划与调整购电策略")
438     ax.set_xlim(0, 24)
439     ax.legend(fontsize=8.5)
440     panel_label(ax, "(b)")
441
442     ax = fig.add_subplot(gs[1, 0])
443     ax.plot(hours, frame["soc_actual_kwh"] / 1e3, color=PALETTE["navy"], linewidth=2.0)
444     ax.fill_between(hours, frame["soc_actual_kwh"] / 1e3, 1.2, color=PALETTE["sky"], alpha
445                     =0.28)
446     ax.axhline(1.2, color=PALETTE["red"], linestyle="--", linewidth=1.1)
447     ax.axhline(10.8, color=PALETTE["red"], linestyle="--", linewidth=1.1)
448     ax.text(0.2, 1.35, "SOC 下限 1200 kWh", fontsize=8.5, color=PALETTE["red"])
449     ax.text(0.2, 10.95, "SOC 上限 10800 kWh", fontsize=8.5, color=PALETTE["red"])
450     ax.set_xlabel("时刻 / h")
451     ax.set_ylabel("储电量 / MWh")
452     ax.set_title("储能荷电状态的日内轨迹")
453     ax.set_xlim(0, 24)
454     ax.set_ylim(0.5, 12.2)
455     panel_label(ax, "(c)")
456
457     ax = fig.add_subplot(gs[1, 1])
458     ax.bar(hours, frame["emergency_kwh"] * 6, width=0.13, color=PALETTE["red"],
459           edgecolor=PALETTE["ink"], linewidth=0.5, label="紧急购电")
460     ax.bar(hours, frame["curtail_actual_kwh"] * 6, width=0.13, color=PALETTE["olive"],
461           edgecolor=PALETTE["ink"], linewidth=0.5, label="弃光")
462     ax.set_xlabel("时刻 / h")
463     ax.set_ylabel("功率 / kW")
464     ax.set_title("紧急购电与弃光的功率分布")
465     ax.set_xlim(0, 24)
466     ax.legend(fontsize=9)
467     panel_label(ax, "(d)")
468     save(fig, f"dispatch_{day_label.replace('-', ' ')}")
469
470 def main() -> None:
471     fig_q2_attribution()
472     fig_q3_forecast_value()
473     fig_q4_price_value()
474     fig_sensitivity()
475     for day in ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"):

```

```

475         try:
476             fig_dispatch_overview(day)
477         except ValueError as exc:
478             print("[warn]", exc)
479     print("[fig] 结果层配图完成 →", FIGS)
480
481
482 if __name__ == "__main__":
483     main()

```

B.23 技术路线图生成 (fig_route.py)

Listing 23:

```

1  """整体技术路线图（五带结构）。
2
3  可编辑源文件同时保留在同目录的 technical_route.drawio; 本脚本用 matplotlib 渲染
4  PNG, 保证在无 draw.io Desktop CLI 的环境下也能出图。
5  """
6  from __future__ import annotations
7
8  import sys
9  from pathlib import Path
10
11  import matplotlib
12
13  matplotlib.use("Agg")
14  import matplotlib.pyplot as plt # noqa: E402
15  from matplotlib.patches import FancyArrowPatch, FancyBboxPatch # noqa: E402
16
17  ROOT = Path(__file__).resolve().parents[1]
18  FIGS = ROOT / "paper" / "figures"
19
20  INK = "#1A2028"
21  NAVY = "#1F3B73"
22  BANDS = [
23      ("数据层", "#E8FOFA", "#3E6FB0"),
24      ("统一内核", "#E6F4F1", "#2E8B84"),
25      ("四问模型", "#FBF0DE", "#D8A22B"),
26      ("求解算法", "#EDE9F6", "#7A5BA6"),
27      ("验证交付", "#EAF3E8", "#5FA05A"),
28  ]
29
30  CONTENT = [
31      ["附件 1\n典型日电价·负载·光伏", "附件 2\n全年 144 时段负载与光伏实际值",
32       "附件 3\n每日 4 次未来 24 h 光伏预报", "附件 4\n全年 144 时段实时电价"],

```

```

33 ["母线能量平衡\n式 (1)", "储能 SOC 状态转移\n $S_t=S_{t-1}+\eta_c C_t-H_t/\eta_d$ \n式
    (2) 核心状态方程",
34 "容量与功率限值\n式 (3) (4)", "购电非负与弃光上界\n式 (5) (6)"],
35 ["问题一\n确定性日前 LP\n首末 SOC 闭环",
36 "问题二\n因果预测+报童定价\n $5\%$  倍电价  $\rightarrow$   $0.2\%$  分位点\n实时储能反馈",
37 "问题三\n四时刻滚动优化\n两种结算口径\n分别重优化",
38 "问题四\n波动电价重算\nperfect / causal\n价格信息边界"],
39 ["HiGHS 线性规划\n原始一对偶容差  $10^{-9}$ ", "三层词典序目标\n费用  $\rightarrow$  吞吐量  $\rightarrow$  弃光",
40 "滚动在线选型\n仅用已发生误差", "因果偏差校正\n分发布节点分位数"],
41 ["物理残差校验\n平衡/SOC 残差  $<10^{-12}$ ", "未来信息泄漏\nmutation 测试",
42 "结算口径单元测试\n与 legacy 回归", "官方 result1-4\nExcel 回填与回读"],
43 ]
44
45
46 def main() -> None:
47     plt.rcParams.update({
48         "font.family": "sans-serif",
49         "font.sans-serif": ["SimSun", "Times New Roman", "DejaVu Sans"],
50         "axes.unicode_minus": False,
51         "mathtext.fontset": "dejavusans",
52     })
53     band_h = 1.0
54     gap = 0.34
55     n_bands = len(BANDS)
56     fig_h = n_bands * band_h + (n_bands - 1) * gap + 0.5
57     fig, ax = plt.subplots(figsize=(13.2, fig_h * 1.35))
58     ax.set_xlim(0, 13.2)
59     ax.set_ylim(0, fig_h)
60     ax.axis("off")
61
62     left = 1.42
63     box_w = 2.79
64     box_gap = 0.155
65
66     for b, (band_name, fill, edge) in enumerate(BANDS):
67         y = fig_h - 0.25 - (b + 1) * band_h - b * gap
68         ax.add_patch(FancyBboxPatch(
69             (0.10, y), 13.00, band_h,
70             boxstyle="round,pad=0.008,rounding_size=0.10",
71             facecolor=fill, edgecolor=edge, linewidth=1.5, alpha=0.55, zorder=1,
72         ))
73         ax.text(0.72, y + band_h / 2, band_name, ha="center", va="center",
74                 fontsize=11.0, fontweight="bold", color=edge, rotation=90, zorder=3)
75         for j, text in enumerate(CONTENT[b]):
76             x = left + j * (box_w + box_gap)
77             emphasized = b == 1 and j == 1
78             ax.add_patch(FancyBboxPatch(

```

```

79         (x, y + 0.09), box_w, band_h - 0.18,
80         boxstyle="round,pad=0.01,rounding_size=0.09",
81         facecolor="white", edgecolor=edge,
82         linewidth=2.0 if emphasized else 1.1, zorder=2,
83     ))
84     ax.text(x + box_w / 2, y + band_h / 2, text, ha="center", va="center",
85            fontsize=9.8 if emphasized else 10.2, color=INK, zorder=3, linespacing
86            =1.55)
87
88     for b in range(n_bands - 1):
89         y_top = fig_h - 0.25 - (b + 1) * band_h - b * gap
90         y_bottom = y_top - gap
91         for j in range(4):
92             x = left + j * (box_w + box_gap) + box_w / 2
93             ax.add_patch(FancyArrowPatch(
94                 (x, y_top), (x, y_bottom + 0.001),
95                 arrowstyle="-|>", mutation_scale=13,
96                 linewidth=1.5, color=NAVY, alpha=0.75, zorder=4,
97                 shrinkA=0, shrinkB=0,
98             ))
99
100     FIGS.mkdir(parents=True, exist_ok=True)
101     fig.savefig(FIGS / "technical_route.png", dpi=320, bbox_inches="tight", facecolor="white")
102     fig.savefig(FIGS / "technical_route.pdf", bbox_inches="tight", facecolor="white")
103     plt.close(fig)
104     print("[fig] technical_route 完成 →", FIGS / "technical_route.png")
105
106 if __name__ == "__main__":
107     main()

```

B.24 绘图总控 (plot_results.py)

Listing 24:

```

1 from __future__ import annotations
2 import csv,json
3 from datetime import datetime
4 from pathlib import Path
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from matplotlib.font_manager import FontProperties
8
9 ROOT=Path(__file__).resolve().parents[1]
10 FONT=Path(r"C:\Windows\Fonts\msyh.ttc")

```



```

11 fp=FontProperties(fname=str(FONT)) if FONT.exists() else None
12 plt.rcParams["axes.unicode_minus"]=False
13 COLORS={"Q2":"#557A95","Q3":"#1B998B","Q4-2":"#D9A441","Q4-3":"#C8553D"}
14
15 def j(path): return json.loads(path.read_text(encoding="utf-8"))
16 def csvrows(path):
17     with path.open(encoding="utf-8-sig",newline="") as f: return list(csv.DictReader(f))
18 def label(ax,x,y,fmt="{:.2f}"):
19     for xi,yi in zip(x,y): ax.text(xi,yi,fmt.format(yi),ha="center",va="bottom",fontsize=9,
20         fontproperties=fp)
21
22 def save(fig,path):
23     path.parent.mkdir(parents=True,exist_ok=True); fig.tight_layout(); fig.savefig(path,
24         bbox_inches="tight"); plt.close(fig)
25
26 def main():
27     # Q1 dispatch
28     rows=csvrows(ROOT/"outputs"/"q1"/"solution.csv"); x=np.arange(144)/6
29     load=np.array([float(r["load_kw"]) for r in rows]); pv=np.array([float(r["pv_kw"]) for r
30         in rows])
31     grid=np.array([float(r["grid_kwh"]) for r in rows])*6
32     ch=np.array([float(r["charge_kwh"]) for r in rows])*6; dis=np.array([float(r["
33         discharge_kwh"]) for r in rows])*6
34     soc=np.array([float(r["soc_kwh"]) for r in rows]); price=np.array([float(r["price"]) for
35         r in rows])
36     fig,(ax1,ax2)=plt.subplots(2,1,figsize=(11,7),sharex=True)
37     ax1.plot(x,load,label="负载",lw=1.8,color="#263238"); ax1.plot(x,pv,label="光伏",lw=1.8,
38         color="#E6A700")
39     ax1.plot(x,grid,label="外网购电",lw=1.6,color="#386FA4")
40     ax1.fill_between(x,0,ch,alpha=.25,color="#1B998B",label="充电功率")
41     ax1.fill_between(x,0,-dis,alpha=.25,color="#C8553D",label="放电功率")
42     ax1.set_ylabel("功率 / kW",fontproperties=fp); ax1.grid(alpha=.2); ax1.legend(ncol=5,
43         prop=fp,loc="upper center")
44     ax2.plot(x,soc,color="#6A4C93",lw=2,label="SOC"); ax2.set_ylabel("储电量 / kWh",
45         fontproperties=fp)
46     ax2.set_xlabel("时刻 / h",fontproperties=fp); ax2.grid(alpha=.2)
47     axp=ax2.twinx(); axp.step(x,price,where="post",color="#8D6E63",alpha=.75,label="电价")
48     axp.set_ylabel("电价 / 元/kWh",fontproperties=fp)
49     fig.suptitle("问题1: 典型日购电-储能联合调度",fontproperties=fp,fontsize=15)
50     save(fig,ROOT/"figures"/"q1"/"q1_dispatch.pdf")
51
52 metrics={name:j(ROOT/"outputs"/folder/"metrics.json") for name,folder in
53     [(("Q2","q2"),("Q3","q3"),("Q4-2","q4-2"),("Q4-3","q4-3"))]}
54 names=list(metrics); xpos=np.arange(4)
55 costs=np.array([metrics[n]["total_cost_yuan"]/1e4 for n in names])
56 fig,ax=plt.subplots(figsize=(8,5)); bars=ax.bar(xpos, costs,color=[COLORS[n] for n in
57     names],width=.62)

```

```

49 ax.set_xticks(xpos,names); ax.set_ylabel("全年总费用 / 万元",fontproperties=fp); ax.grid
    (axis="y",alpha=.2)
50 ax.set_title("固定与波动电价下的全年费用对比",fontproperties=fp,fontsize=14); label(ax,
    xpos, costs)
51 save(fig,ROOT/"figures"/"comparison"/"annual_cost_comparison.pdf")
52
53 emergency=np.array([metrics[n]["emergency_energy_kwh"]/1e4 for n in names])
54 fig,ax=plt.subplots(figsize=(8,5)); ax.bar(xpos,emergency,color=[COLORS[n] for n in
    names],width=.62)
55 ax.set_xticks(xpos,names); ax.set_ylabel("紧急购电量 / 万kWh",fontproperties=fp); ax.
    grid(axis="y",alpha=.2)
56 ax.set_title("滚动预报对 5 倍紧急购电的抑制",fontproperties=fp,fontsize=14); label(ax,
    xpos,emergency,fmt="{:.3f}")
57 save(fig,ROOT/"figures"/"comparison"/"emergency_energy_comparison.pdf")
58
59 variants=[("原始阶梯","q3/variants/raw_step"),("校正阶梯","q3/variants/calibrated_step")
    ,("校正线性","q3")]
60 vm=[j(ROOT/"outputs"/p/"metrics.json") for _,p in variants]
61 vcost=np.array([m["total_cost_yuan"]/1e4 for m in vm]); vem=np.array([m["
    emergency_energy_kwh"]/1e4 for m in vm])
62 fig,ax=plt.subplots(figsize=(9,5.5)); xx=np.arange(3); width=.34
63 b1=ax.bar(xx-width/2,vcost,width,color="#557A95",label="总费用 (万元)")
64 ax.set_ylabel("总费用 / 万元",fontproperties=fp); ax.set_xticks(xx,[v[0] for v in
    variants],fontproperties=fp)
65 ax2=ax.twinx(); b2=ax2.bar(xx+width/2,vem,width,color="#C8553D",label="紧急购电 (万kWh)
    ")
66 ax2.set_ylabel("紧急购电量 / 万kWh",fontproperties=fp); ax.grid(axis="y",alpha=.2)
67 ax.set_title("Q3 策略变更的完整对照",fontproperties=fp,fontsize=14)
68 ax.bar_label(b1,fmt="%.2f",padding=3,fontsize=8)
69 ax2.bar_label(b2,fmt="%.3f",padding=3,fontsize=8)
70 ax.legend(handles=[b1,b2],labels=["总费用","紧急购电"],prop=fp,loc="upper right")
71 save(fig,ROOT/"figures"/"comparison"/"q3_variant_comparison.pdf")
72
73 fig,ax=plt.subplots(figsize=(11,5.5))
74 for name,folder in [("Q2","q2"),("Q3","q3"),("Q4-2","q4-2"),("Q4-3","q4-3")]:
75     rows=csvrows(ROOT/"outputs"/folder/"daily_metrics.csv")
76     dates=np.array([datetime.fromisoformat(r["date"]) for r in rows])
77     daily=np.array([float(r["total_cost"]) for r in rows])
78     smooth=np.convolve(daily,np.ones(7)/7,mode="same")
79     ax.plot(dates[3:-3],smooth[3:-3]/1e4,label=name,color=COLORS[name],lw=1.5)
80 ax.set_ylabel("7日移动平均费用 / 万元/日",fontproperties=fp); ax.grid(alpha=.2)
81 ax.set_title("全年日费用变化 (7日移动平均)",fontproperties=fp,fontsize=14); ax.legend(
    prop=fp,ncol=4)
82 save(fig,ROOT/"figures"/"comparison"/"daily_cost_timeseries.pdf")
83 print("generated 5 PDF figures")
84
85 if __name__=="__main__": main()

```

B.25 mutation 测试辅助函数 (mutate.py)

Listing 25:

```
1  """测试辅助：在临时目录构造被人为篡改的附件数据，用于因果性 mutation test."""
2  from __future__ import annotations
3
4  import shutil
5  from pathlib import Path
6
7  import openpyxl
8
9  from ..solve.io_data import DEFAULT_DATA_ROOT
10
11  ATTACHMENTS = ("附件1.xlsx", "附件2.xlsx", "附件3.xlsx", "附件4.xlsx")
12
13
14  def make_root(tmp: Path, source: Path = DEFAULT_DATA_ROOT) -> Path:
15      """复制附件 1~4 到临时目录，返回可直接传给 run_q2/run_q3 的 data_root."""
16      tmp.mkdir(parents=True, exist_ok=True)
17      for name in ATTACHMENTS:
18          target = tmp / name
19          if not target.exists():
20              shutil.copy2(source / name, target)
21      return tmp
22
23
24  def _mutate_sheet(path: Path, sheet: str, day_index: int, from_slot: int, factor: float) ->
    None:
25      """把第 day_index 天 (0 基) 从 from_slot 开始的真实值乘以 factor."""
26      workbook = openpyxl.load_workbook(path)
27      worksheet = workbook[sheet] if sheet in workbook.sheetnames else workbook.worksheets[0]
28      row = day_index + 2 # 第 1 行为表头
29      for col in range(from_slot + 2, worksheet.max_column + 1):
30          cell = worksheet.cell(row, col)
31          if cell.value is None:
32              continue
33          cell.value = float(cell.value) * factor
34      workbook.save(path)
35      workbook.close()
36
37
38  def mutate_attachment2_load(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
```

```

39     _mutate_sheet(root / "附件2.xlsx", "小区负载", day_index, from_slot, factor)
40
41
42 def mutate_attachment2_pv(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
43     _mutate_sheet(root / "附件2.xlsx", "光伏发电实际功率", day_index, from_slot, factor)
44
45
46 def mutate_attachment4_price(root: Path, day_index: int, from_slot: int = 0, factor: float =
    10.0) -> None:
47     _mutate_sheet(root / "附件4.xlsx", "Sheet1", day_index, from_slot, factor)
48
49
50 def max_abs_diff(left, right) -> float:
51     import numpy as np
52
53     return float(np.max(np.abs(np.asarray(left, dtype=float) - np.asarray(right, dtype=float
        ))))

```

B.26 未来信息泄漏 mutation 测试 (test_causality.py)

Listing 26:

```

1  """未来信息泄漏自动测试（任务书 §9）。
2
3  核心判据：在 causal 信息集模式下，人为篡改**决策时刻之后**才会发生的
4  真实负载 / 真实光伏 / 真实电价，决策时刻的优化输入必须完全不变。
5  """
6  from __future__ import annotations
7
8  import shutil
9  import tempfile
10 import unittest
11 from pathlib import Path
12
13 import numpy as np
14
15 from ..solve.io_data import read_attachment4
16 from ..solve.q2 import run_q2
17 from ..solve.q3 import run_q3
18 from .mutate import (
19     make_root,
20     mutate_attachment2_load,
21     mutate_attachment2_pv,
22     mutate_attachment4_price,
23 )

```

```

24
25 TARGET_DAY = 35
26 DAYS = 36
27 TOL = 1e-10
28
29
30 class CausalityTestBase(unittest.TestCase):
31     @classmethod
32     def setUpClass(cls) -> None:
33         cls._tmpdir = tempfile.mkdtemp(prefix="mathmodel_causal_")
34         cls.tmp = Path(cls._tmpdir)
35         cls.base = make_root(cls.tmp / "base")
36         cls.mut_load = make_root(cls.tmp / "mut_load")
37         mutate_attachment2_load(cls.mut_load, TARGET_DAY, 0, 10.0)
38         cls.mut_pv = make_root(cls.tmp / "mut_pv")
39         mutate_attachment2_pv(cls.mut_pv, TARGET_DAY, 0, 10.0)
40         cls.mut_both_after_6 = make_root(cls.tmp / "mut_both_after6")
41         mutate_attachment2_load(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
42         mutate_attachment2_pv(cls.mut_both_after_6, TARGET_DAY, 36, 10.0)
43         cls.mut_price_after_6 = make_root(cls.tmp / "mut_price_after6")
44         mutate_attachment4_price(cls.mut_price_after_6, TARGET_DAY, 36, 10.0)
45
46     @classmethod
47     def tearDownClass(cls) -> None:
48         shutil.rmtree(cls._tmpdir, ignore_errors=True)
49
50     @staticmethod
51     def _run_q2(root: Path, **kwargs):
52         return run_q2(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
53             , **kwargs)
54
55     @staticmethod
56     def _run_q3(root: Path, **kwargs):
57         kwargs.setdefault("settlement_mode", "replacement")
58         return run_q3(data_root=root, max_days=DAYS, write_outputs=False, load_mode="causal"
59             , **kwargs)
60
61 class Q2LoadCausalityTest(CausalityTestBase):
62     def test_future_load_cannot_change_plan(self):
63         base = self._run_q2(self.base)
64         mutated = self._run_q2(self.mut_load)
65         b = base["records"][TARGET_DAY]["load_plan_kw"]
66         m = mutated["records"][TARGET_DAY]["load_plan_kw"]
67         self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实负载被篡改后 0:00 计划输入发生变化")

```

```

68     def test_future_load_changes_execution_only(self):
69         """篡改真实负载应当改变紧急购电，说明该数据确实通过执行通道发挥作用。"""
70         base = self._run_q2(self.base)
71         mutated = self._run_q2(self.mut_load)
72         b = base["records"][TARGET_DAY]["execution"].emergency.sum()
73         m = mutated["records"][TARGET_DAY]["execution"].emergency.sum()
74         self.assertNotAlmostEqual(b, m, places=3)
75
76
77     class Q2PVCausalityTest(CausalityTestBase):
78         def test_future_pv_cannot_change_plan(self):
79             base = self._run_q2(self.base)
80             mutated = self._run_q2(self.mut_pv)
81             b = base["records"][TARGET_DAY]["pv_plan_kw"]
82             m = mutated["records"][TARGET_DAY]["pv_plan_kw"]
83             self.assertLess(float(np.max(np.abs(b - m))), TOL, "第 d 天真实光伏被篡改后 0:00 计
                划输入发生变化")
84
85
86     class Q3NodeCausalityTest(CausalityTestBase):
87         def test_node_inputs_do_not_see_future_load_or_pv(self):
88             base = self._run_q3(self.base)
89             mutated = self._run_q3(self.mut_both_after_6)
90             for day in (TARGET_DAY,):
91                 # 只冻结 6:00 这一次决策的输入；12:00/18:00 的预测可以合法使用 6:00-12:00
92                 # 已经执行完的观测（这正是滚动在线预测的应有之义）。
93                 b_load = base["records"][day]["load_plan_kw"][36:72]
94                 m_load = mutated["records"][day]["load_plan_kw"][36:72]
95                 self.assertLess(float(np.max(np.abs(b_load - m_load))), TOL, "6:00 负载预测输入
                    受到未来真实负载影响")
96                 b_block = base["records"][day]["adjusted_grid"][36:72]
97                 m_block = mutated["records"][day]["adjusted_grid"][36:72]
98                 self.assertLess(float(np.max(np.abs(b_block - m_block))), TOL, "6:00 优化结果受
                    到未来真实负载/光伏影响")
99                 b_base = base["records"][day]["baseline_plan"].grid
100                m_base = mutated["records"][day]["baseline_plan"].grid
101                self.assertLess(float(np.max(np.abs(b_base - m_base))), TOL, "0:00 基准计划受到
                    未来真实负载/光伏影响")
102
103
104     class Q4PriceCausalityTest(CausalityTestBase):
105         def test_causal_price_ignores_future_realized_price(self):
106             prices = read_attachment4(self.base).price
107             base = self._run_q3(self.base, price_matrix=prices, price_mode="causal")
108             mutated = self._run_q3(self.mut_price_after_6, price_matrix=prices, price_mode="
                causal")
109             b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]

```

```

110         m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
111         self.assertLess(float(np.max(np.abs(b - m))), TOL, "causal 电价模式下提前看到未来真
            实电价")
112
113     def test_perfect_price_does_see_future_realized_price(self):
114         """反证: perfect 模式下同一篡改确实会改变结果 (说明测试本身有鉴别力)。"""
115         base = self._run_q3(self.base, price_matrix=read_attachment4(self.base).price,
            price_mode="perfect")
116         mutated = self._run_q3(
117             self.mut_price_after_6,
118             price_matrix=read_attachment4(self.mut_price_after_6).price,
119             price_mode="perfect",
120         )
121         b = base["records"][TARGET_DAY]["adjusted_grid"][36:72]
122         m = mutated["records"][TARGET_DAY]["adjusted_grid"][36:72]
123         self.assertGreater(float(np.max(np.abs(b - m))), 1e-6)
124
125
126 if __name__ == "__main__":
127     unittest.main()

```

B.27 主结果与计划持续性测试 (test_q2_q3.py)

Listing 27:

```

1  import unittest
2  import numpy as np
3  from ..solve.io_data import read_attachment1,read_attachment2,read_attachment3,
    read_attachment4,hourly_to_ten_min
4  from ..solve.forecast import OnlinePVForecaster
5  from ..solve.lp_core import solve_dispatch,solve_adjustment_dispatch,SOC_MIN,SOC_MAX
6  from ..solve.q3 import run_q3
7  from ..solve.settle import execute_with_realtime_storage_feedback
8
9  class Q2Q3Tests(unittest.TestCase):
10     def test_attachment_dimensions(self):
11         data=read_attachment2(); forecast=read_attachment3(); prices=read_attachment4()
12         self.assertEqual(data.load_kw.shape,(365,144)); self.assertEqual(data.pv_kw.shape
            ,(365,144))
13         self.assertEqual(forecast.hourly_kw.shape,(365,4,24)); self.assertEqual(prices.price
            .shape,(365,144))
14         step=hourly_to_ten_min(forecast.hourly_kw[0,0],"step")
15         self.assertAlmostEqual(float(step.sum()/6),float(forecast.hourly_kw[0,0].sum()),
            places=9)
16
17     def test_forecast_is_sequential(self):

```

```

18         f=OnlinePVForecaster(np.ones(144)); first=f.predict(); self.assertEqual(first.
           history_days_used,0)
19         actual=np.r_[np.zeros(40),np.ones(60)*100,np.zeros(44)]; f.observe(first,actual)
20         second=f.predict(); self.assertEqual(second.history_days_used,1)
21         self.assertTrue(np.allclose(second.bases["persistence"],actual))
22         with self.assertRaises(ValueError): f.observe(first,actual)
23
24     def test_realtime_controller_constraints(self):
25         day=read_attachment1(); plan=solve_dispatch(day.load_kw,np.zeros(144),day.price,
           soc_initial=6000,soc_terminal=6000)
26         actual=np.maximum(day.pv_kw*1.3,0); execution=execute_with_realtime_storage_feedback
           (plan,day.load_kw,actual,soc_initial=6000)
27         balance=execution.grid+actual/6+execution.discharge+execution.emergency-day.load_kw
           /6-execution.charge-execution.curtail
28         self.assertLess(float(np.max(np.abs(balance))),1e-7)
29         self.assertGreaterEqual(float(execution.soc.min()),SOC_MIN-1e-7)
30         self.assertLessEqual(float(execution.soc.max()),SOC_MAX+1e-7)
31         self.assertLess(float(np.max(np.minimum(execution.charge,execution.discharge))),1e
           -7)
32
33     def test_adjustment_linearization(self):
34         gp=np.array([900.0,900.0,700.0]); ga=np.array([700.0,900.0,900.0])
35         b=np.maximum(ga-gp,0); linear=0.5*gp+0.5*ga+b
36         piece=np.minimum(gp,ga)+0.5*np.maximum(gp-ga,0)+1.5*np.maximum(ga-gp,0)
37         self.assertTrue(np.allclose(linear,[800,900,1000])); self.assertTrue(np.allclose(
           linear,piece))
38
39     def test_adjustment_solver_balance(self):
40         day=read_attachment1(); base=solve_dispatch(day.load_kw,day.pv_kw,day.price,
           soc_initial=6000,soc_terminal=6000)
41         result=solve_adjustment_dispatch(day.load_kw,day.pv_kw*0.9,day.price,base.grid,np.
           ones(144,dtype=bool),
42                                     soc_initial=6000,soc_terminal=6000)
43         d=result.dispatch
44         residual=d.grid+day.pv_kw*0.9/6+d.discharge-day.load_kw/6-d.charge-d.curtail
45         self.assertLess(float(np.max(np.abs(residual))),1e-7)
46         self.assertLess(float(np.max(np.abs(result.increase-np.maximum(d.grid-base.grid,0))
           ),1e-6)
47
48     def test_additive_milp_enforces_storage_exclusivity(self):
49         day=read_attachment1(); base=solve_dispatch(day.load_kw,day.pv_kw,day.price,
           soc_initial=6000,soc_terminal=6000)
50         result=solve_adjustment_dispatch(day.load_kw,day.pv_kw*0.9,day.price,base.grid,np.
           ones(144,dtype=bool),
51                                     soc_initial=6000,soc_terminal=6000,settlement_mode=
           "additive")
52         self.assertLess(float(np.max(np.minimum(result.dispatch.charge,result.dispatch.

```



```

        discharge))),1e-9)
53
54     def test_disabled_nodes_keep_latest_active_plan(self):
55         result=run_q3(max_days=2,write_outputs=False,update_nodes=(0,36),settlement_mode="
            replacement")
56         record=result["records"][-1]
57         self.assertTrue(np.all(record["plan_source_nodes"][:36]==0))
58         self.assertTrue(np.all(record["plan_source_nodes"][36:]==36))
59         self.assertLess(float(np.max(np.abs(record["adjusted_grid"][72:]-record["revisions"
            ][1,72:]))),1e-9)
60
61     if __name__=="__main__": unittest.main()

```

B.28 论文数值与提交口径一致性测试 (test_paper_consistency.py)

Listing 28:

```

1  """V4 论文数字、强制表格与提交工作簿的一致性测试。"""
2  from __future__ import annotations
3
4  import hashlib
5  import json
6  from pathlib import Path
7
8
9  ROOT = Path(__file__).resolve().parents[1]
10 VARIANTS = ROOT / "outputs" / "variants" / "model_audit_v4"
11 DATES = ("2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21")
12
13
14 def _json(path: Path) -> dict:
15     return json.loads(path.read_text(encoding="utf-8"))
16
17
18 def _sha256(path: Path) -> str:
19     return hashlib.sha256(path.read_bytes()).hexdigest()
20
21
22 def test_additive_main_cost_decomposition_and_exclusivity() -> None:
23     metrics = _json(VARIANTS / "q3_additive" / "S3_0_6_12_18" / "metrics.json")
24     components = (
25         metrics["baseline_purchase_cost_yuan"]
26         + metrics["adjustment_cost_yuan"]
27         + metrics["emergency_cost_yuan"]
28     )
29     assert abs(metrics["total_cost_yuan"] - components) < 1e-6

```

```

30     assert metrics["max_simultaneous_charge_discharge"] < 1e-5
31     assert metrics["passed"] is True
32
33
34 def test_paired_node_decomposition_is_exact() -> None:
35     summary = _json(VARIANTS / "q3_additive" / "summary.json")
36     chain = (
37         ("S0_0", "C1_load6", "S1_0_6"),
38         ("S1_0_6", "C2_load12", "S2_0_6_12"),
39         ("S2_0_6_12", "C3_load18", "S3_0_6_12_18"),
40     )
41     for before, control, after in chain:
42         total = summary[before]["total_cost_yuan"] - summary[after]["total_cost_yuan"]
43         load_part = summary[before]["total_cost_yuan"] - summary[control]["total_cost_yuan"]
44         pv_part = summary[control]["total_cost_yuan"] - summary[after]["total_cost_yuan"]
45         assert abs(total - load_part - pv_part) < 1e-6
46
47
48 def test_required_date_tables_cover_all_contract_dates() -> None:
49     generated = ROOT / "paper" / "generated"
50     for name in ("q2_required_tables.tex", "q3_required_tables.tex", "q4_required_tables.tex"):
51         text = (generated / name).read_text(encoding="utf-8")
52         for date in DATES:
53             assert date in text
54
55
56 def test_q4_submission_uses_attachment4_direct_results() -> None:
57     pairs = (
58         ("result4-2.xlsx", VARIANTS / "q4" / "q42_perfect_price" / "result4-2.xlsx"),
59         ("result4-3.xlsx", VARIANTS / "q4" / "q43_perfect_price" / "result4-3.xlsx"),
60     )
61     for name, source in pairs:
62         assert _sha256(ROOT / "submit" / name) == _sha256(source)

```

B.29 结算口径单元测试 (test_settlement.py)

Listing 29:

```

1  """Q3 结算口径单元测试 (任务书 §10) 。
2
3  replacement:  $C = p[\min(G_p, G_a) + 0.5(G_p - G_a)^+ + 1.5(G_a - G_p)^+]$ 
4  additive    :  $C = p \cdot G_p + 0.5p(G_p - G_a)^+ + 1.5p(G_a - G_p)^+$ 
5  同时验证优化目标与直接回代公式一致 ( $gap < 1e-7$ ) ; additive 使用互斥 MILP。
6  """
7  from __future__ import annotations

```

```

8
9 import unittest
10
11 import numpy as np
12
13 from ..solve.lp_core import settlement_cost, solve_adjustment_dispatch
14 from ..solve.params import DEFAULT_STORAGE
15
16
17 class ReplacementFormulaTest(unittest.TestCase):
18     def test_purchase_below_plan(self):
19         #  $G_p=900$ ,  $G_a=700 \rightarrow 700 + 0.5 \cdot 200 = 800$ 
20         value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
21                                 replacement")
22
23         self.assertAlmostEqual(value, 800.0, places=9)
24
25     def test_purchase_above_plan(self):
26         #  $G_p=700$ ,  $G_a=900 \rightarrow 700 + 1.5 \cdot 200 = 1000$ 
27         value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
28                                 replacement")
29
30         self.assertAlmostEqual(value, 1000.0, places=9)
31
32     def test_unchanged(self):
33         value = settlement_cost(np.array([800.0]), np.array([800.0]), np.array([1.0]), "
34                                 replacement")
35
36         self.assertAlmostEqual(value, 800.0, places=9)
37
38
39 class AdditiveFormulaTest(unittest.TestCase):
40     def test_purchase_below_plan(self):
41         #  $G_p=900$ ,  $G_a=700 \rightarrow 900 + 0.5 \cdot 200 = 1000$ 
42         value = settlement_cost(np.array([900.0]), np.array([700.0]), np.array([1.0]), "
43                                 additive")
44
45         self.assertAlmostEqual(value, 1000.0, places=9)
46
47     def test_purchase_above_plan(self):
48         #  $G_p=700$ ,  $G_a=900 \rightarrow 700 + 1.5 \cdot 200 = 1000$ 
49         value = settlement_cost(np.array([700.0]), np.array([900.0]), np.array([1.0]), "
50                                 additive")
51
52         self.assertAlmostEqual(value, 1000.0, places=9)
53
54
55 def _case(seed: int, t: int = 24, mask_all: bool = True):
56     rng = np.random.default_rng(seed)
57     load = 800.0 + 200.0 * rng.random(t)
58     pv = 400.0 * rng.random(t)
59     price = 0.3 + 0.7 * rng.random(t)

```

```

50     baseline = np.maximum((load - pv) * (1 / 6), 0.0)
51     mask = np.ones(t, dtype=bool) if mask_all else np.r_[np.ones(t - 6, dtype=bool), np.
        zeros(6, dtype=bool)]
52     return load, pv, price, baseline, mask
53
54
55 def _model_objective(result, price, baseline, mode: str) -> float:
56     """模型侧分段线性目标（只统计允许调整的时段）。"""
57     mask = result.adjustment_mask
58     p = price[mask]
59     if mode == "replacement":
60         return float(np.dot(p, result.dispatch.grid[mask] + 0.5 * result.increase[mask] +
            0.5 * result.decrease[mask]))
61     return float(np.dot(p, baseline[mask]) + np.dot(p, 0.5 * result.decrease[mask] + 1.5 *
        result.increase[mask]))
62
63
64 class LinearizationTest(unittest.TestCase):
65     """两种口径下模型目标与按题意直接回代的差异必须 < 1e-7。"""
66
67     def _gap(self, mode: str, seed: int) -> float:
68         load, pv, price, baseline, mask = _case(seed)
69         result = solve_adjustment_dispatch(
70             load, pv, price, baseline, mask,
71             soc_initial=6000.0, soc_terminal=6000.0, settlement_mode=mode,
72         )
73         direct = settlement_cost(baseline, result.dispatch.grid, price, mode)
74         return abs(direct - _model_objective(result, price, baseline, mode))
75
76     def test_replacement_linearization(self):
77         for seed in range(5):
78             self.assertLess(self._gap("replacement", seed), 1e-7, f"seed={seed}")
79
80     def test_additive_linearization(self):
81         for seed in range(5):
82             self.assertLess(self._gap("additive", seed), 1e-7, f"seed={seed}")
83
84     def test_two_modes_are_reoptimized_not_revalued(self):
85         """两种口径应给出各自最优的调度，不能只换算式计费。"""
86         for seed in range(5):
87             load, pv, price, baseline, mask = _case(seed)
88             kwargs = dict(soc_initial=6000.0, soc_terminal=6000.0)
89             replacement = solve_adjustment_dispatch(
90                 load, pv, price, baseline, mask, settlement_mode="replacement", **kwargs
91             )
92             additive = solve_adjustment_dispatch(
93                 load, pv, price, baseline, mask, settlement_mode="additive", **kwargs

```

```

94         )
95         best_replacement = settlement_cost(baseline, replacement.dispatch.grid, price, "
            replacement")
96         cross_replacement = settlement_cost(baseline, additive.dispatch.grid, price, "
            replacement")
97         best_additive = settlement_cost(baseline, additive.dispatch.grid, price, "
            additive")
98         cross_additive = settlement_cost(baseline, replacement.dispatch.grid, price, "
            additive")
99         # 容差取 HiGHS 求解精度量级。
100        tol_r = 1e-6 * max(1.0, abs(best_replacement))
101        tol_a = 1e-6 * max(1.0, abs(best_additive))
102        self.assertLessEqual(best_replacement, cross_replacement + tol_r, f"seed={seed}"
            )
103        self.assertLessEqual(best_additive, cross_additive + tol_a, f"seed={seed}")
104
105
106    class StorageParamTest(unittest.TestCase):
107        def test_energy_limit(self):
108            self.assertAlmostEqual(DEFAULT_STORAGE.energy_limit, 5000.0 / 6.0, places=9)
109
110
111    if __name__ == "__main__":
112        unittest.main()

```

B.30 历史配置回归测试 (test_regression.py)

Listing 30:

```

1  """legacy 结果回归测试（任务书 S11）。
2
3  在 legacy 参数组合下（perfect 负载 / perfect 价格 / 实时反馈 ON / 非对称 PV 风险 /
4  replacement 结算 / daily_cycle 终端 / 不使用典型日先验），五类结果必须复现
5  原工程的既有数值。
6  """
7  from __future__ import annotations
8
9  import json
10 import unittest
11 from pathlib import Path
12
13 from ..solve.io_data import read_attachment4
14 from ..solve.q1 import run_q1
15 from ..solve.q2 import run_q2
16 from ..solve.q3 import run_q3
17

```

```

18 FIXTURE = Path(__file__).with_name("legacy_metrics.json")
19 LEGACY = json.loads(FIXTURE.read_text(encoding="utf-8"))
20
21 # legacy 总费用尺度下的数值容差 (元)
22 ABS_TOL = 1e-3
23 REL_TOL = 1e-9
24
25
26 def _close(test: unittest.TestCase, label: str, new: float, old: float, abs_tol: float =
    ABS_TOL) -> None:
27     tolerance = max(abs_tol, REL_TOL * abs(old))
28     test.assertLess(
29         abs(new - old),
30         tolerance,
31         f"{label} 未复现 legacy 数值: new={new!r} legacy={old!r} diff={new - old!r}",
32     )
33
34
35 class LegacyRegressionTest(unittest.TestCase):
36     @classmethod
37     def setUpClass(cls) -> None:
38         cls.prices = read_attachment4().price
39
40     def test_q1_unchanged(self):
41         metrics = run_q1()["metrics"]
42         _close(self, "Q1 费用", metrics["cost_yuan"], LEGACY["q1"]["cost_yuan"])
43         _close(self, "Q1 购电量", metrics["grid_energy_kwh"], LEGACY["q1"]["grid_energy_kwh"
44             ])
45         _close(self, "Q1 充电量", metrics["charge_energy_kwh"], LEGACY["q1"][""
46             charge_energy_kwh"])
47
48     def test_q2_legacy_benchmark(self):
49         metrics = run_q2(
50             max_days=365,
51             write_outputs=False,
52             load_mode="perfect",
53             pv_risk_mode="asymmetric",
54             realtime_feedback=True,
55             price_mode="perfect",
56             terminal_policy="daily_cycle",
57             use_typical_prior=False,
58         )["metrics"]
59         _close(self, "Q2 总费用", metrics["total_cost_yuan"], LEGACY["q2"]["total_cost_yuan"
60             ])
61         _close(self, "Q2 计划费用", metrics["plan_purchase_cost_yuan"], LEGACY["q2"][""
62             plan_purchase_cost_yuan"])
63         _close(self, "Q2 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q2"][""

```

```

        emergency_energy_kwh"]])
60     _close(self, "Q2 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q2"][""
        curtailment_energy_kwh"])
61
62     def test_q3_legacy_benchmark(self):
63         metrics = run_q3(
64             max_days=365,
65             write_outputs=False,
66             downscale="linear",
67             calibrate=True,
68             update_nodes=(0, 36, 72, 108),
69             settlement_mode="replacement",
70             load_mode="perfect",
71             price_mode="perfect",
72             realtime_feedback=True,
73             terminal_policy="daily_cycle",
74             use_typical_prior=False,
75         )["metrics"]
76     _close(self, "Q3 总费用", metrics["total_cost_yuan"], LEGACY["q3"]["total_cost_yuan"
        ])
77     _close(self, "Q3 基准计划费用", metrics["baseline_purchase_cost_yuan"], LEGACY["q3"
        ]["baseline_purchase_cost_yuan"])
78     _close(self, "Q3 紧急电量", metrics["emergency_energy_kwh"], LEGACY["q3"][""
        emergency_energy_kwh"])
79     # HiGHS 在退化最优面上的分配可产生毫瓦时量级差异，费用与可行性均不受影响。
80     _close(self, "Q3 弃光量", metrics["curtailment_energy_kwh"], LEGACY["q3"][""
        curtailment_energy_kwh"], abs_tol=1e-2)
81
82     def test_q4_legacy_benchmark(self):
83         metrics2 = run_q2(
84             max_days=365,
85             write_outputs=False,
86             price_matrix=self.prices,
87             load_mode="perfect",
88             pv_risk_mode="asymmetric",
89             realtime_feedback=True,
90             price_mode="perfect",
91             use_typical_prior=False,
92         )["metrics"]
93     _close(self, "Q4-2 总费用", metrics2["total_cost_yuan"], LEGACY["q4-2"][""
        total_cost_yuan"])
94     metrics3 = run_q3(
95         max_days=365,
96         write_outputs=False,
97         price_matrix=self.prices,
98         update_nodes=(0, 36, 72, 108),
99         settlement_mode="replacement",

```

```
100         load_mode="perfect",
101         price_mode="perfect",
102         realtime_feedback=True,
103         use_typical_prior=False,
104     )["metrics"]
105     _close(self, "Q4-3 总费用", metrics3["total_cost_yuan"], LEGACY["q4-3"]["
        total_cost_yuan"])
106
107
108 if __name__ == "__main__":
109     unittest.main()
```